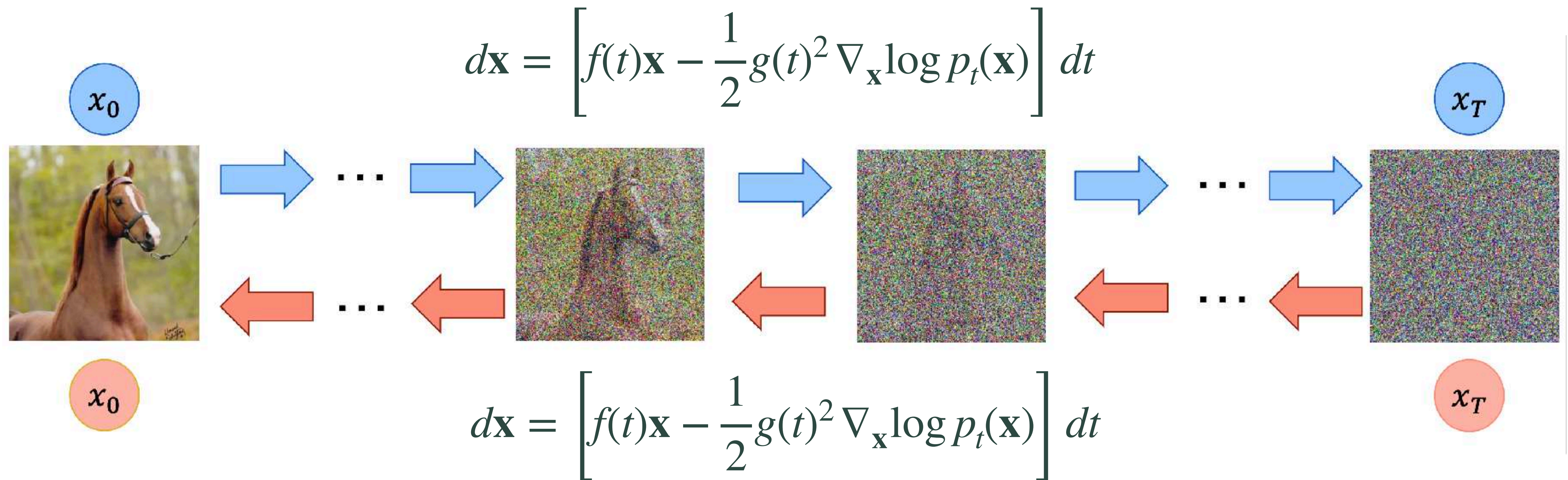


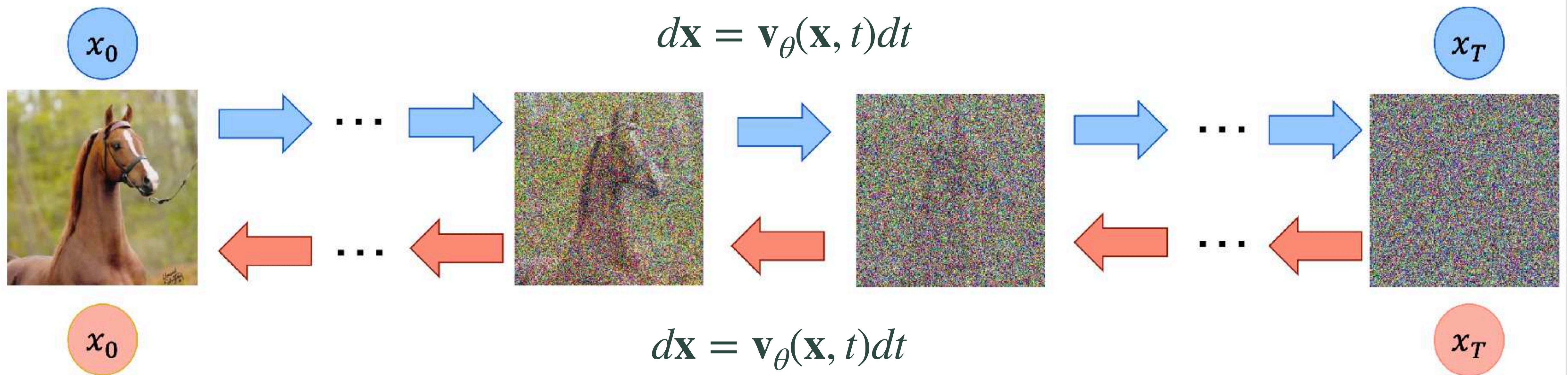
ODE-FREE FEW-STEP MODELS!



Probability Flow ODE



Flow Matching



Numerical solvers offer 15-20 NFE sampling. What about 1-4 steps?

Few-step Models

Flow Map Models (ODE Integrators) 

Knowledge distillation

Consistency Models

Consistency Trajectory Models

Shortcut Models

Mean Flow

ODE-free Few-step Models 

Distribution Matching Distillation

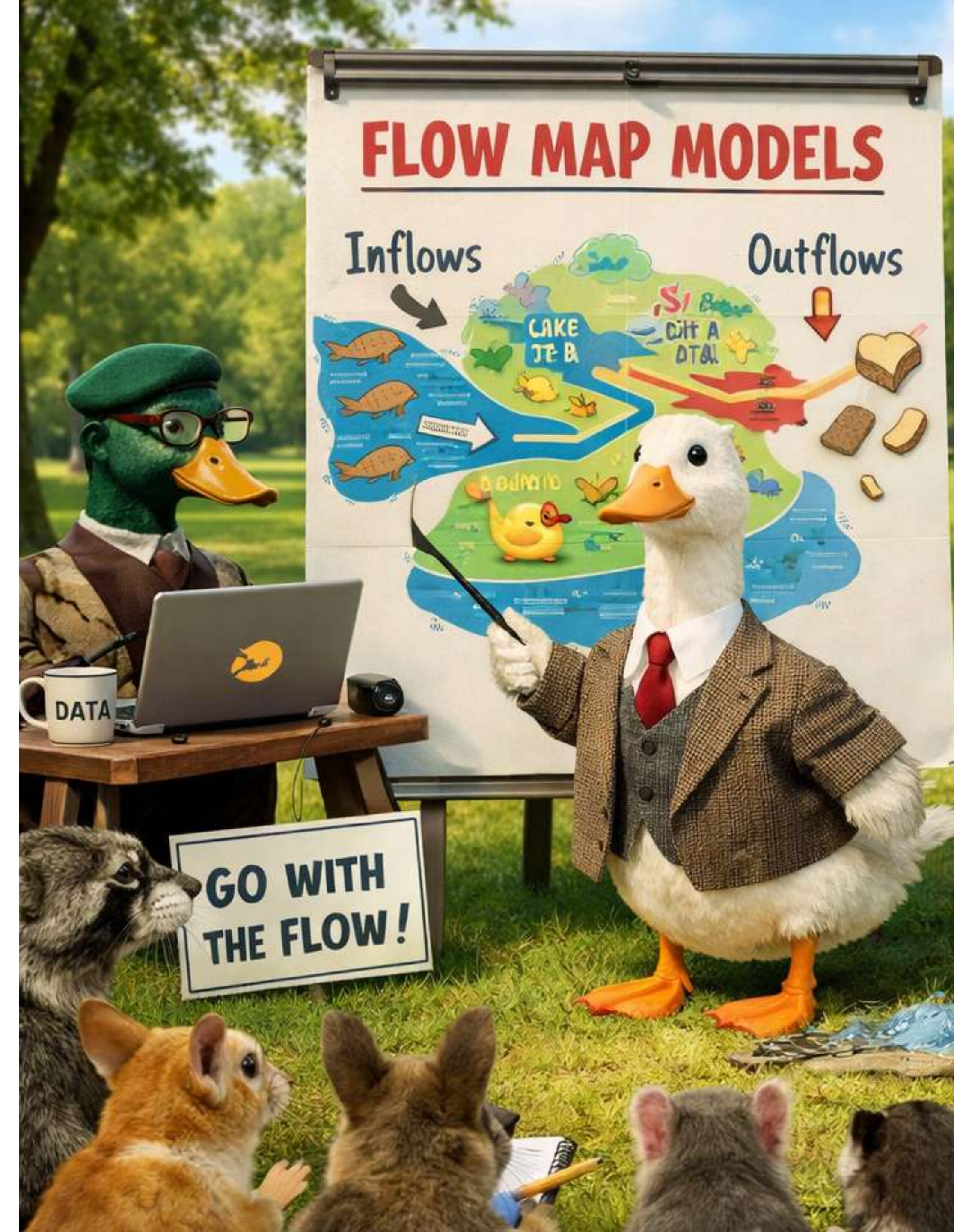
Adversarial Diffusion Distillation

MMD Distillation

Drifting Models

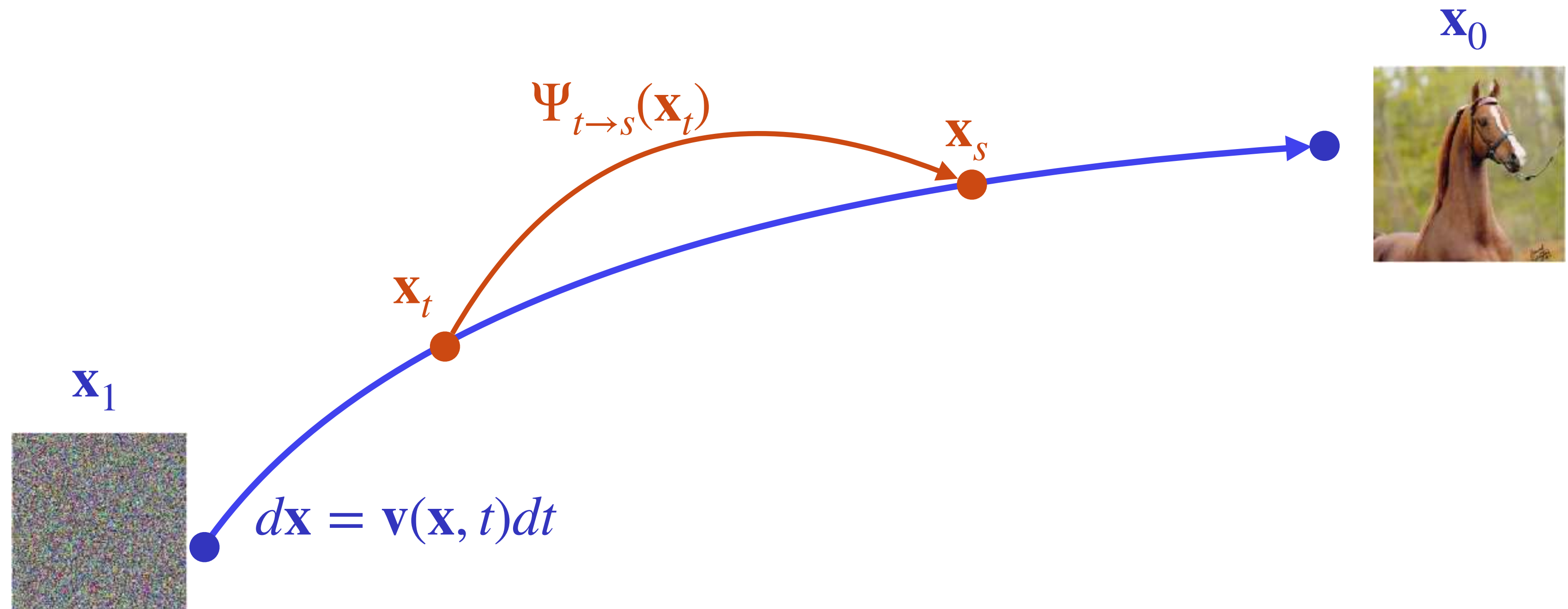


Recap of Flow Map Models



Flow-Map Formulation

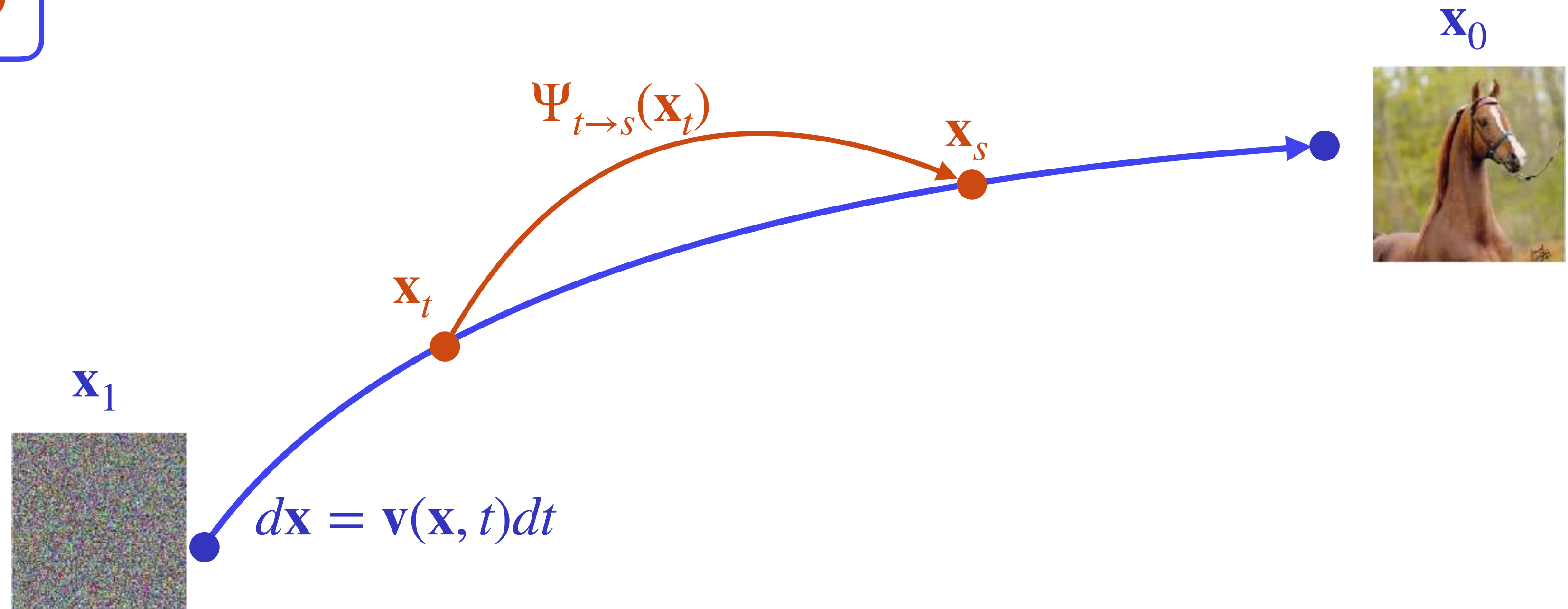
Flow Map: $\Psi_{t \rightarrow s}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^s \mathbf{v}(\mathbf{x}, u) du \rightarrow$ Oracle integrator for $d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt$



Flow-Map Formulation

Flow Map: $\Psi_{t \rightarrow s}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^s \mathbf{v}(\mathbf{x}, u) du \rightarrow$ Oracle integrator for $d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt$

Goal: train $F_\theta(\mathbf{x}_t, t, s) \approx \Psi_{t \rightarrow s}(\mathbf{x}_t)$

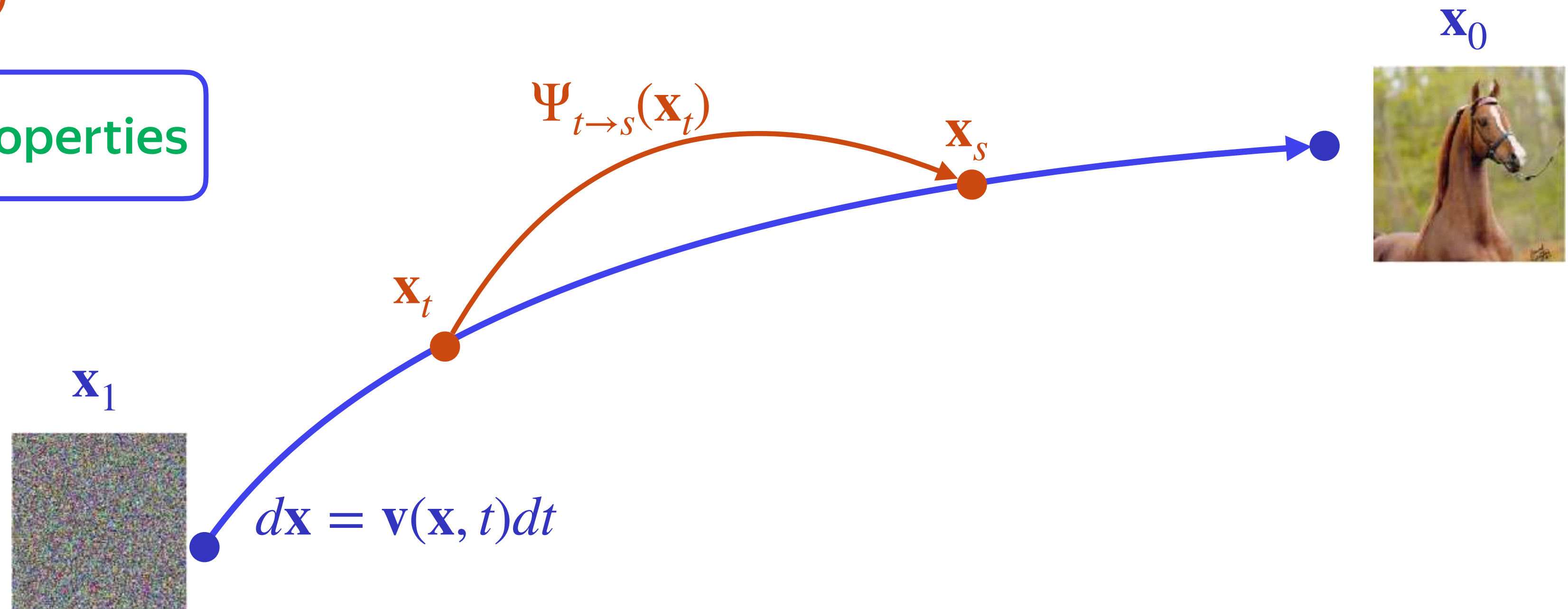


Flow-Map Formulation

Flow Map: $\Psi_{t \rightarrow s}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^s \mathbf{v}(\mathbf{x}, u) du \rightarrow$ Oracle integrator for $d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt$

Goal: train $F_\theta(\mathbf{x}_t, t, s) \approx \Psi_{t \rightarrow s}(\mathbf{x}_t)$

How? Design loss via integral properties



Knowledge Distillation

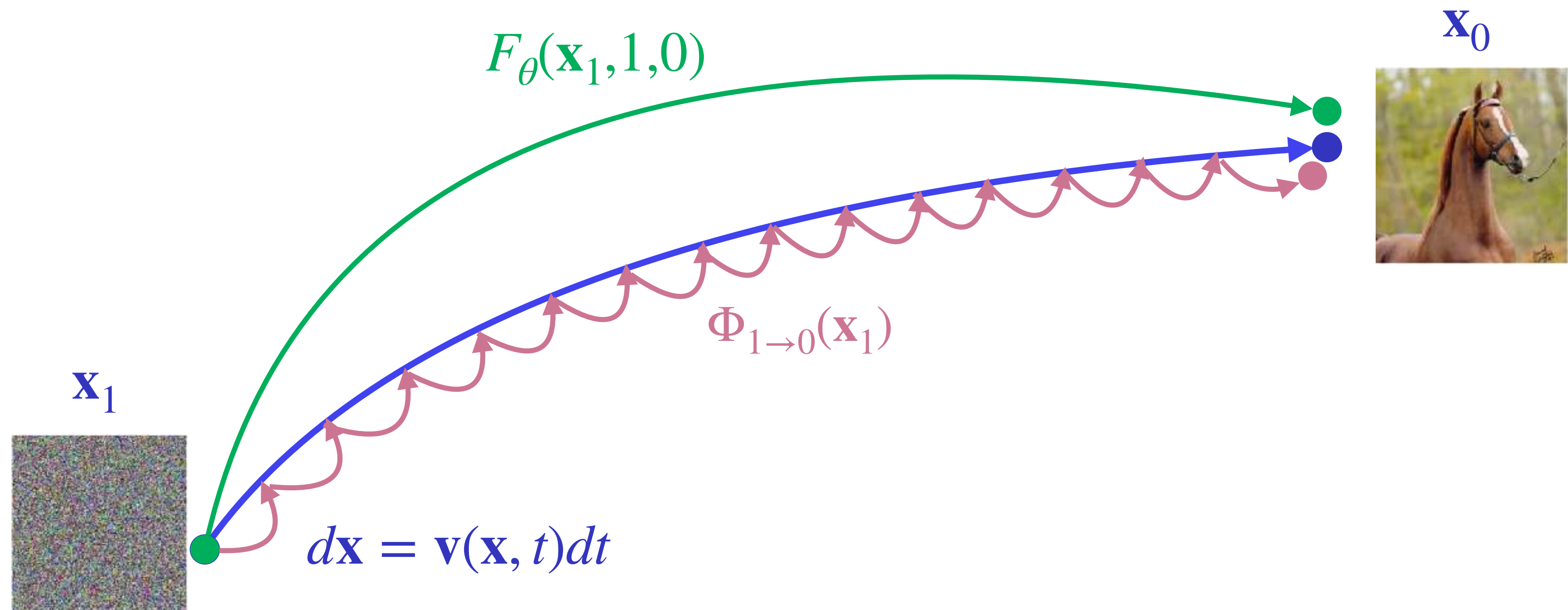
Solve entire ODE: $\Psi_{1 \rightarrow 0}(\mathbf{x}_t) = \mathbf{x}_1 + \int_1^0 \mathbf{v}(\mathbf{x}, u) du$

Goal: train $F_\theta(\mathbf{x}_1, 1, 0) \approx \Psi_{t \rightarrow 0}(\mathbf{x}_1)$

How? $\|F_\theta(\mathbf{x}_1, 1, 0) - \Phi_{1 \rightarrow 0}(\mathbf{x}_1)\|_2^2$
Approximate teacher solutions

$\Phi_{1 \rightarrow 0}(\mathbf{x}_1)$ — “oracle” DM

Sample with the pretrained DM



Consistency Models

Fix a zero boundary: $\Psi_{t \rightarrow 0}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^0 \mathbf{v}(\mathbf{x}, u) du$

Goal: train $F_\theta(\mathbf{x}_t, t, 0) \approx \Psi_{t \rightarrow 0}(\mathbf{x}_t)$

How? $\|F_\theta(\mathbf{x}_t, t, 0) - \text{sg}(F_\theta(\mathbf{x}_r, r, 0))\|_2^2$

*sg(·) — stopgrad

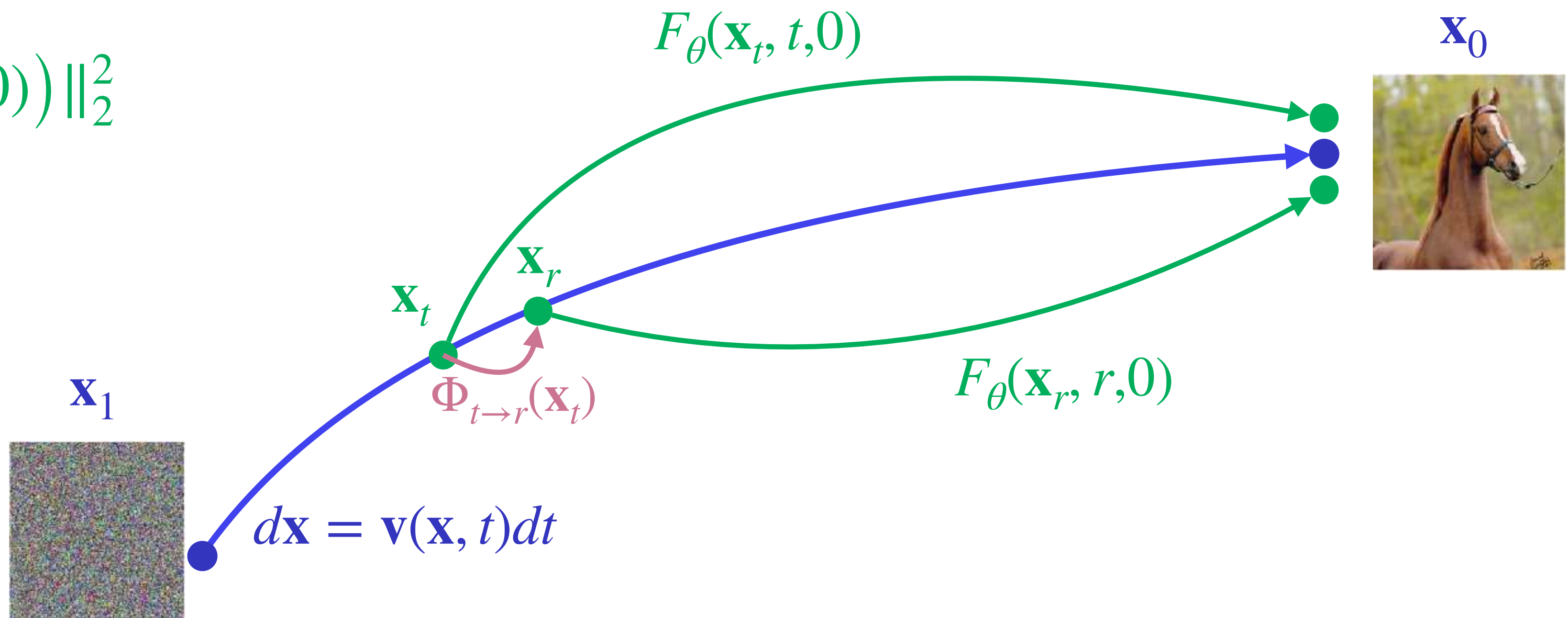
Self-consistency

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — “oracle” DM

$[t, r]$ is small



one step using teacher or analytical score



Multi-boundary Consistency Models

K boundaries $s = \{s_0, \dots, s_K\}$: $\Psi_{t \rightarrow s_k}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^{s_k} \mathbf{v}(\mathbf{x}, u) du$

Goal: train $F_\theta(\mathbf{x}_t, t, s_k) \approx \Psi_{t \rightarrow s_k}(\mathbf{x}_t)$

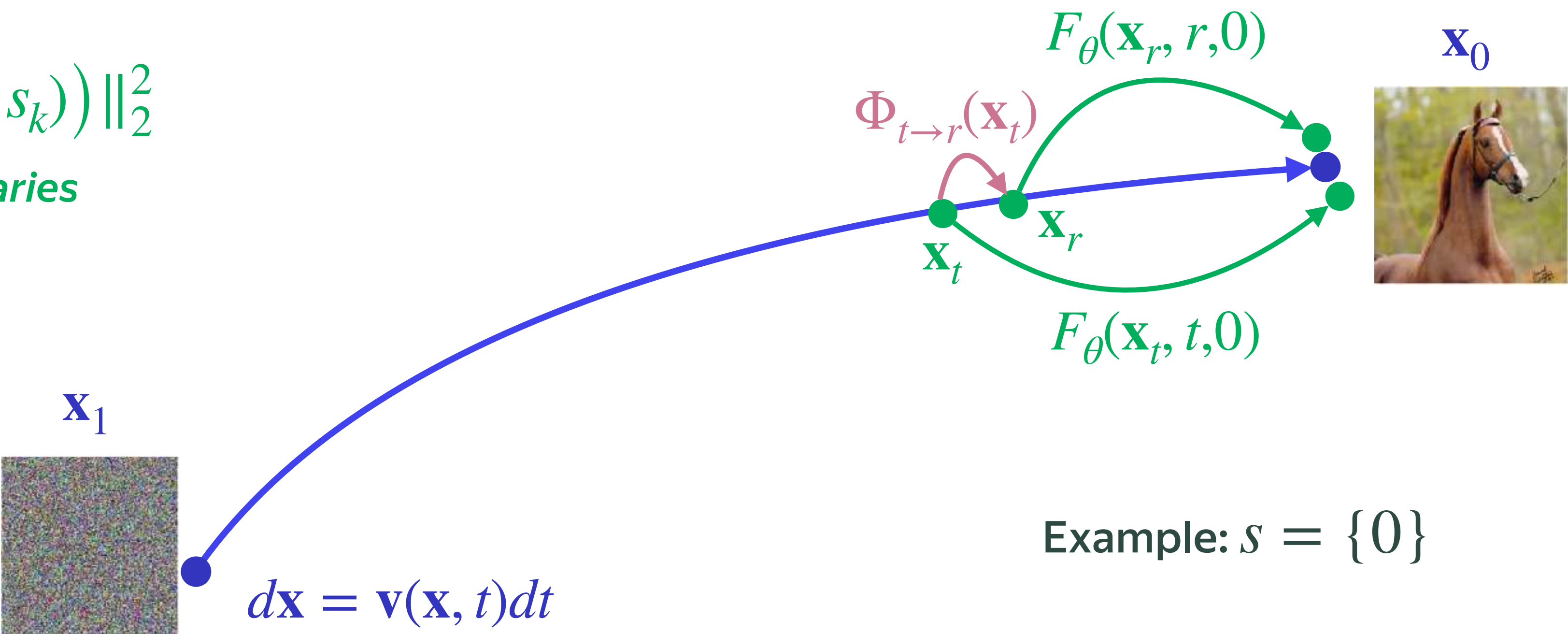
How? $\|F_\theta(\mathbf{x}_t, t, s_k) - sg(F_\theta(\mathbf{x}_r, r, s_k))\|_2^2$
Self-consistency for K boundaries

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — “oracle” DM

$[t, r]$ is small



one step using teacher or analytical score



Multi-boundary Consistency Models

K boundaries $s = \{s_0, \dots, s_K\}$: $\Psi_{t \rightarrow s_k}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^{s_k} \mathbf{v}(\mathbf{x}, u) du$

Goal: train $F_\theta(\mathbf{x}_t, t, s_k) \approx \Psi_{t \rightarrow s_k}(\mathbf{x}_t)$

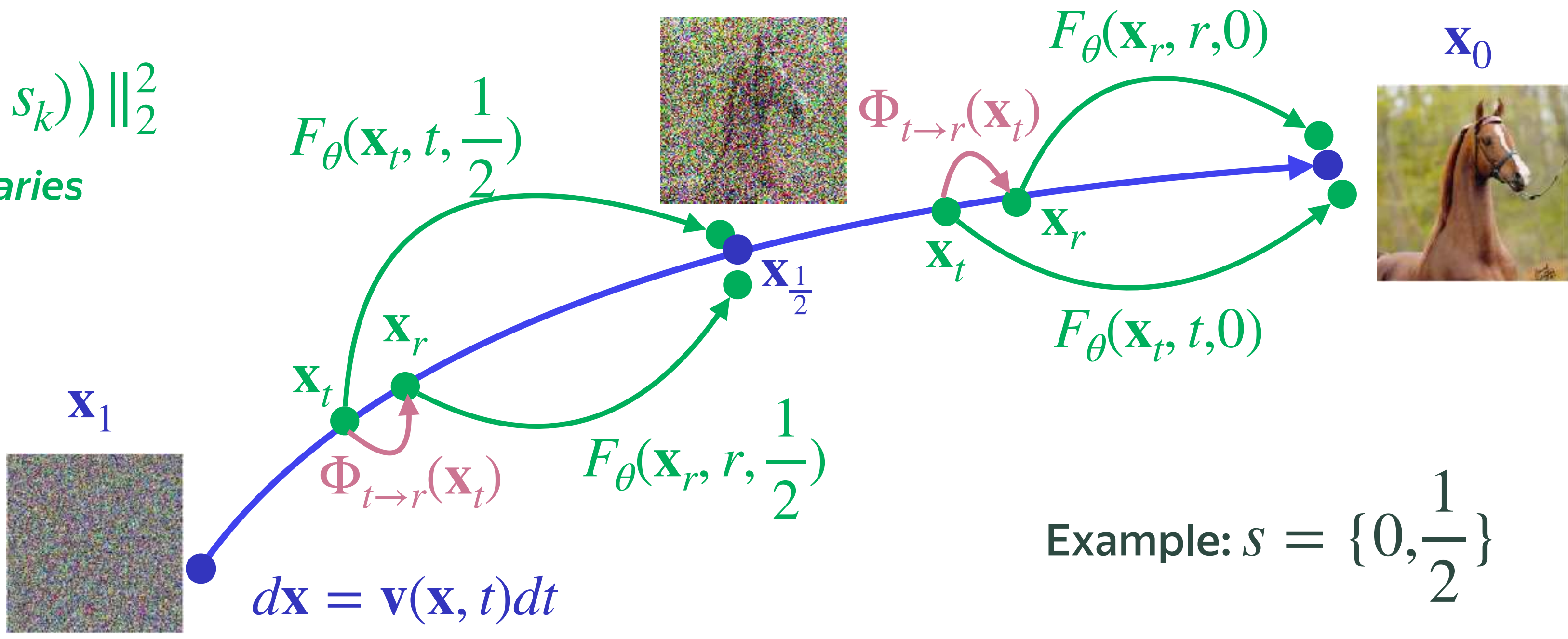
How? $\|F_\theta(\mathbf{x}_t, t, s_k) - sg(F_\theta(\mathbf{x}_r, r, s_k))\|_2^2$
Self-consistency for K boundaries

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — “oracle” DM

$[t, r]$ is small



one step using teacher or analytical score



Example: $s = \{0, \frac{1}{2}\}$

Consistency Trajectory Models

Continuous $s \in (1,0]$: $\Psi_{t \rightarrow s}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^s \mathbf{v}(\mathbf{x}, u) du$

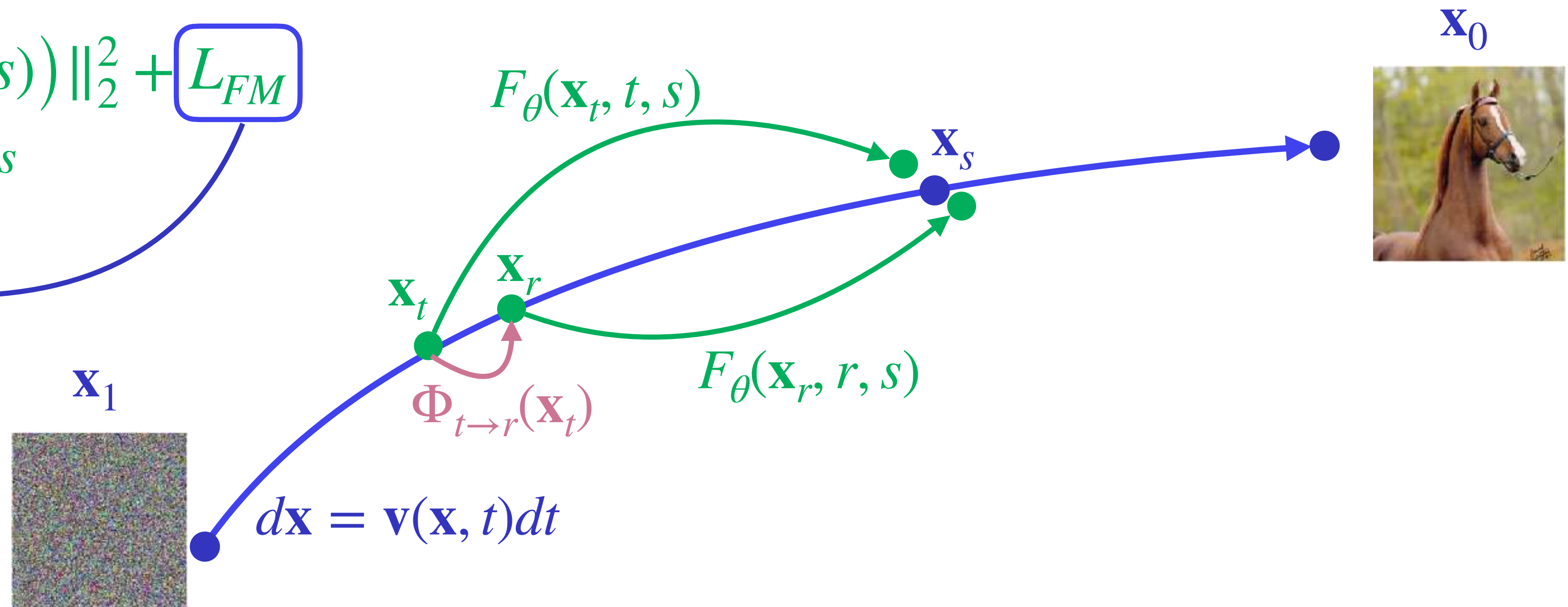
Goal: train $F_\theta(\mathbf{x}_t, t, s) \approx \Psi_{t \rightarrow s}(\mathbf{x}_t)$

How? $\|F_\theta(\mathbf{x}_t, t, s) - sg(F_\theta(\mathbf{x}_r, r, s))\|_2^2 + L_{FM}$
Self-consistency for arbitrary s

$$F_\theta(\mathbf{x}_t, t, s) = \mathbf{x}_t + (s - t)\mathbf{v}_\theta(\mathbf{x}_t, t)$$

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — “oracle” DM

one step using teacher or $\mathbf{v}_\theta(\mathbf{x}_t, t)$

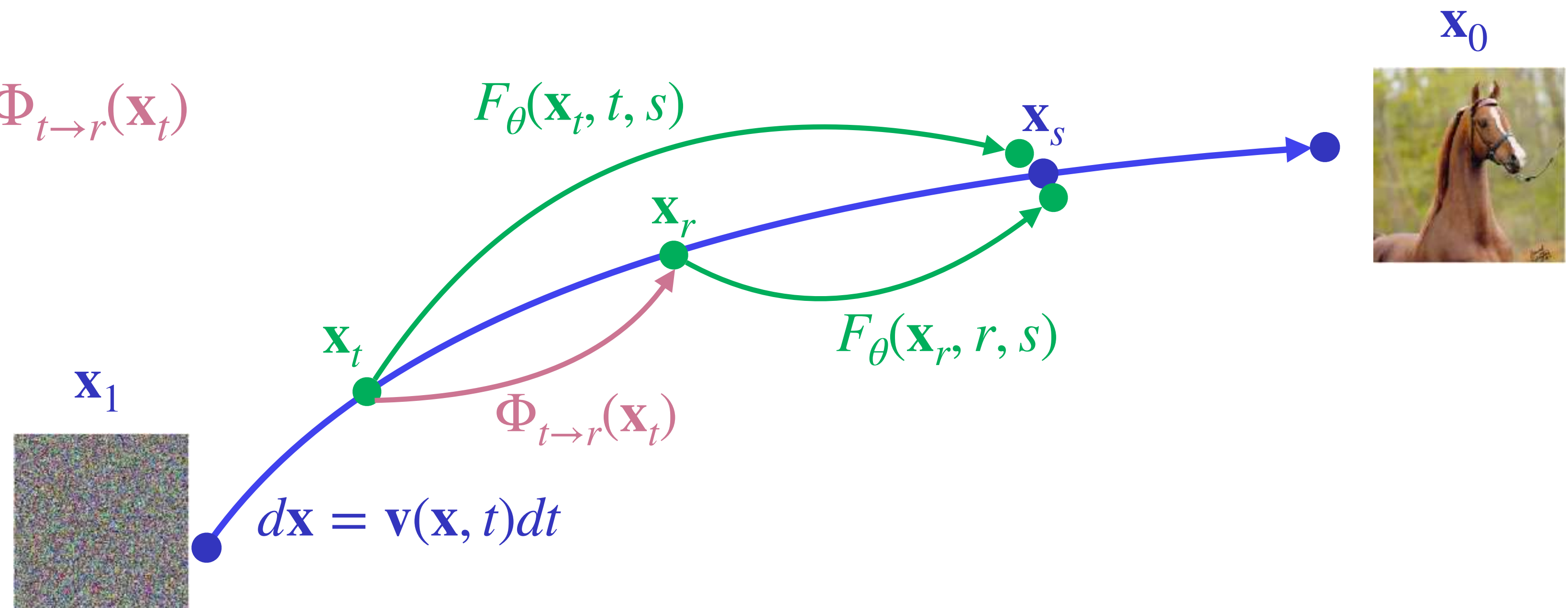


General Consistency Trajectory Models

$[t, r]$ can be arbitrary: small or large



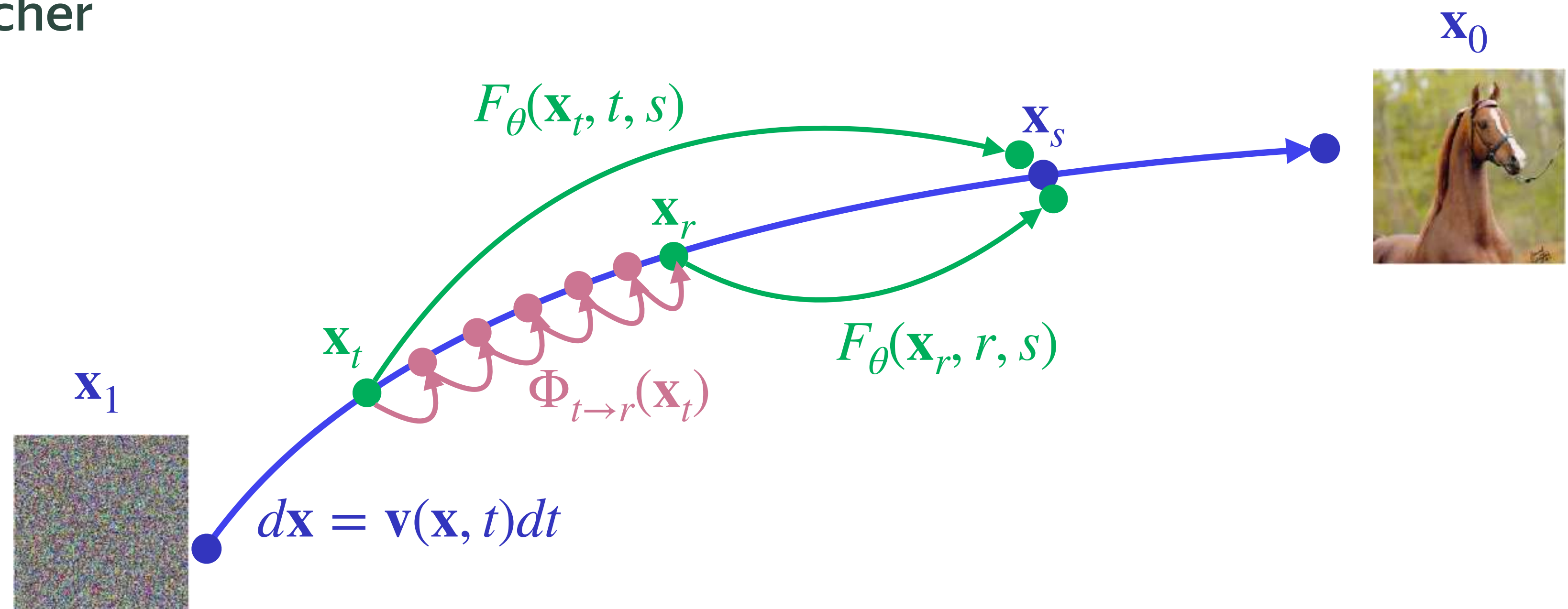
There are general choices for $\Phi_{t \rightarrow r}(\mathbf{x}_t)$



General Consistency Trajectory Models

“Oracle” diffusion choices

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — multi-step teacher



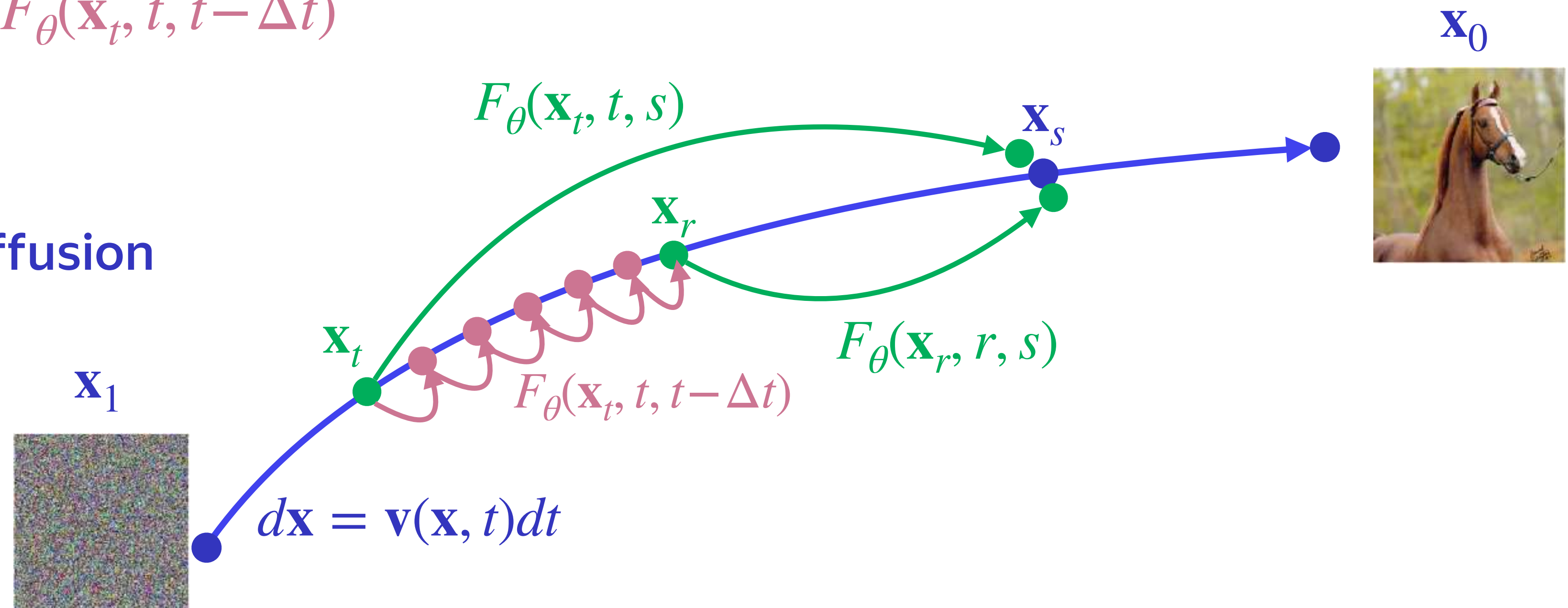
General Consistency Trajectory Models

“Oracle” diffusion choices

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — multi-step flow-map $F_{\theta}(\mathbf{x}_t, t, t - \Delta t)$



No need in pretrained diffusion



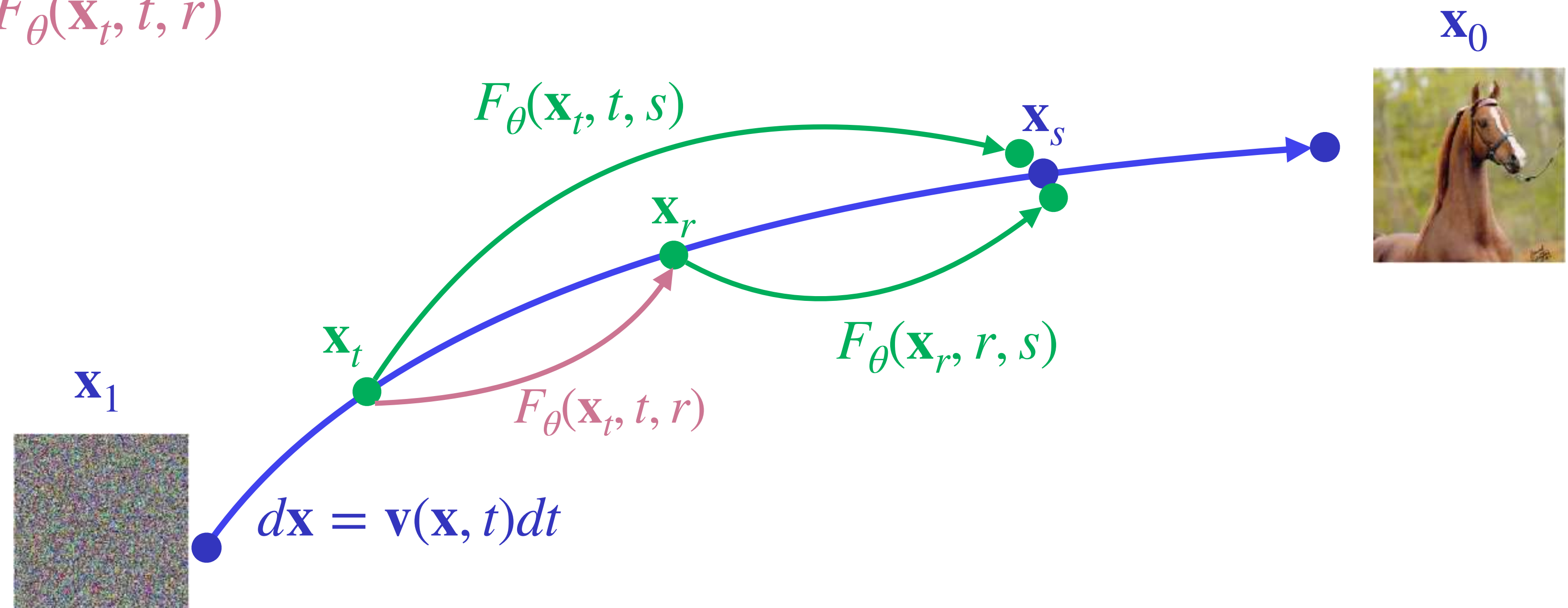
General Consistency Trajectory Models

“Oracle” diffusion choices

$\Phi_{t \rightarrow r}(\mathbf{x}_t)$ — one-step flow-map $F_{\theta}(\mathbf{x}_t, t, r)$



Shortcut Models



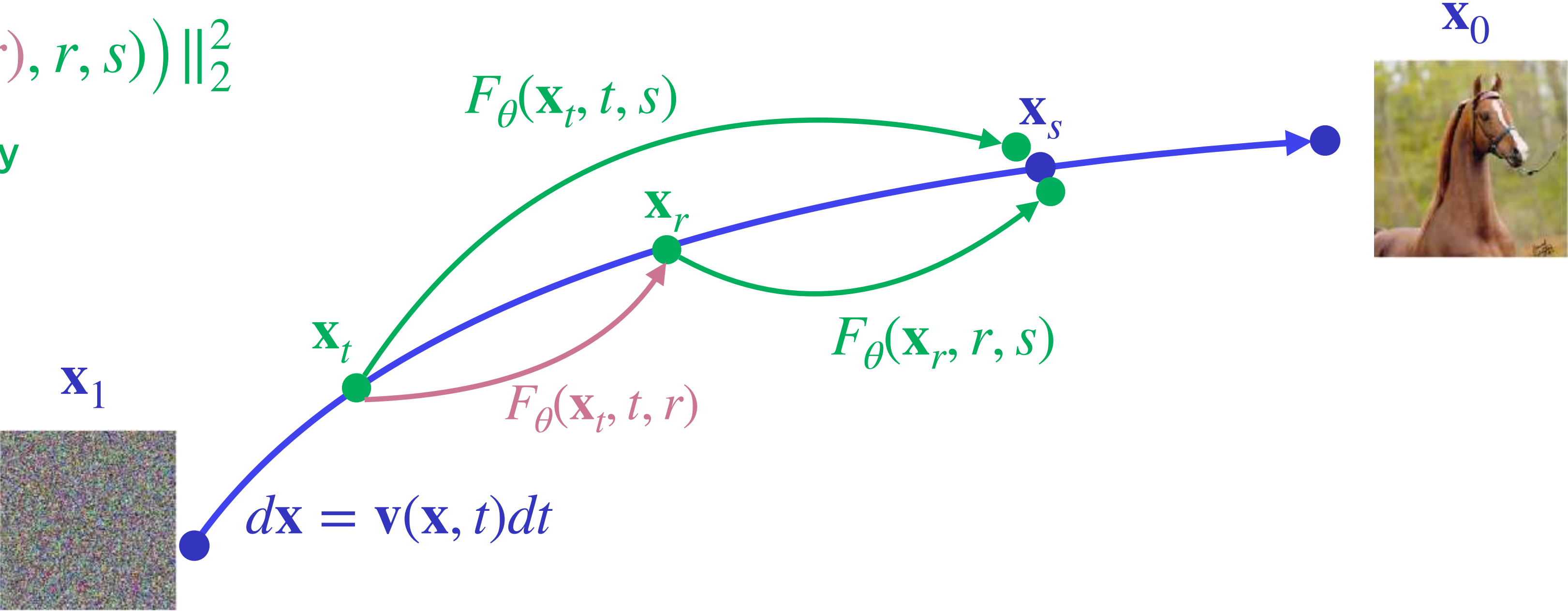
Shortcut Models

Interval additivity property

$$\int_t^s \mathbf{v}(\mathbf{x}, u) du = \int_t^r \mathbf{v}(\mathbf{x}, u) du + \int_r^s \mathbf{v}(\mathbf{x}, u) du$$

$$\|F_{\theta}(\mathbf{x}_t, t, s) - sg(F_{\theta}(F_{\theta}(\mathbf{x}_t, t, r), r, s))\|_2^2$$

Interval additivity property



Mean Flow

$$F_{\theta}(\mathbf{x}_t, t, s) \approx \frac{1}{s - t} \int_t^s \mathbf{v}(\mathbf{x}, u) du$$

Mean Flow

$$F_{\theta}(\mathbf{x}_t, t, s) \approx \frac{1}{s - t} \int_t^s \mathbf{v}(\mathbf{x}, u) du \Rightarrow \Psi_{t \rightarrow s}(\mathbf{x}_t) \approx \mathbf{x}_t + (s - t)F_{\theta}(\mathbf{x}_t, t, s)$$

Mean Flow

$$F_{\theta}(\mathbf{x}_t, t, s) \approx \frac{1}{s-t} \int_t^s \mathbf{v}(\mathbf{x}, u) du \Rightarrow \Psi_{t \rightarrow s}(\mathbf{x}_t) \approx \mathbf{x}_t + (s-t)F_{\theta}(\mathbf{x}_t, t, s)$$

$$(s-t)F_{\theta}(\mathbf{x}_t, t, s) = \int_t^s \mathbf{v}(\mathbf{x}, u) du$$



Mean Flow Identity

$$F_{\theta}(\mathbf{x}_t, t, s) = \underbrace{\mathbf{v}(\mathbf{x}_t, t)}_{\text{Inst. velocity}} + (s-t) \underbrace{\frac{d}{dt} F_{\theta}(\mathbf{x}_t, t, s)}_{\text{Time derivative}}$$

Avg. velocity

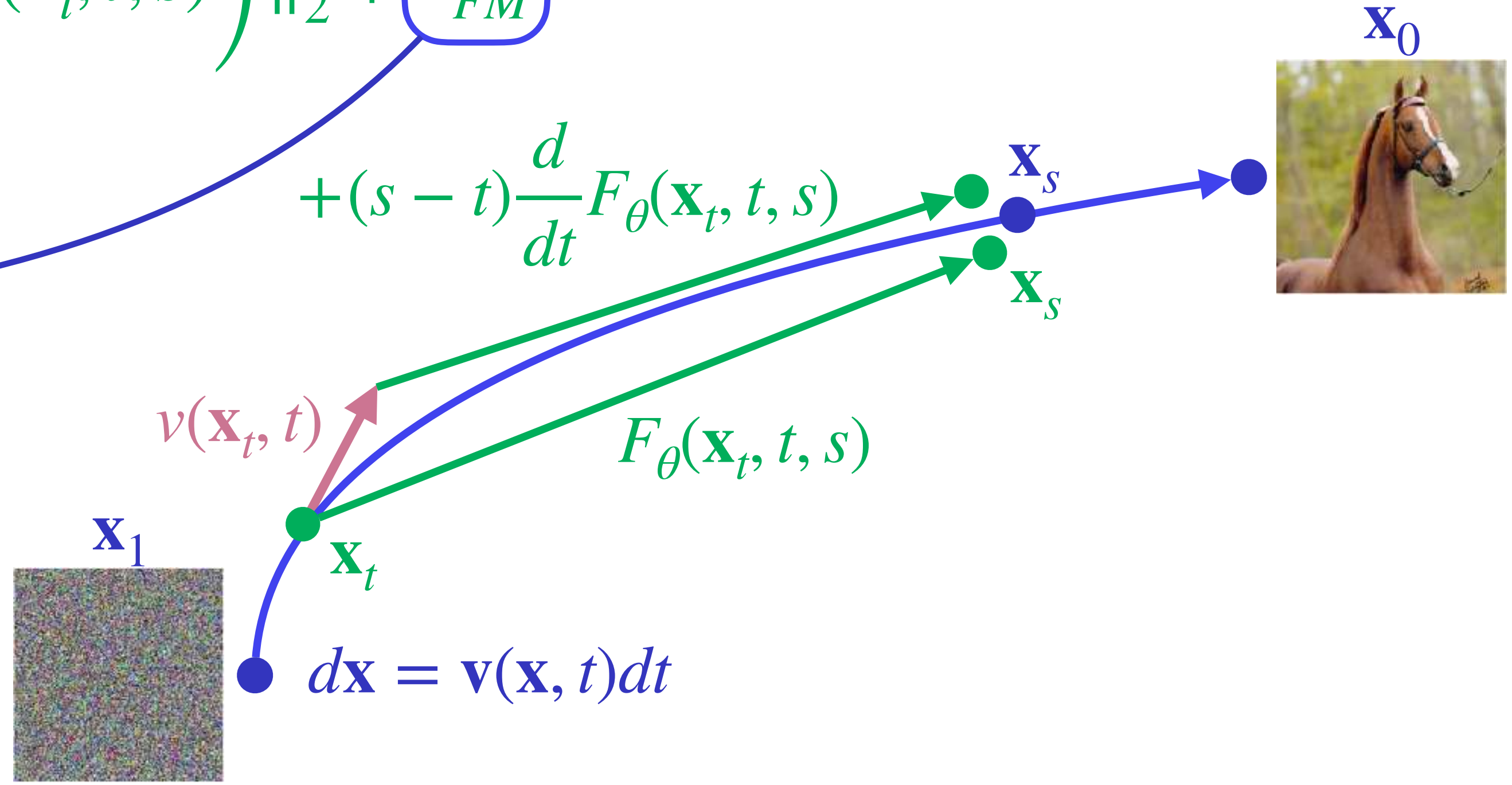
Mean Flow

Goal: train $F_{\theta}(\mathbf{x}_t, t, s) \approx \frac{1}{s - t} \int_t^s \mathbf{v}(\mathbf{x}, u) du$

How? $\|F_{\theta}(\mathbf{x}_t, t, s) - sg\left(\mathbf{v}(\mathbf{x}_t, t) + (s - t)\frac{d}{dt}F_{\theta}(\mathbf{x}_t, t, s)\right)\|_2^2 + L_{FM}$
Mean Flow Identity

$F_{\theta}(\mathbf{x}_t, t, t) = \mathbf{v}_{\theta}(\mathbf{x}_t, t)$

$\mathbf{v}(\mathbf{x}_t, t) = \dot{\mathbf{x}}_t$ or DM prediction $\mathbf{v}_{\phi}(\mathbf{x}_t, t)$



Takeaways

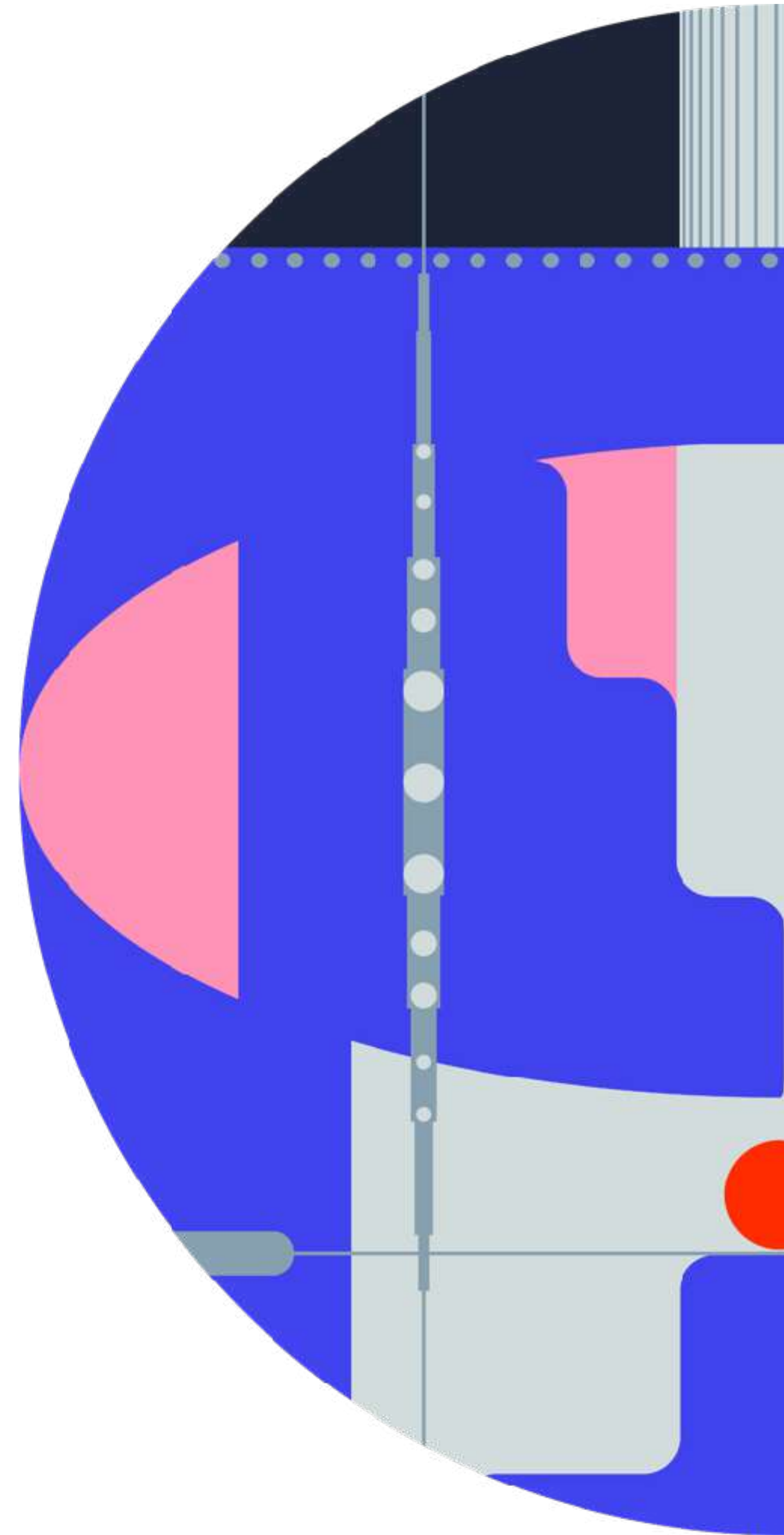
KD — the most naïve approach. Still useful for regularization, or for warming up few-step models

Multi-boundary (-step) CM — a strong and practical method, particularly effective for 6–8 steps.

CTM / Shortcut Model — elegant and productive ideas. I did not see large-scale applications

Continuous CM / MeanFlow — elegant and promising formulations, but too complex in practice

Preliminaries



Notation

Generator $G_\theta(\mathbf{z})$ produces $p_{fake}(\mathbf{x})$, where $\mathbf{z} \sim \mathcal{N}(0, I)$

Data distribution $p_{real}(\mathbf{x})$

Forward process: $q(\mathbf{x}_t | \mathbf{x}) = \mathcal{N}(\alpha_t \mathbf{x}, \sigma_t^2 I)$

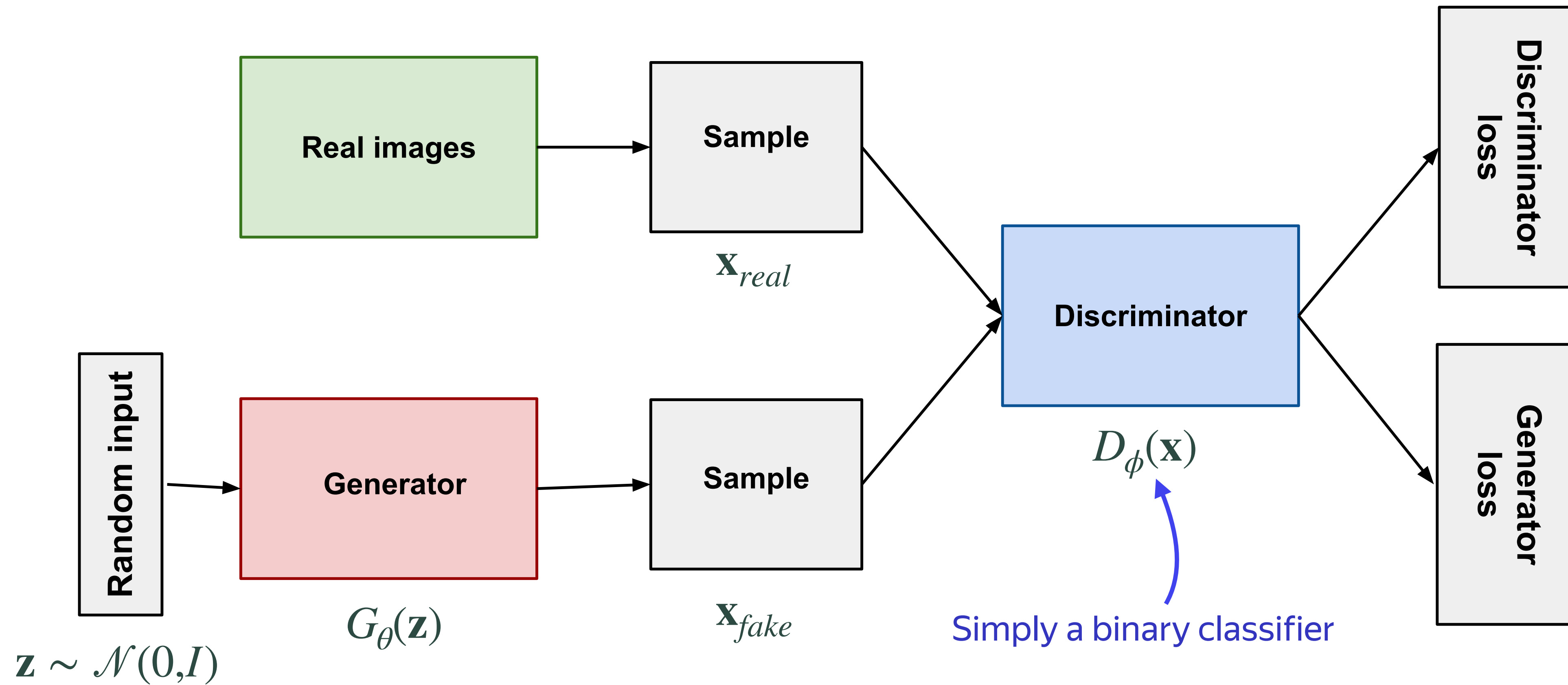
Pretrained diffusion model (DM) $\epsilon_\psi(\mathbf{x}_t, t)$

Tweedie's formula

$$\nabla_{\mathbf{x}_t} \log p_{real}(\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = \mathbb{E} \left[\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) | \mathbf{x}_t \right] \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) = -\frac{\mathbf{x}_t - \alpha_t \mathbf{x}}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$$

Recap of GANs

Generator $G_{\theta}(\mathbf{z}) \rightarrow \mathbf{x}$ tries to fool the discriminator $D_{\phi}(\mathbf{x})$



Recap of GANs

Standard Min-Max loss

Discriminator loss: $\max_{\phi} \left[\mathbb{E}_{\mathbf{x} \sim p_{real}} \log D_{\phi}(\mathbf{x}) + \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} \log(1 - D_{\phi}(G_{\theta}(\mathbf{z}))) \right]$

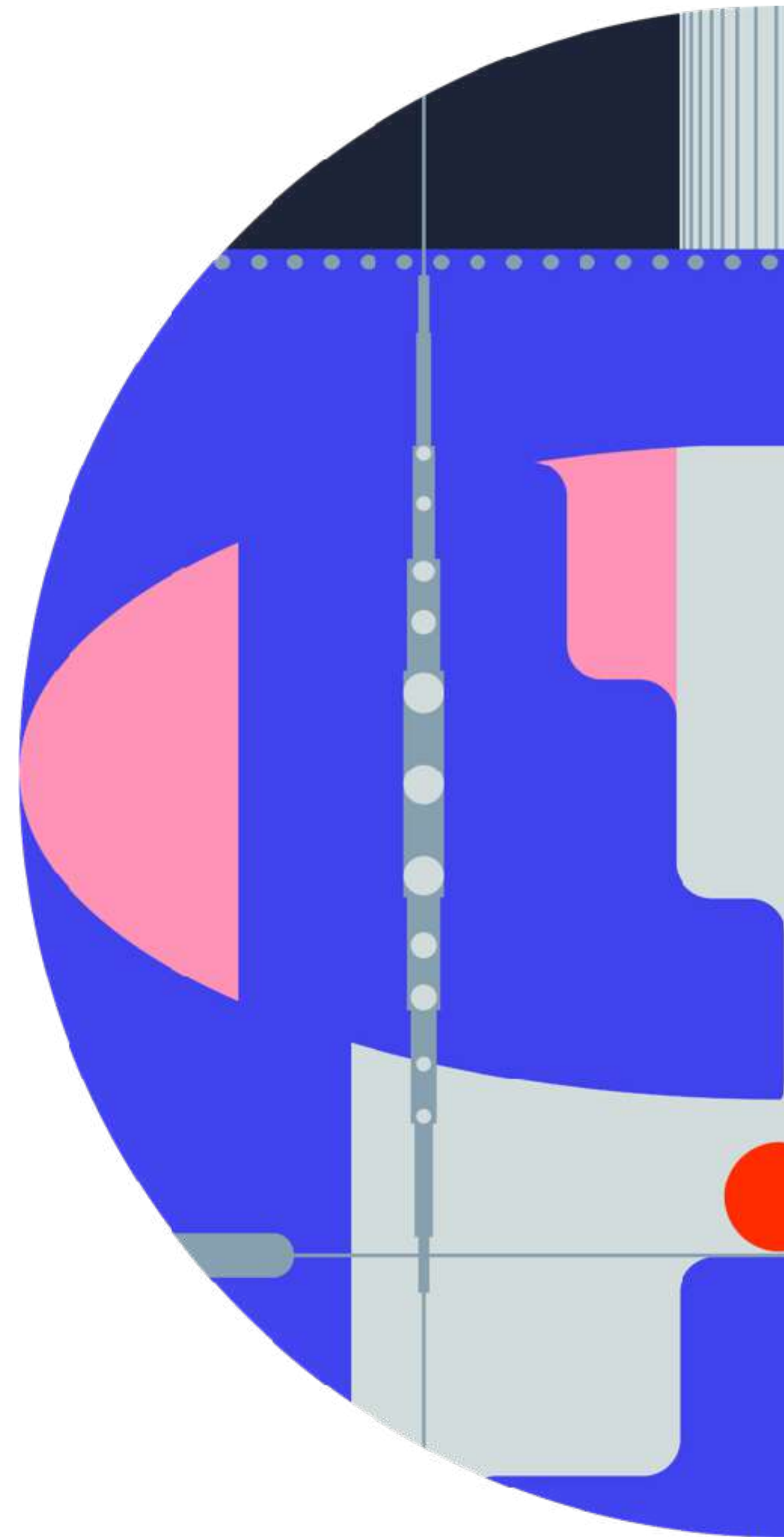
Generator loss: $\min_{\theta} \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} \log(1 - D_{\phi}(G_{\theta}(\mathbf{z})))$

Non-saturating generator loss: $\min_{\theta} \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} -\log D_{\phi}(G_{\theta}(\mathbf{z})))$

Recap of GANs

GANs are powerful **single step** generative models
but **struggle to achieve s.o.t.a. performance w/o some help**, e.g., **diffusion**

Distribution Matching Distillation



Training objective

Let's consider reverse KL objective

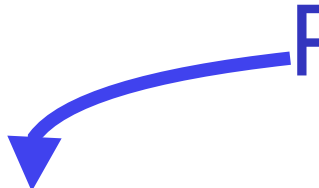
$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{p_{fake}(\mathbf{x})} \left[\log p_{fake}(\mathbf{x}) - \log p_{real}(\mathbf{x}) \right]$$

Training objective

Let's consider reverse KL objective

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{p_{fake}(\mathbf{x})} \left[\log p_{fake}(\mathbf{x}) - \log p_{real}(\mathbf{x}) \right] = \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} \left[\log p_{fake}(\underline{G_{\theta}(\mathbf{z})}) - \log p_{real}(\underline{G_{\theta}(\mathbf{z})}) \right]$$

Reparameterization trick



$\log p_{fake}(\mathbf{x})$ and $\log p_{real}(\mathbf{x})$ are unavailable

Training objective

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log p_{fake}(G_{\theta}(\mathbf{z})) - \log p_{real}(G_{\theta}(\mathbf{z})) \right]$$

$$\nabla_{\theta} D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\nabla_{\theta} \log p_{fake}(G_{\theta}(\mathbf{z})) - \nabla_{\theta} \log p_{real}(G_{\theta}(\mathbf{z})) \right] =$$

Training objective

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log p_{fake}(G_{\theta}(\mathbf{z})) - \log p_{real}(G_{\theta}(\mathbf{z})) \right]$$

$$\nabla_{\theta} D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\nabla_{\theta} \log p_{fake}(G_{\theta}(\mathbf{z})) - \nabla_{\theta} \log p_{real}(G_{\theta}(\mathbf{z})) \right] = \text{Chain rule}$$

$$= \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\underbrace{(\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x}))}_{\text{blue}} - \underbrace{\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})}_{\text{blue}} \right] \nabla_{\theta} G_{\theta}(\mathbf{z}), \quad \text{where } \mathbf{x} = G_{\theta}(\mathbf{z})$$

Can we compute $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$?

Training objective

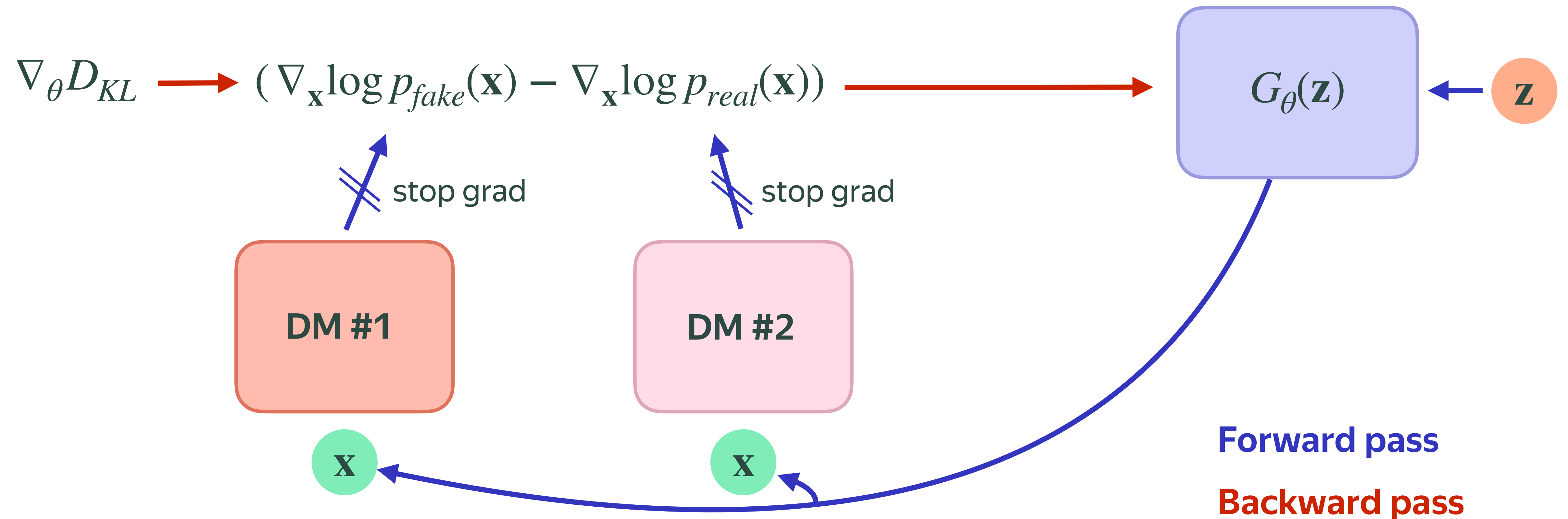
$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log p_{fake}(G_{\theta}(\mathbf{z})) - \log p_{real}(G_{\theta}(\mathbf{z})) \right]$$

$$\nabla_{\theta} D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\nabla_{\theta} \log p_{fake}(G_{\theta}(\mathbf{z})) - \nabla_{\theta} \log p_{real}(G_{\theta}(\mathbf{z})) \right] = \text{Chain rule}$$

$$= \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\underbrace{(\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x}))}_{\text{DM forward}} - \underbrace{\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})}_{\text{DM forward}} \right] \nabla_{\theta} G_{\theta}(\mathbf{z}), \quad \text{where } \mathbf{x} = G_{\theta}(\mathbf{z})$$

Use DM forward passes to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

Generator update



Key idea

Pretrained DM estimates $\nabla_{\mathbf{x}_t} \log p_{real}(\mathbf{x}_t) \approx s_{real}^{\psi}(\mathbf{x}_t, t) = -\frac{\epsilon_{\psi}(\mathbf{x}_t, t)}{\sigma_t}$

For $t = 0$, we have $s_{real}^{\psi}(\mathbf{x}_0, 0) \approx \nabla_{\mathbf{x}_0} \log p_{real}(\mathbf{x}_0) \approx \nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

Key idea

Pretrained DM estimates $\nabla_{\mathbf{x}_t} \log p_{real}(\mathbf{x}_t) \approx s_{real}^{\psi}(\mathbf{x}_t, t) = -\frac{\epsilon_{\psi}(\mathbf{x}_t, t)}{\sigma_t}$

For $t = 0$, we have $s_{real}^{\psi}(\mathbf{x}_0, 0) \approx \nabla_{\mathbf{x}_0} \log p_{real}(\mathbf{x}_0) \approx \nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

Important: DM is pretrained on $p_{real}(\mathbf{x}) \rightarrow$ it can estimate only $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

How to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$?

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

For $t = 0$, we have $s_{fake}^\phi(\mathbf{x}_0, 0) \approx \nabla_{\mathbf{x}_0} \log p_{fake}(\mathbf{x}_0) \approx \nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

For $t = 0$, we have $s_{fake}^\phi(\mathbf{x}_0, 0) \approx \nabla_{\mathbf{x}_0} \log p_{fake}(\mathbf{x}_0) \approx \nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$

Problem: $p_{fake}(\mathbf{x})$ changes after each G_θ update

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

For $t = 0$, we have $s_{fake}^\phi(\mathbf{x}_0, 0) \approx \nabla_{\mathbf{x}_0} \log p_{fake}(\mathbf{x}_0) \approx \nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$

Problem: $p_{fake}(\mathbf{x})$ changes after each G_θ update

Solution: re-train $s_{fake}^\phi(\mathbf{x}_t, t)$ after each G_θ update

For efficiency, a single update per training iteration is okay

Algorithm

1. Update step for G_θ

$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \left[(s_{fake}^\phi(\mathbf{x}, 0) - s_{real}^\psi(\mathbf{x}, 0)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

Algorithm

1. Update step for G_θ

$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \left[(s_{fake}^\phi(\mathbf{x}, 0) - s_{real}^\psi(\mathbf{x}, 0)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

2. Update step for fake DM ϵ_ϕ

Typical DM training step

$$\nabla_\phi \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \|s_{fake}^\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x})\|_2^2 = \nabla_\phi \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \|\epsilon_\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \epsilon\|_2^2$$

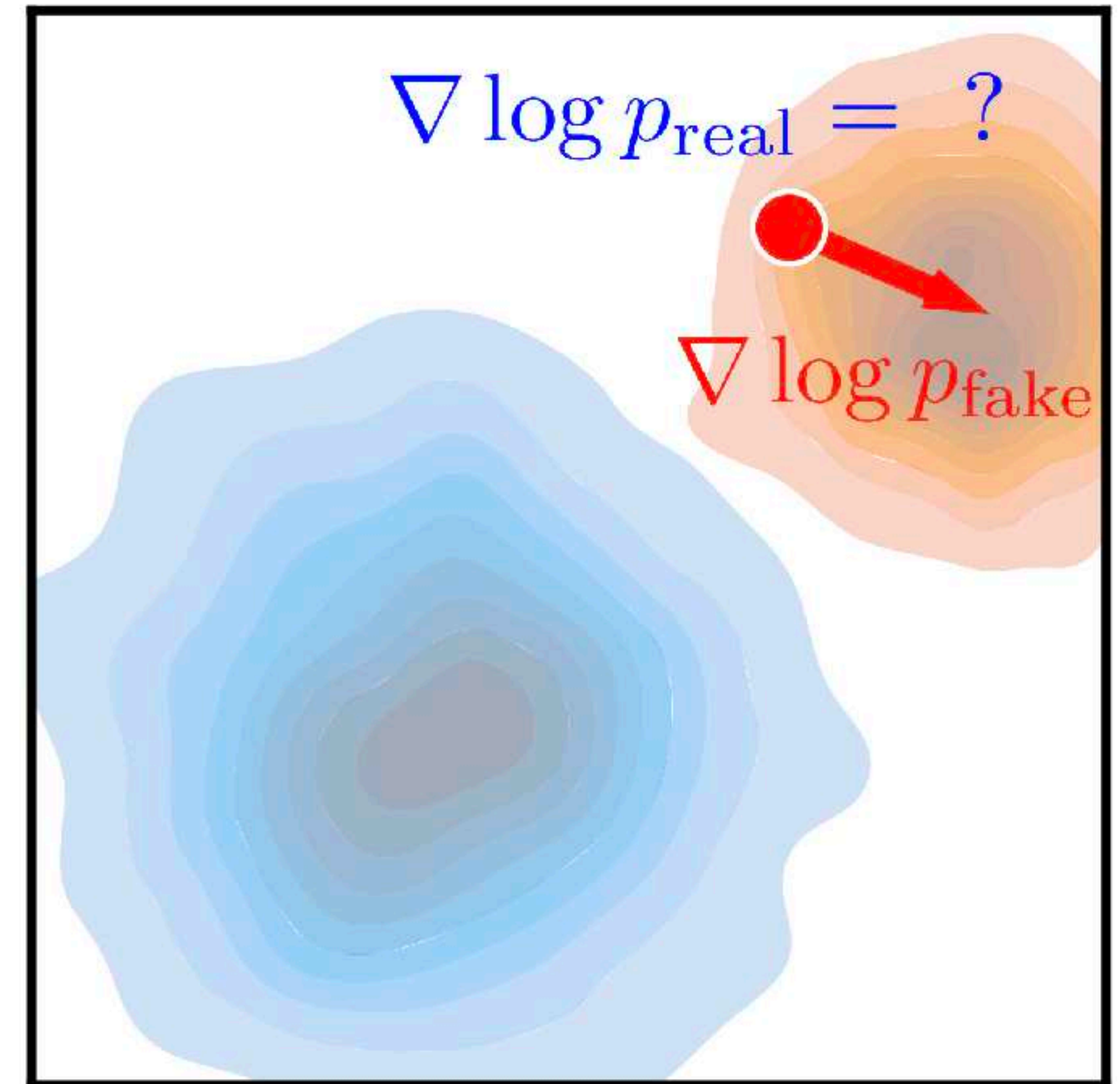
What is a problem here?

Undefined scores

G_θ produces low quality samples at the beginning

Pretrained DM observed nearly real samples for $t = 0$

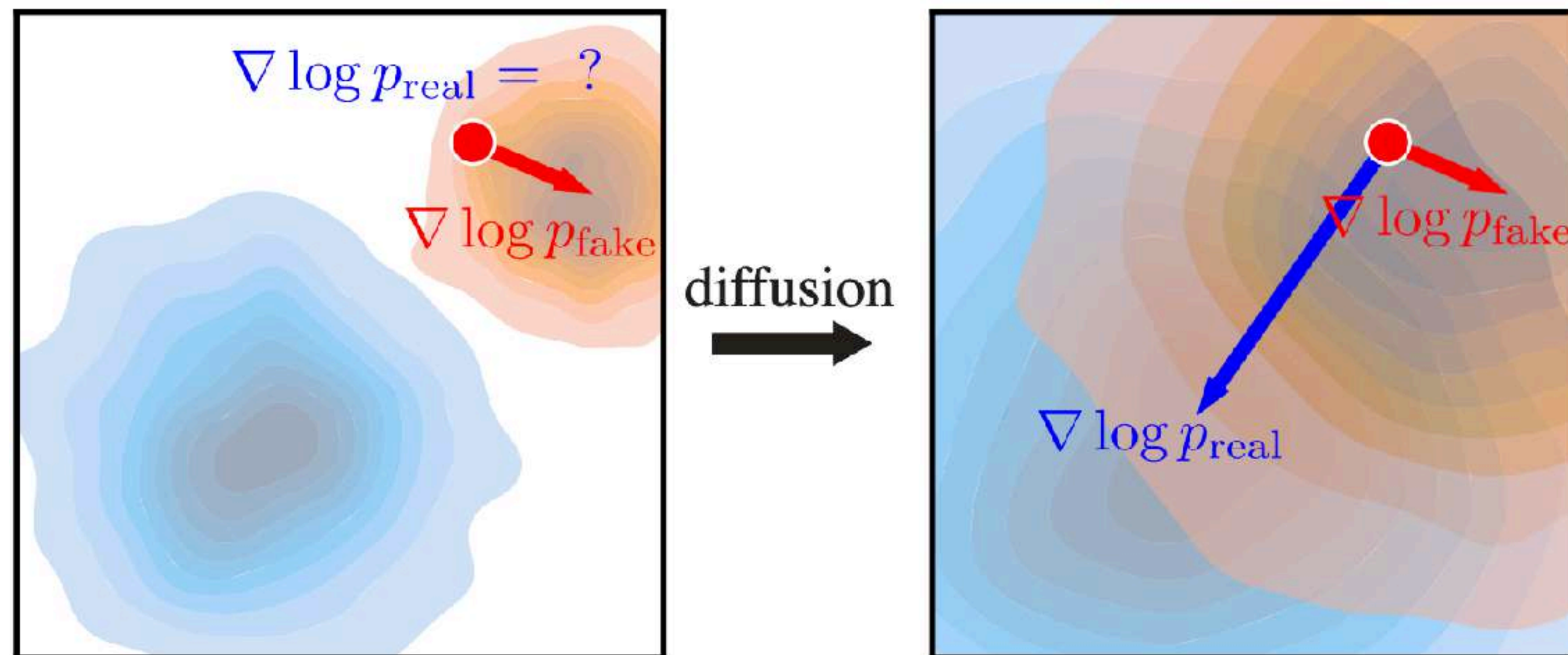
$\rightarrow s_{real}^\psi(\mathbf{x}, 0)$ is inaccurate for $\mathbf{x} \sim p_{fake}(\mathbf{x})$



Undefined scores

Gaussian smoothing makes real and fake scores well-defined.

DMD samples $t \sim \mathcal{U}(0.02T, 0.98T)$ throughout training.

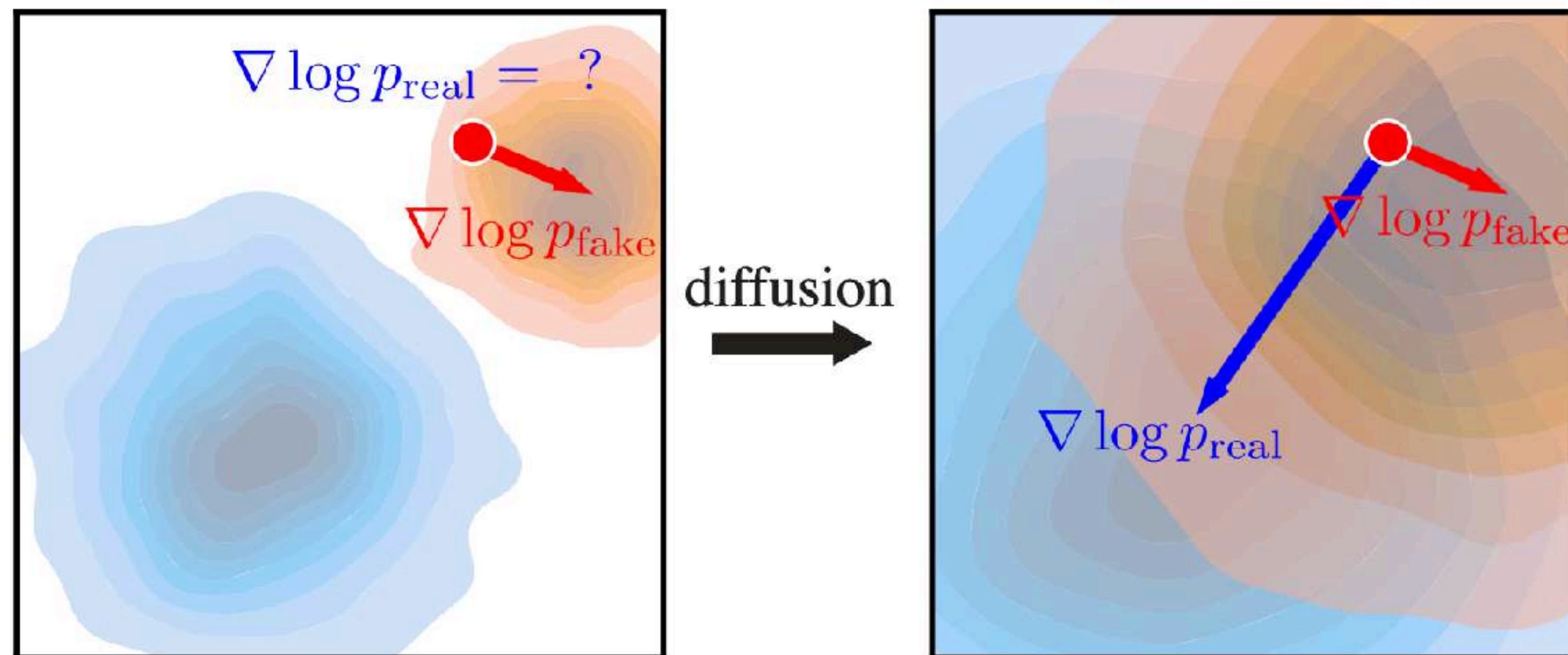


Undefined scores

Gaussian smoothing makes real and fake scores well-defined.

DMD samples $t \sim \mathcal{U}(0.02T, 0.98T)$ throughout training.

The sampling bounds stay fixed as the generator improves.



Overall algorithm

1. Update step for G_θ

$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{z}, \mathbf{x}, \mathbf{x}_t} \left[(s_{fake}^\phi(\mathbf{x}_t, t) - s_{real}^\psi(\mathbf{x}_t, t)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

Overall algorithm

1. Update step for G_θ

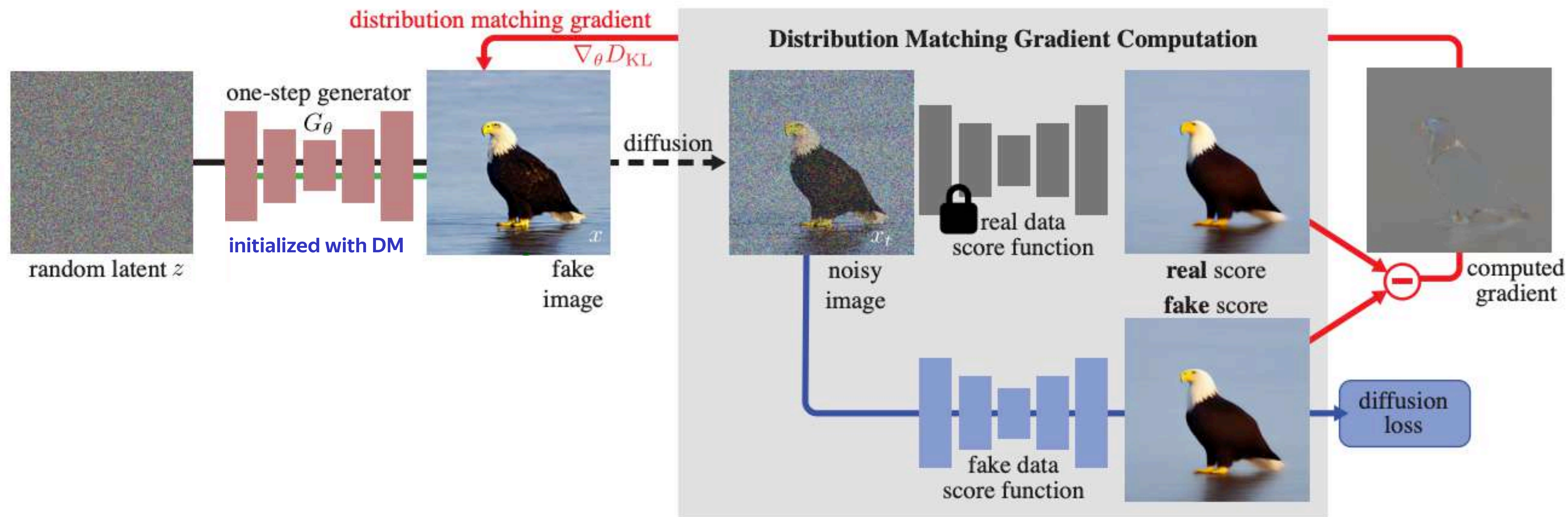
$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \left[(s_{fake}^\phi(\mathbf{x}_t, t) - s_{real}^\psi(\mathbf{x}_t, t)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

2. Update step for fake DM ϵ_ϕ

$$\nabla_\phi \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \|s_{fake}^\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x})\|_2^2 = \nabla_\phi \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \|\epsilon_\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \epsilon\|_2^2$$

P.S.: pretty similar to GANs, where $s_{fake}^\phi(\mathbf{x}_t, t)$ and $s_{real}^\psi(\mathbf{x}_t, t)$ are “half-discriminators”

Distribution Matching Distillation



DMD loss

Practical detail: how to implement $\nabla_{\theta} D_{KL}$?

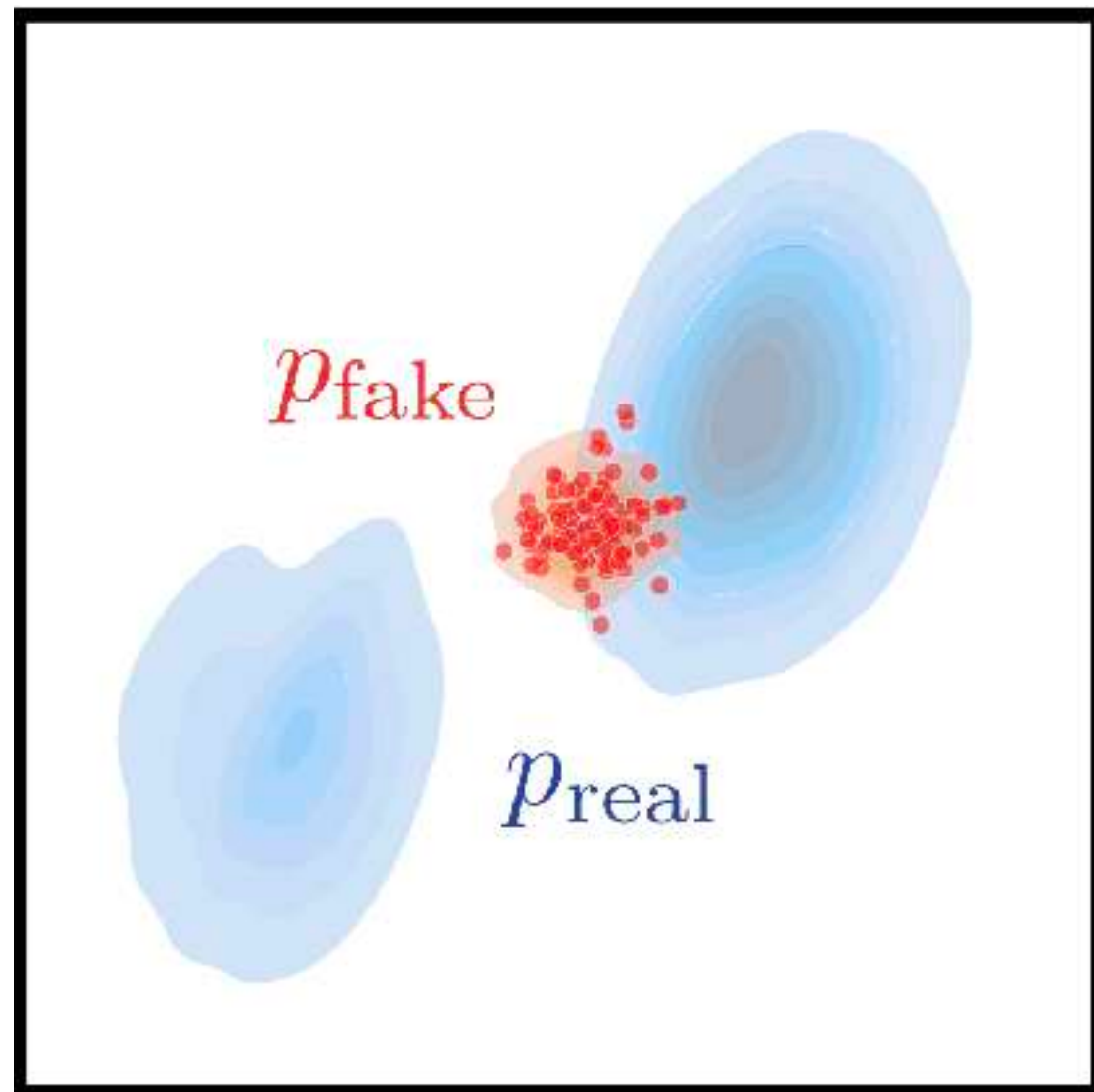
DMD loss

Practical detail: how to implement $\nabla_{\theta} D_{KL}$?

Surrogate loss:
$$L = \frac{1}{2} \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \left[\|G_{\theta}(\mathbf{z}) - \text{sg} \left(G_{\theta}(\mathbf{z}) - s_{fake}^{\phi}(\mathbf{x}_t, t) + s_{real}^{\psi}(\mathbf{x}_t, t) \right)\|_2^2 \right]$$

It can be shown that $\nabla_{\theta} L = \nabla_{\theta} D_{KL}$ via chain rule.

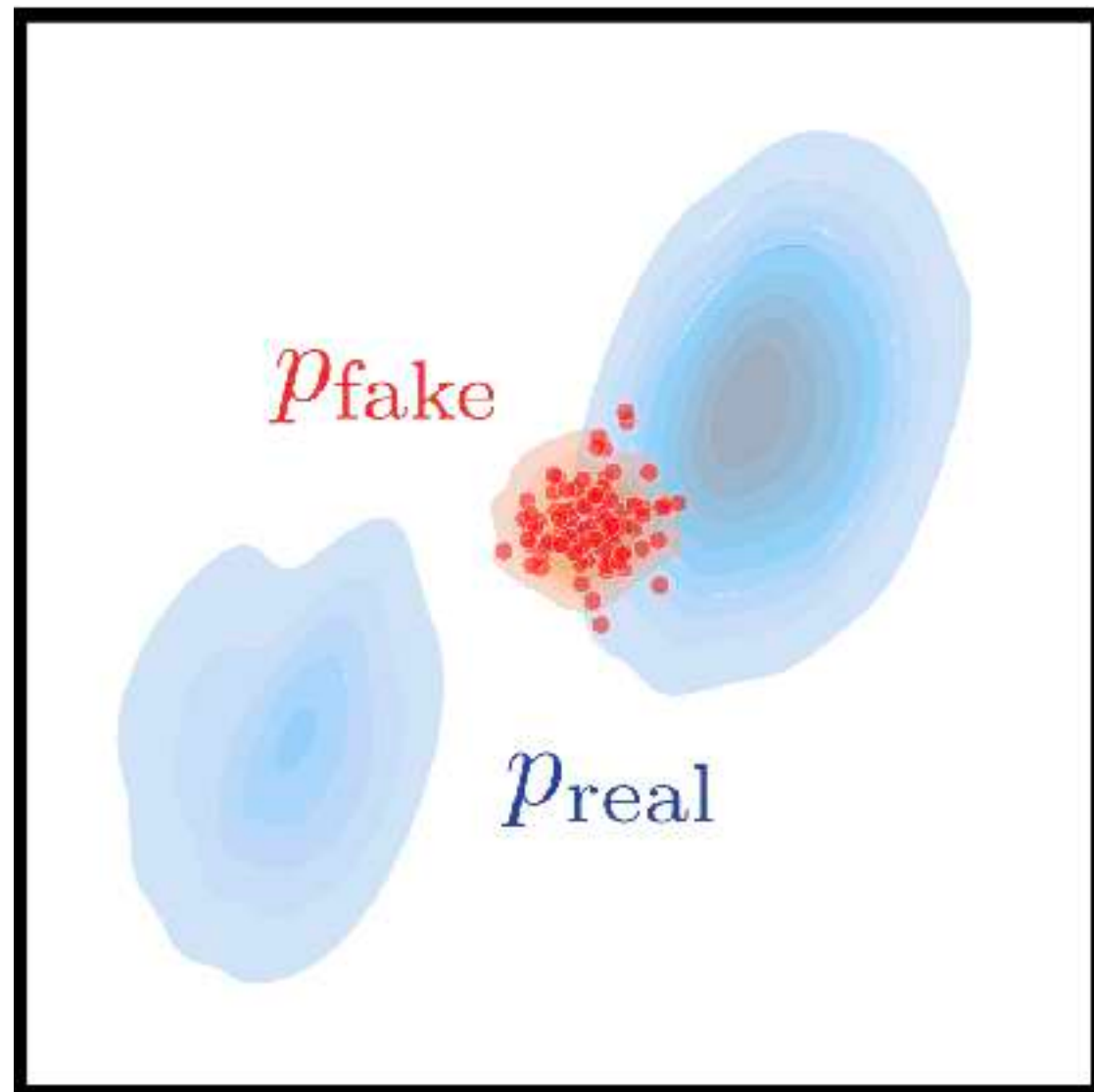
Reverse KL problem



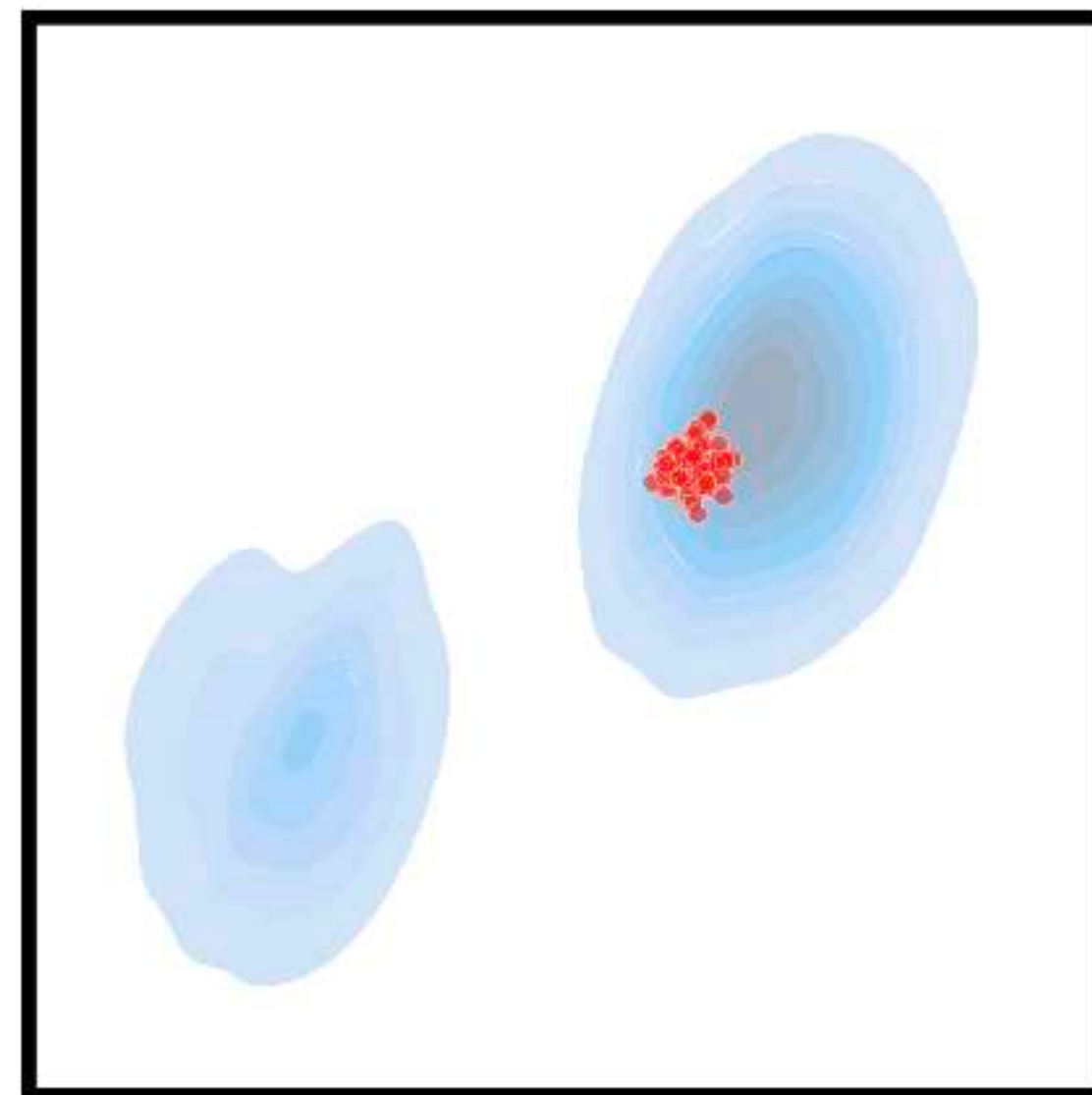
Initial state

Reverse KL problem

$$\mathbb{E}_{\substack{z \sim \mathcal{N}(0; \mathbf{I}) \\ x = G_{\theta}(z)}} - (\log p_{\text{real}}(x) - \log \cancel{p_{\text{fake}}(x)})$$



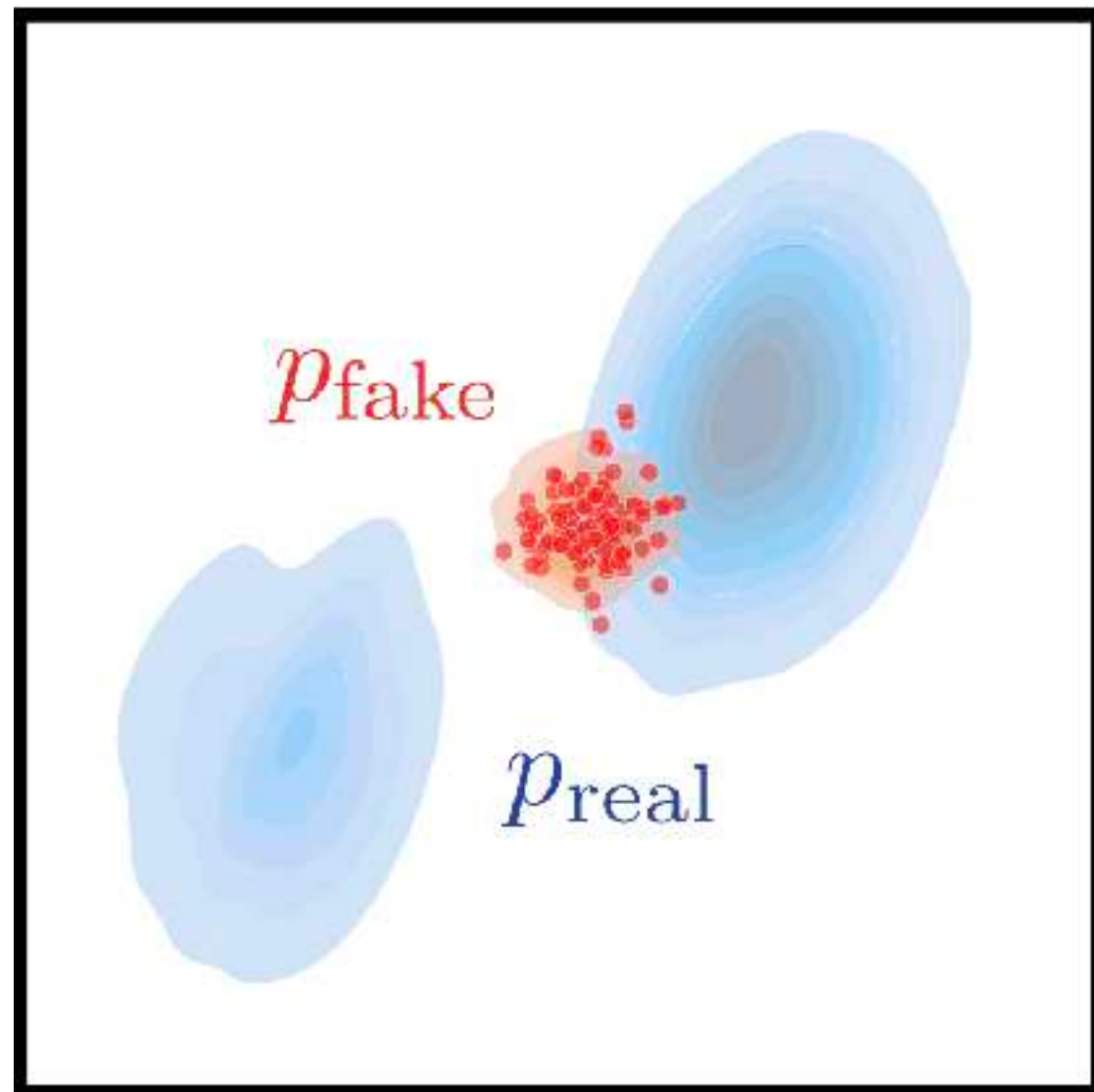
Initial state



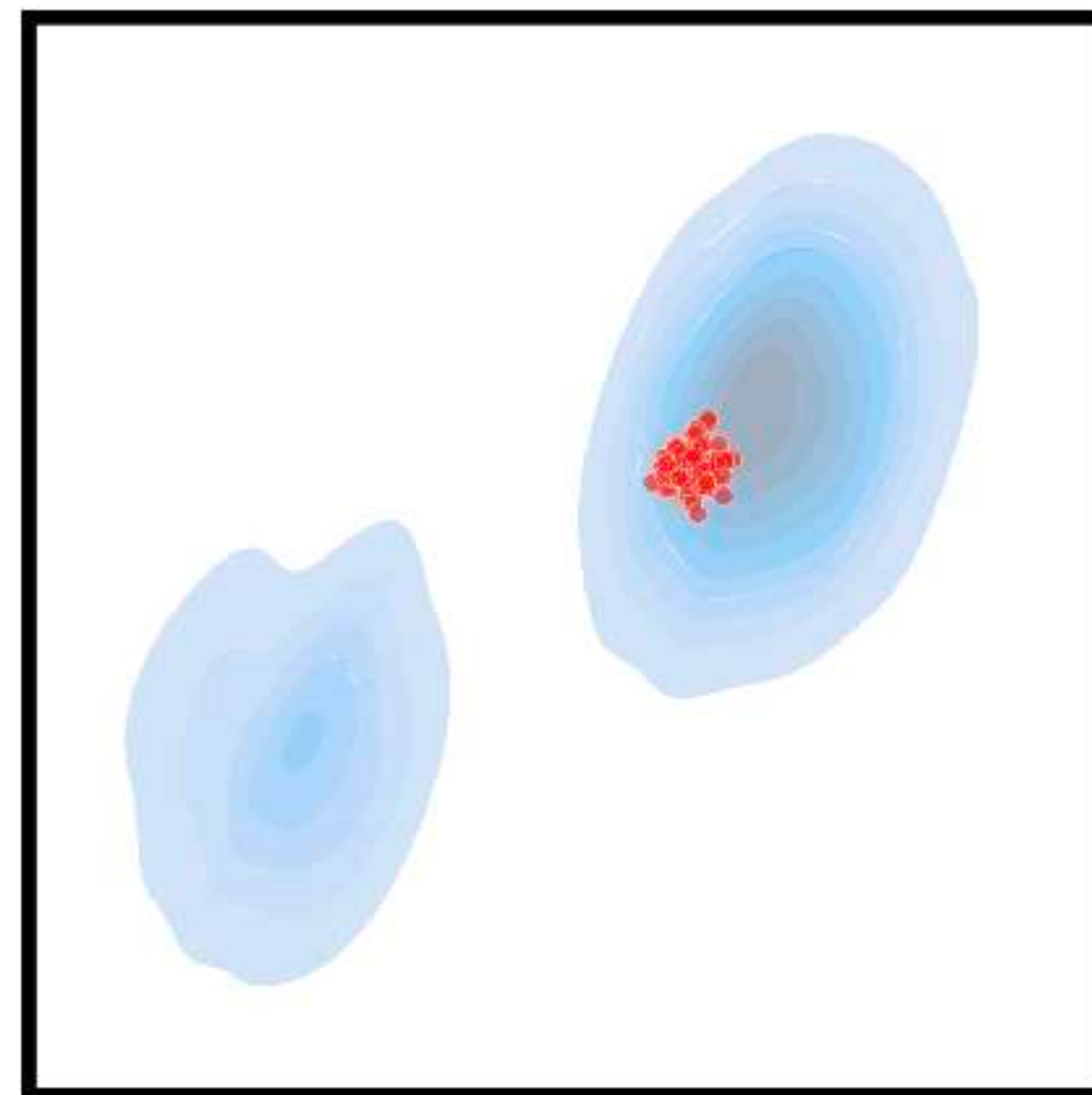
Real score only

Reverse KL problem

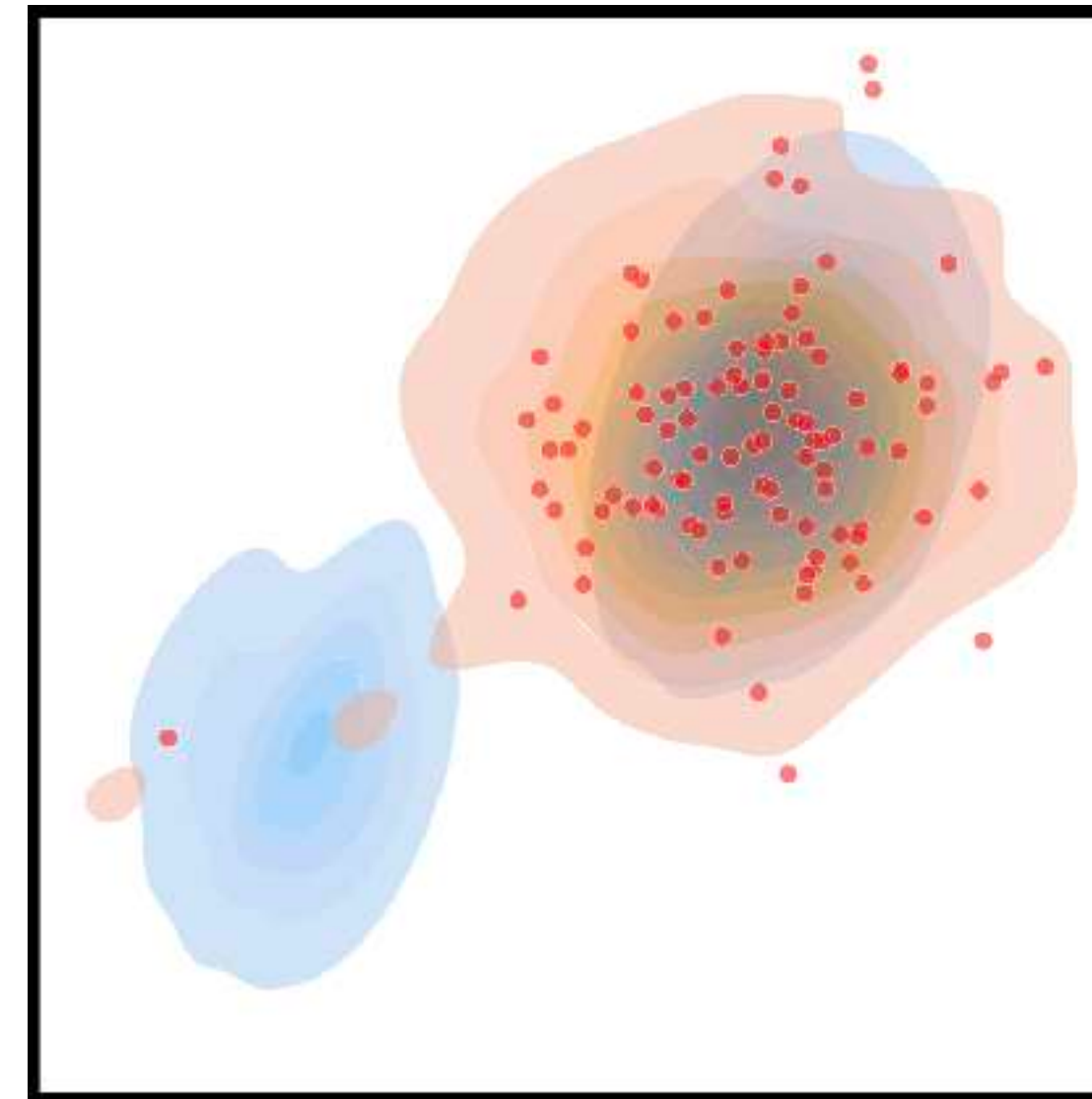
$$\mathbb{E}_{\substack{z \sim \mathcal{N}(0; \mathbf{I}) \\ x = G_{\theta}(z)}} - (\log p_{\text{real}}(x) - \log p_{\text{fake}}(x))$$



Initial state



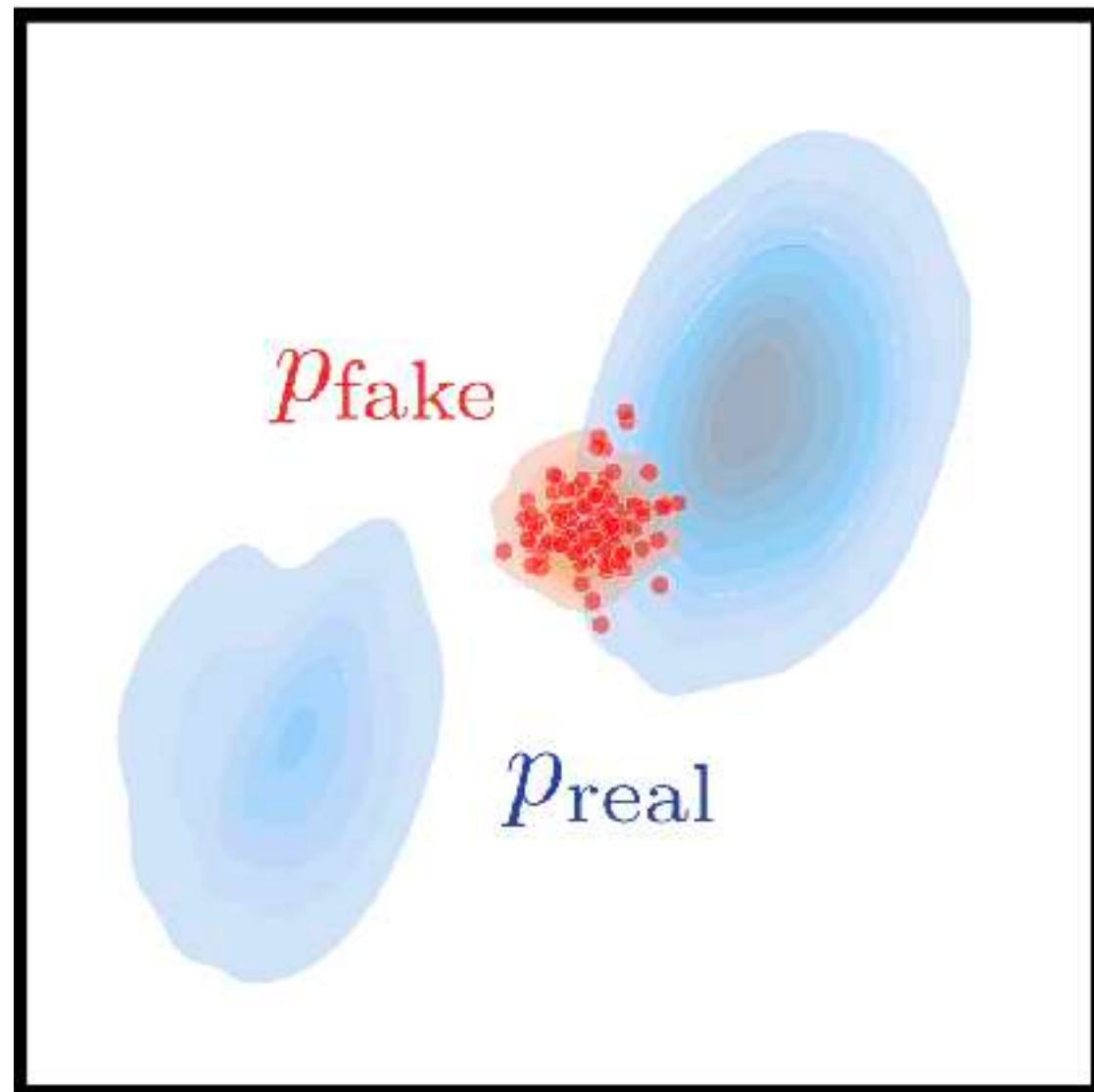
Real score only



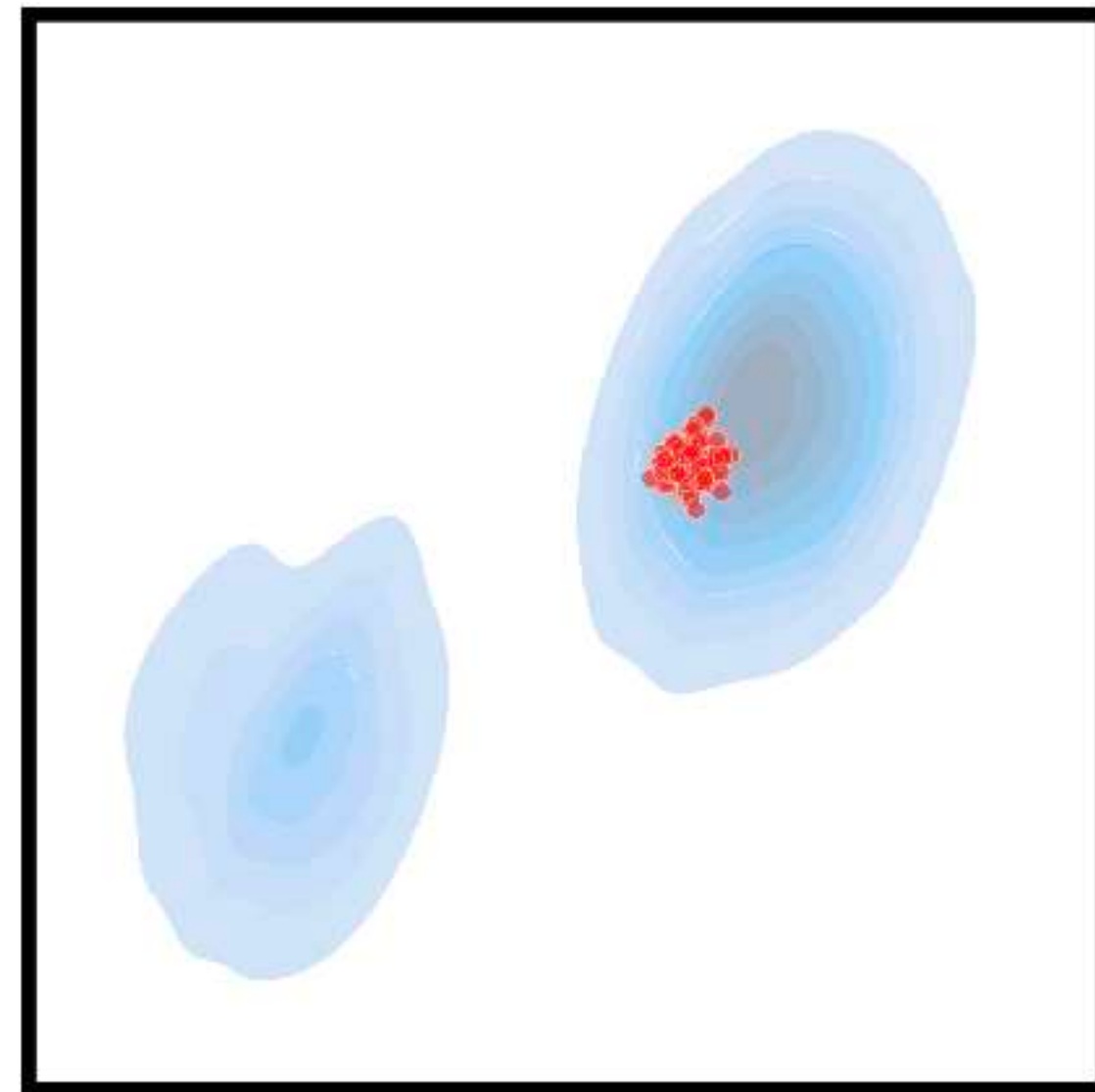
Real + fake scores

Reverse KL problem

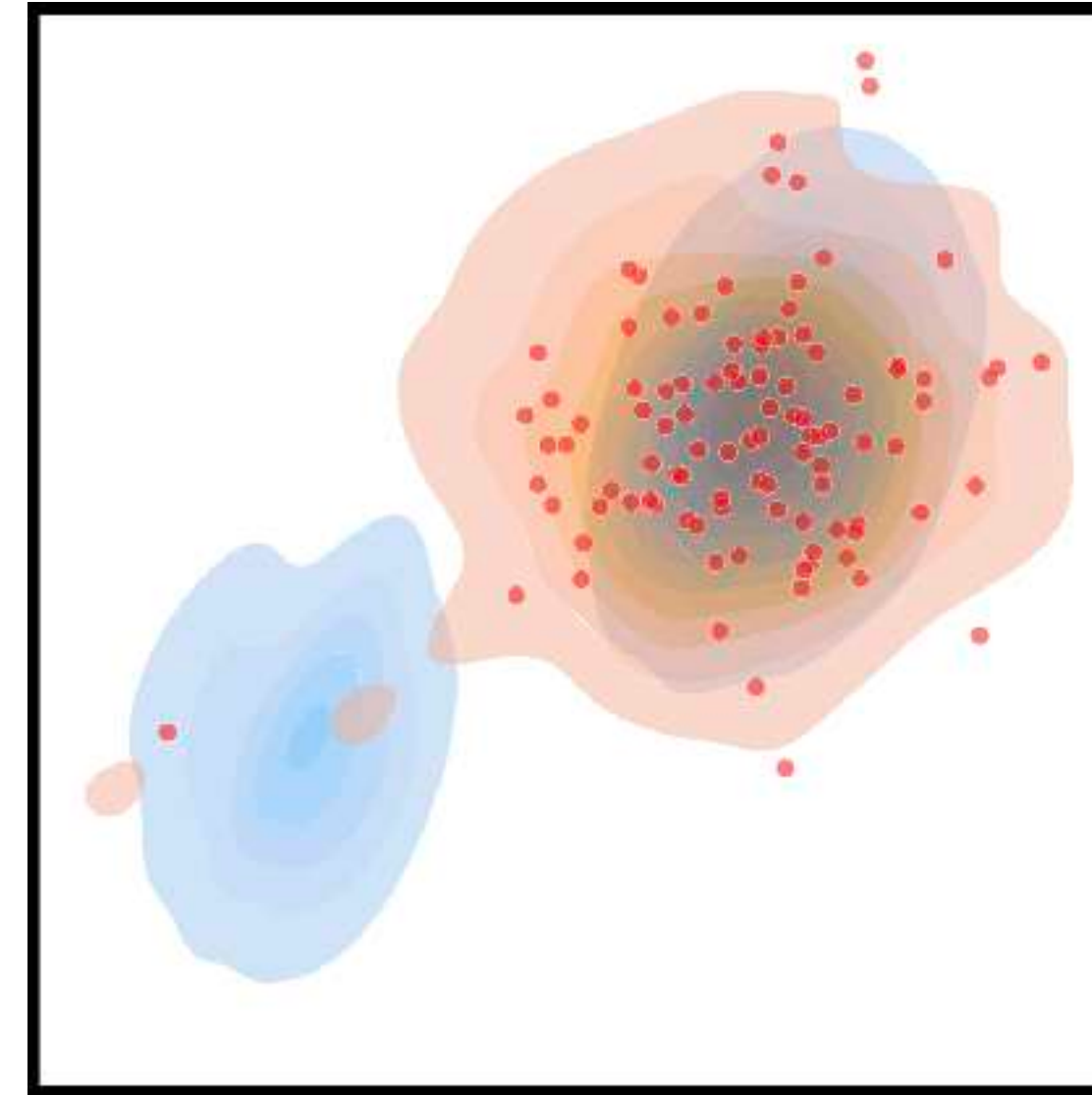
How to avoid mode collapse?



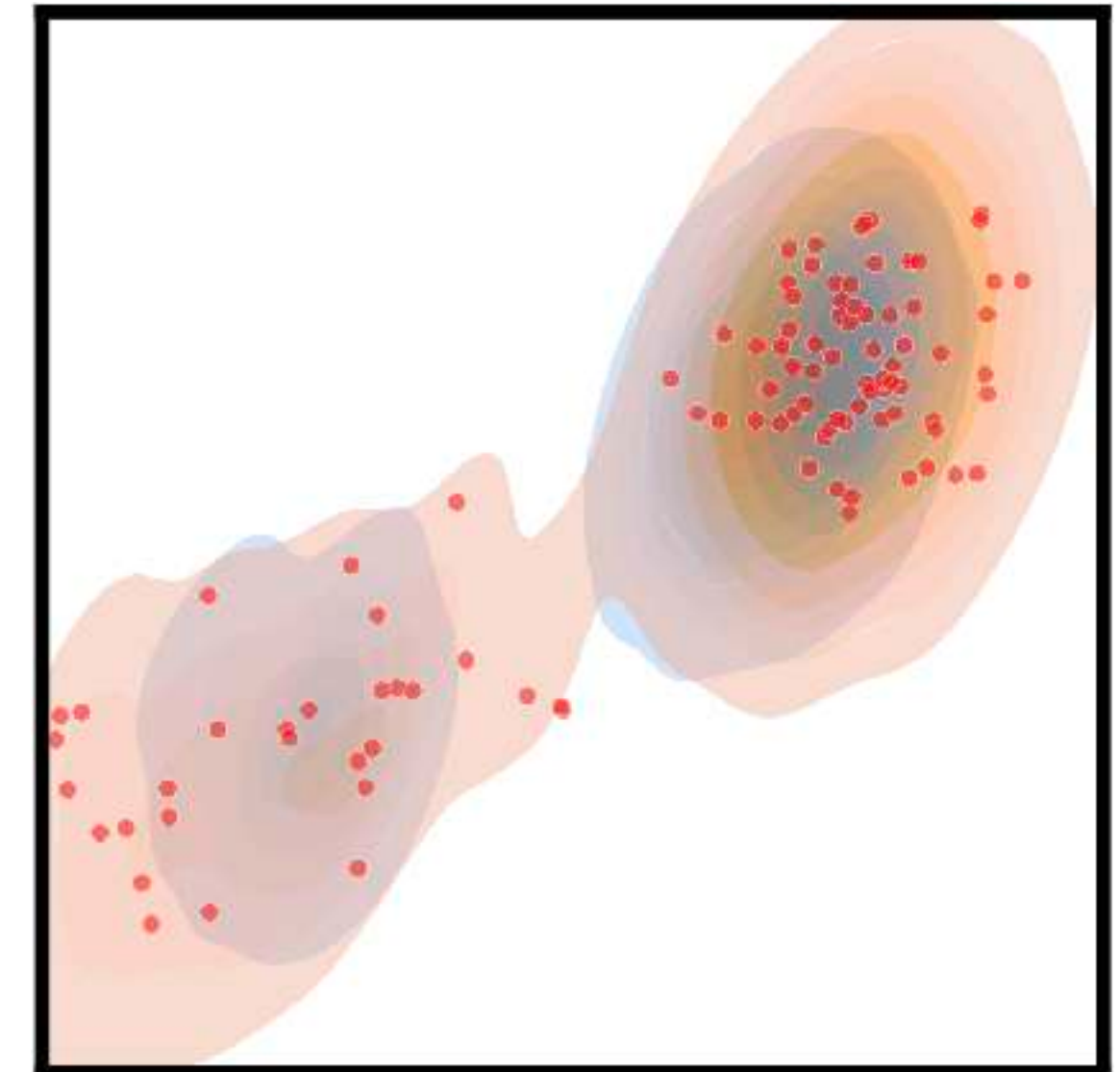
Initial state



Real score only

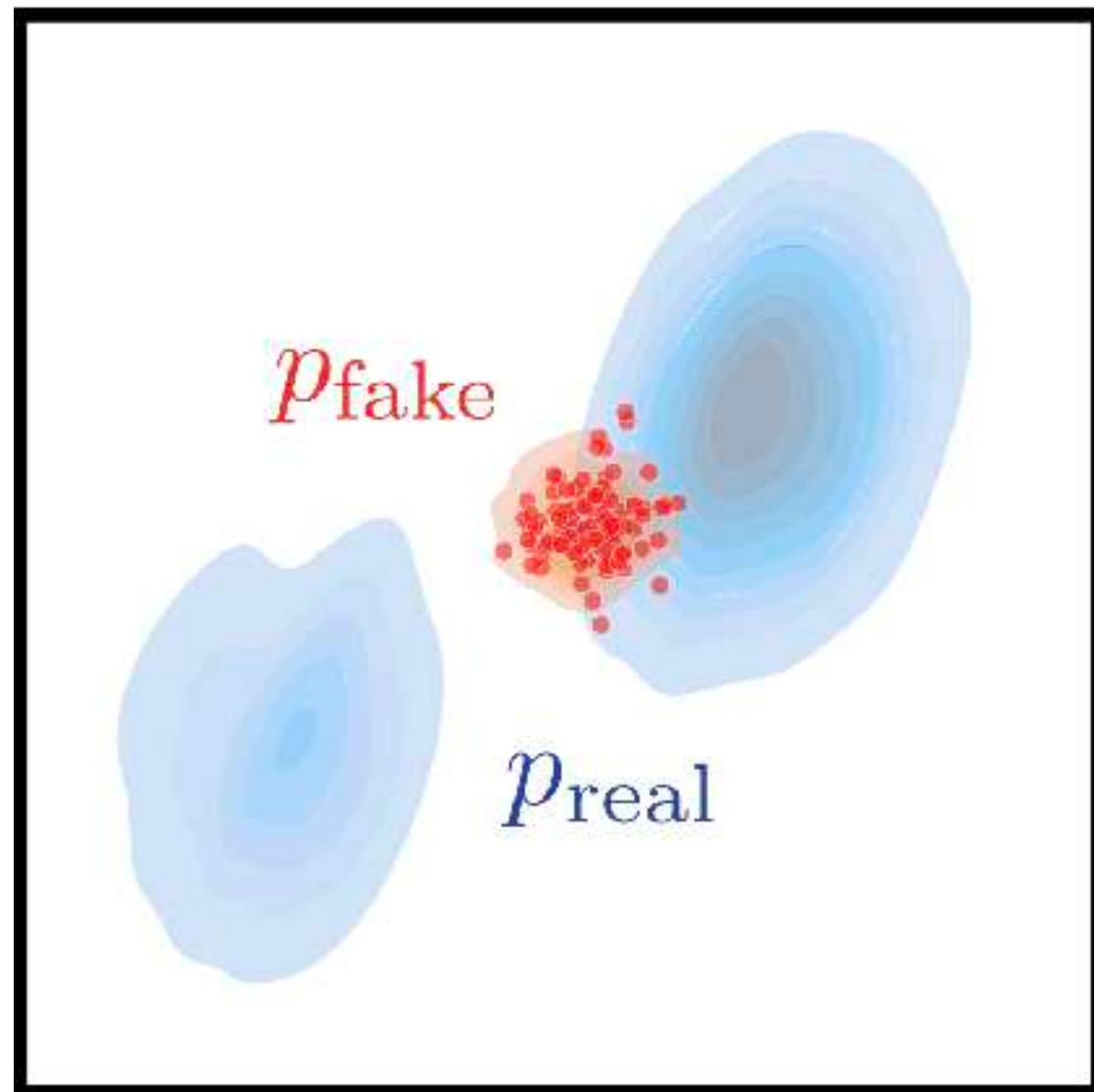


Real + fake scores

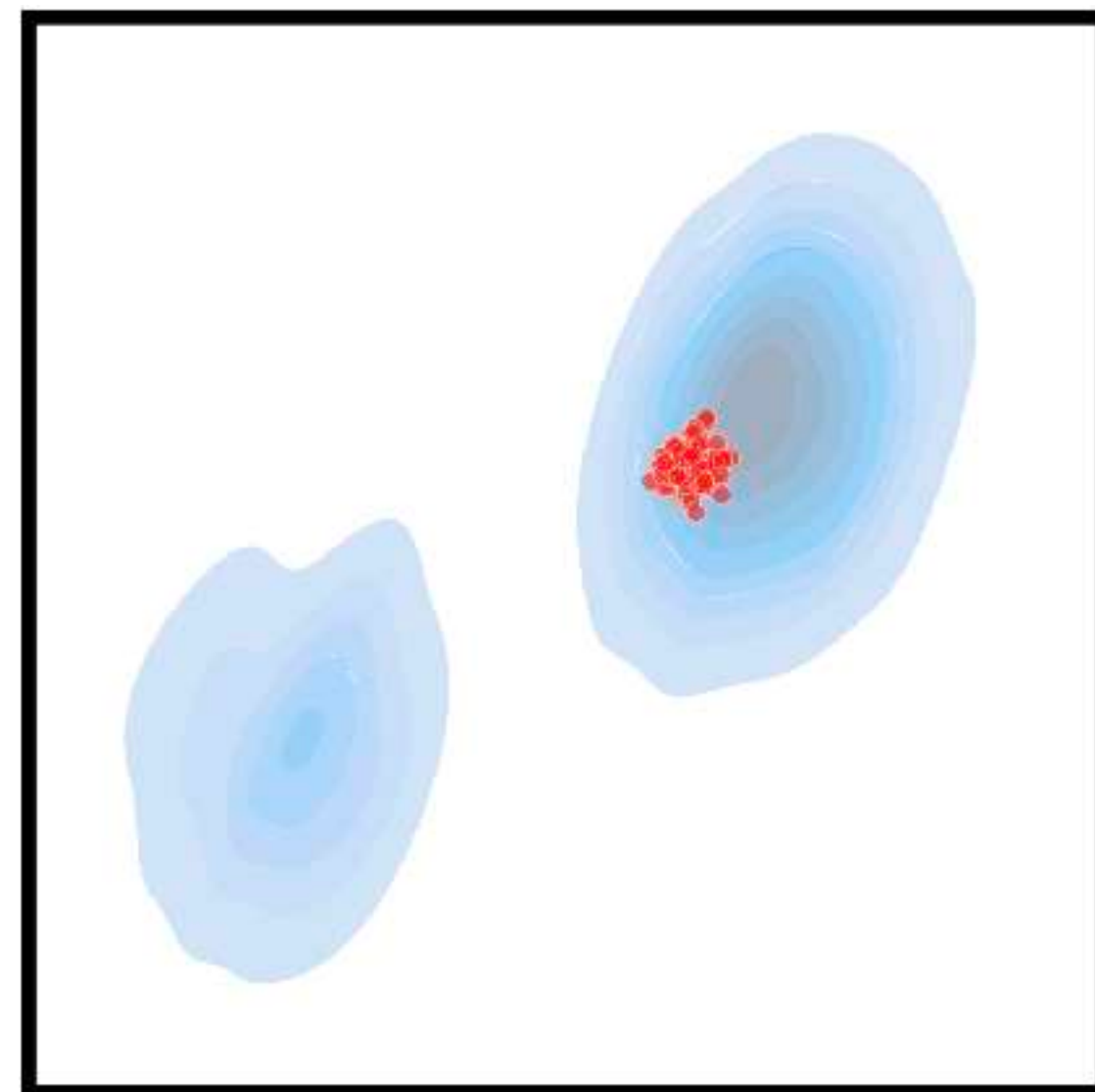


Reverse KL problem

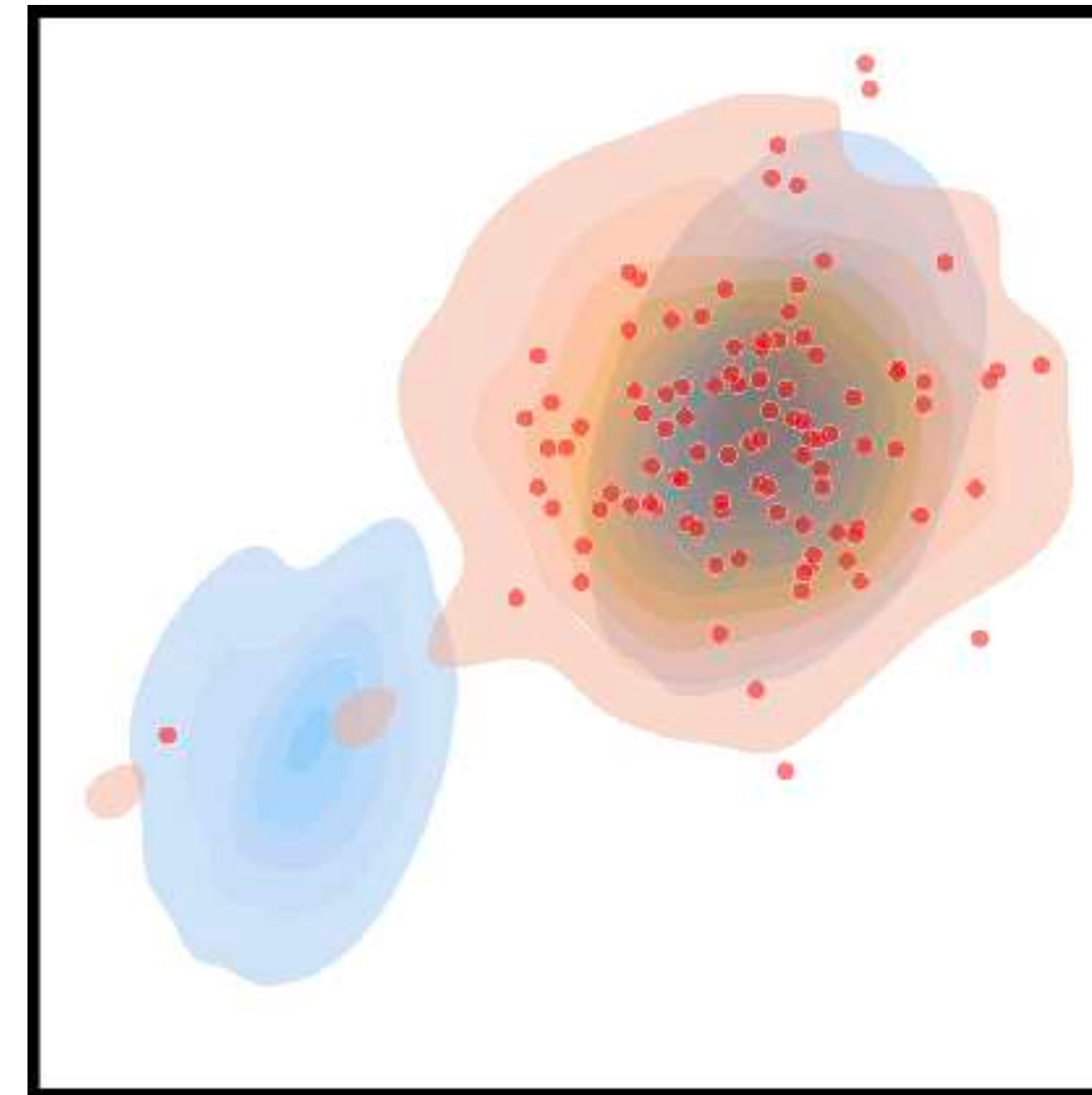
Additionally use a classical Knowledge Distillation approach



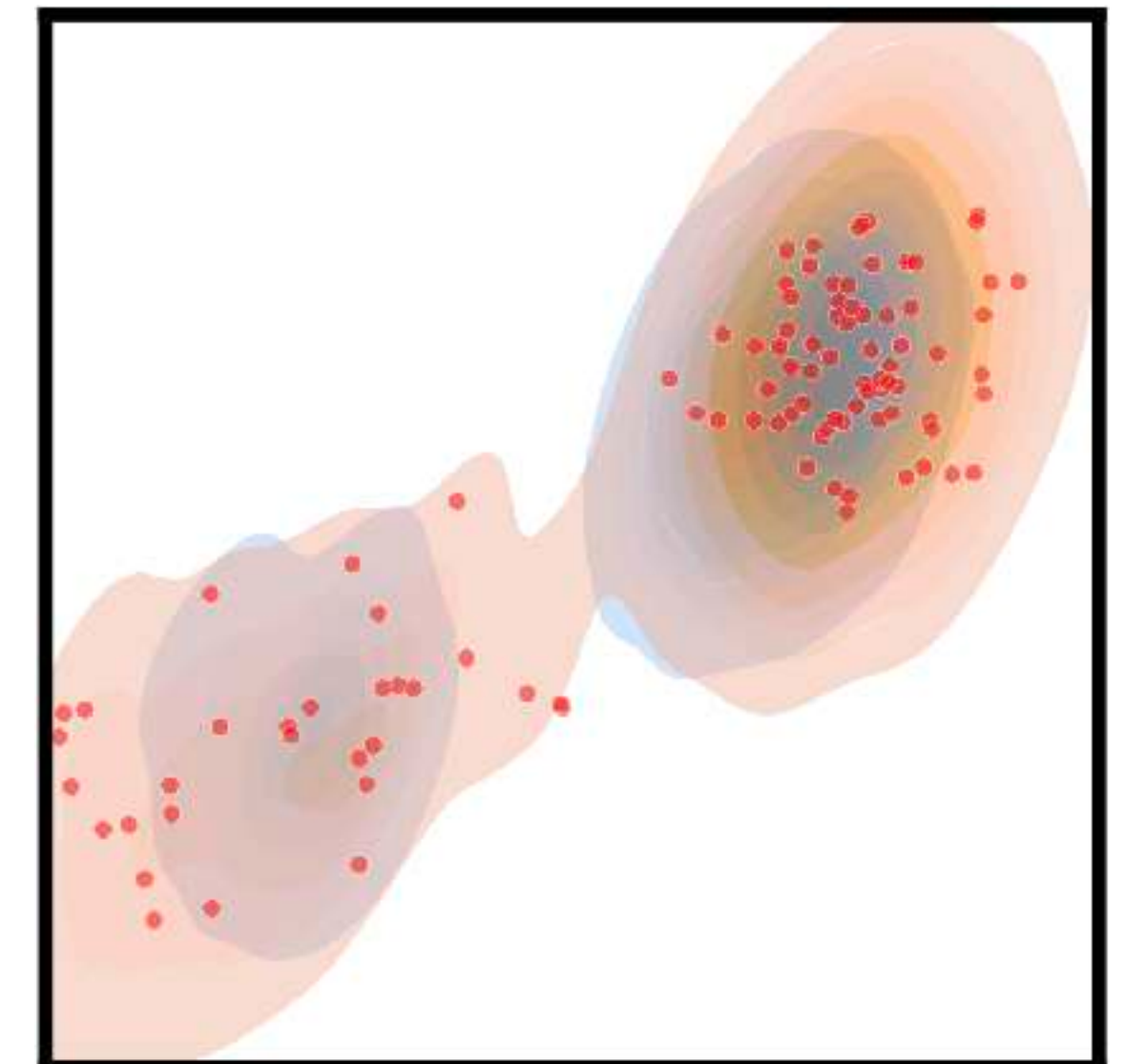
Initial state



Real score only

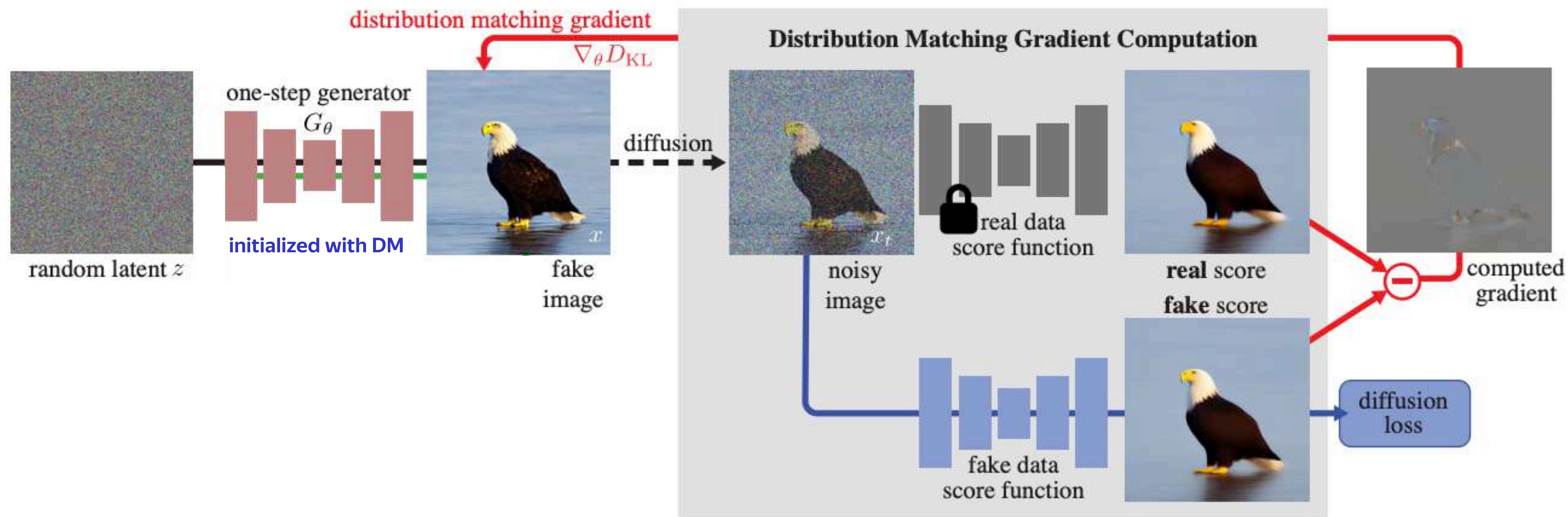


Real + fake scores

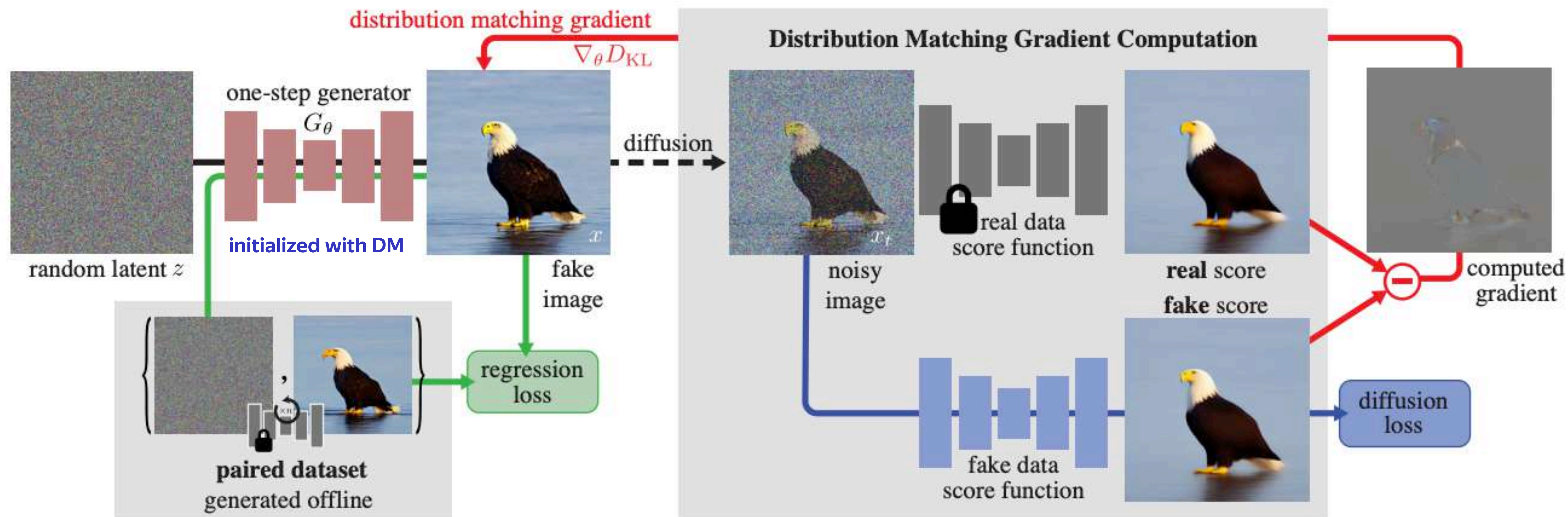


Real + fake scores + KD

Distribution Matching Distillation



Distribution Matching Distillation



Score Distillation Sampling | SDS

Can we get rid of the fake DM?

Score Distillation Sampling | SDS

SDS uses the conditional noising score in place of the learned fake score.

The posterior score identity is:

$$s_{\text{fake}}(x_t, t) = \mathbb{E}_{x \sim p_{\text{fake}}(x|x_t)} \left[\nabla_{x_t} \log q_t(x_t | x) \right], \quad \nabla_{x_t} \log q_t(x_t | x) = -\frac{x_t - \alpha_t x}{\sigma_t^2}.$$

Inside the generator gradient, its Jacobian depends on the same clean sample.

This substitution changes the objective; it is generally a biased DMD-gradient estimate.

Score Distillation Sampling | SDS

Can we get rid of the fake DM?

Let's use unbiased score function estimation for $s_{fake}(\mathbf{x}_t, t)$ on $\mathbf{x} \sim G_\theta(\mathbf{z})$:

$$s_{fake}(\mathbf{x}_t, t) = \nabla_{\mathbf{x}_t} \log p_{fake}(\mathbf{x}_t) = \mathbb{E}_{\mathbf{x} \sim p_{fake}} [\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) | \mathbf{x}_t] \approx \nabla_{x_t} \log q(\mathbf{x}_t | \mathbf{x}) = -\frac{\mathbf{x}_t - \alpha_t \mathbf{x}}{\sigma_t^2}$$

- **Performs poorly alone** → used only as a minor polishing loss
- Alternative cheap ways to avoid the fake DM are still unsuccessful

DMD summary

Key idea: use DM forward passes to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

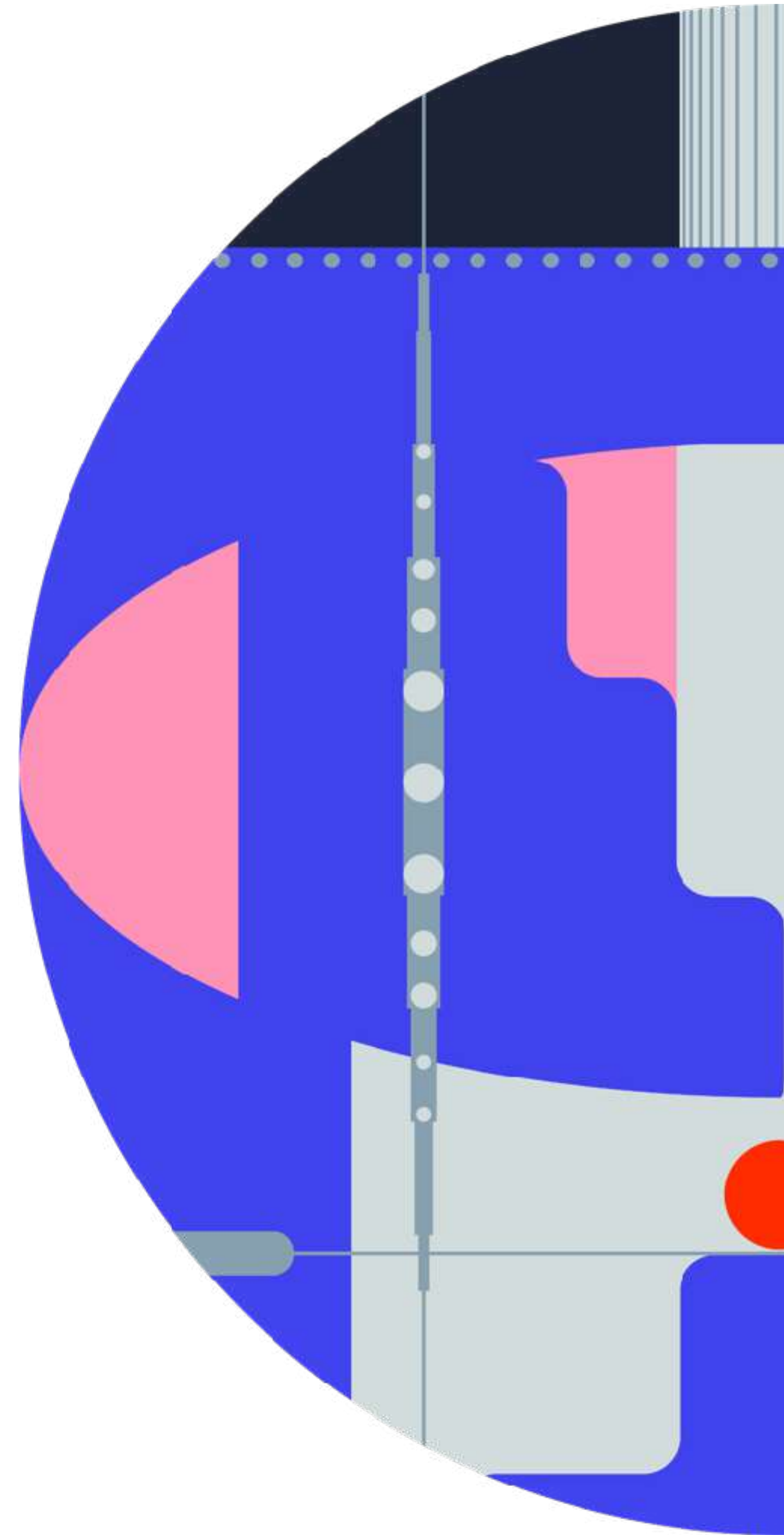
Pros

- Fruitful idea for future research and promising results

Cons

- Requires additional DM training in parallel
- Relatively expensive and unstable
- Prone to mode collapse → needs additional losses

Adversarial Diffusion Distillation

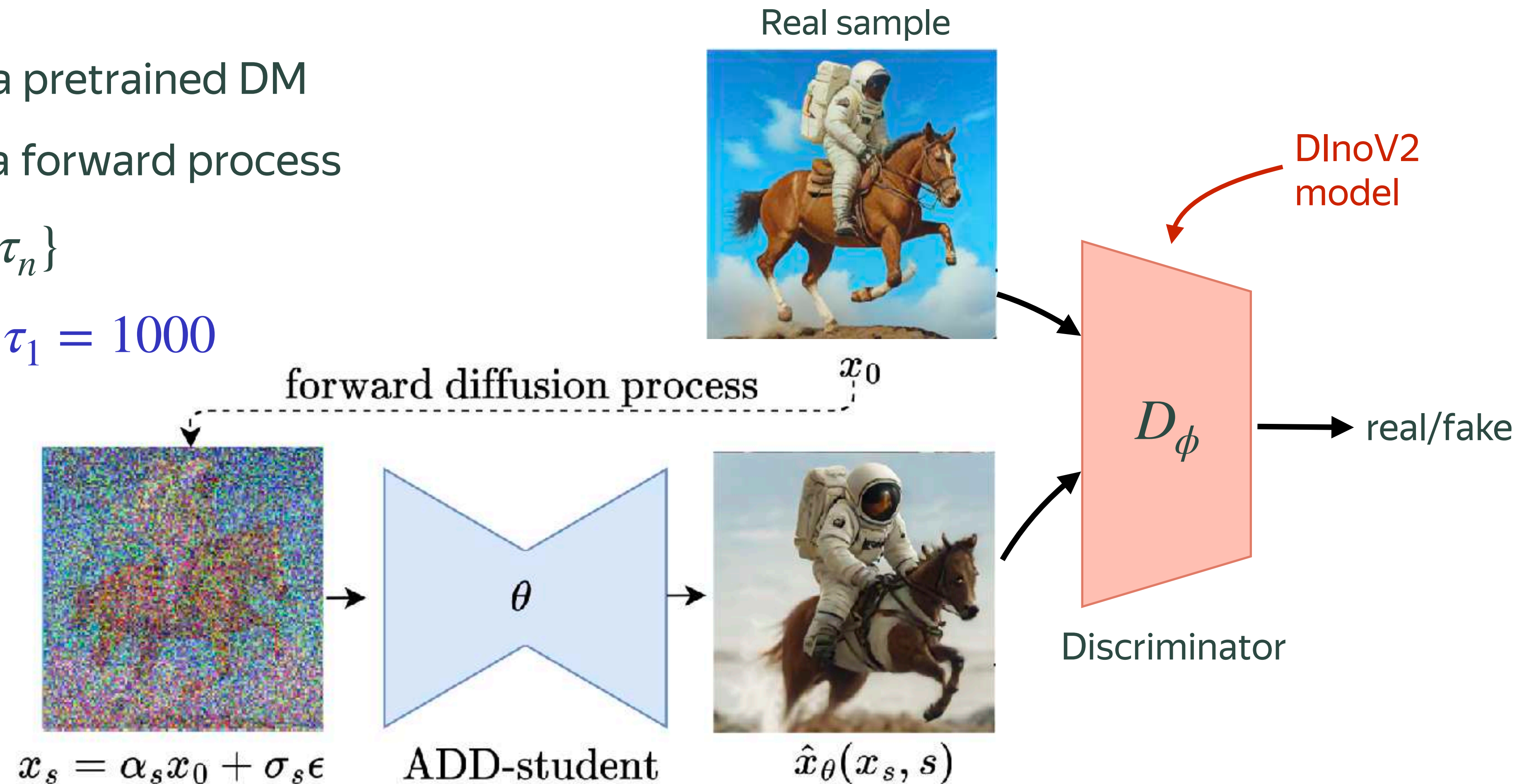


Adversarial Diffusion Distillation

1. $G_\theta(\mathbf{z})$ is initialized with a pretrained DM
2. Multistep sampling via a forward process

$$s \in T_{student} = \{\tau_1, \dots, \tau_n\}$$

ADD = GAN for $n = 1$ and $\tau_1 = 1000$

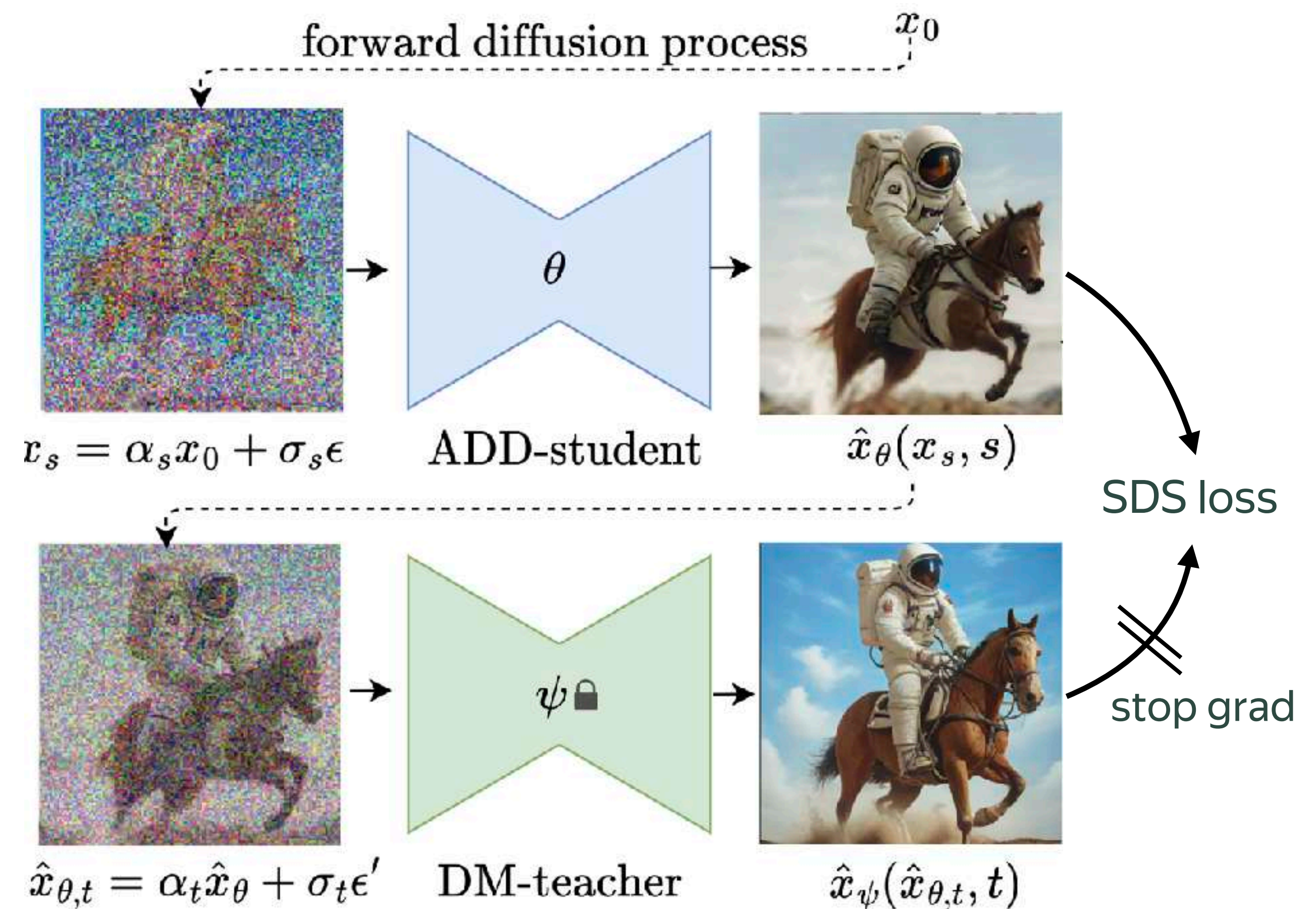


Adversarial Diffusion Distillation

Combine GAN and SDS losses

Here SDS is reparameterized via $\mathbf{x}_0$ -prediction

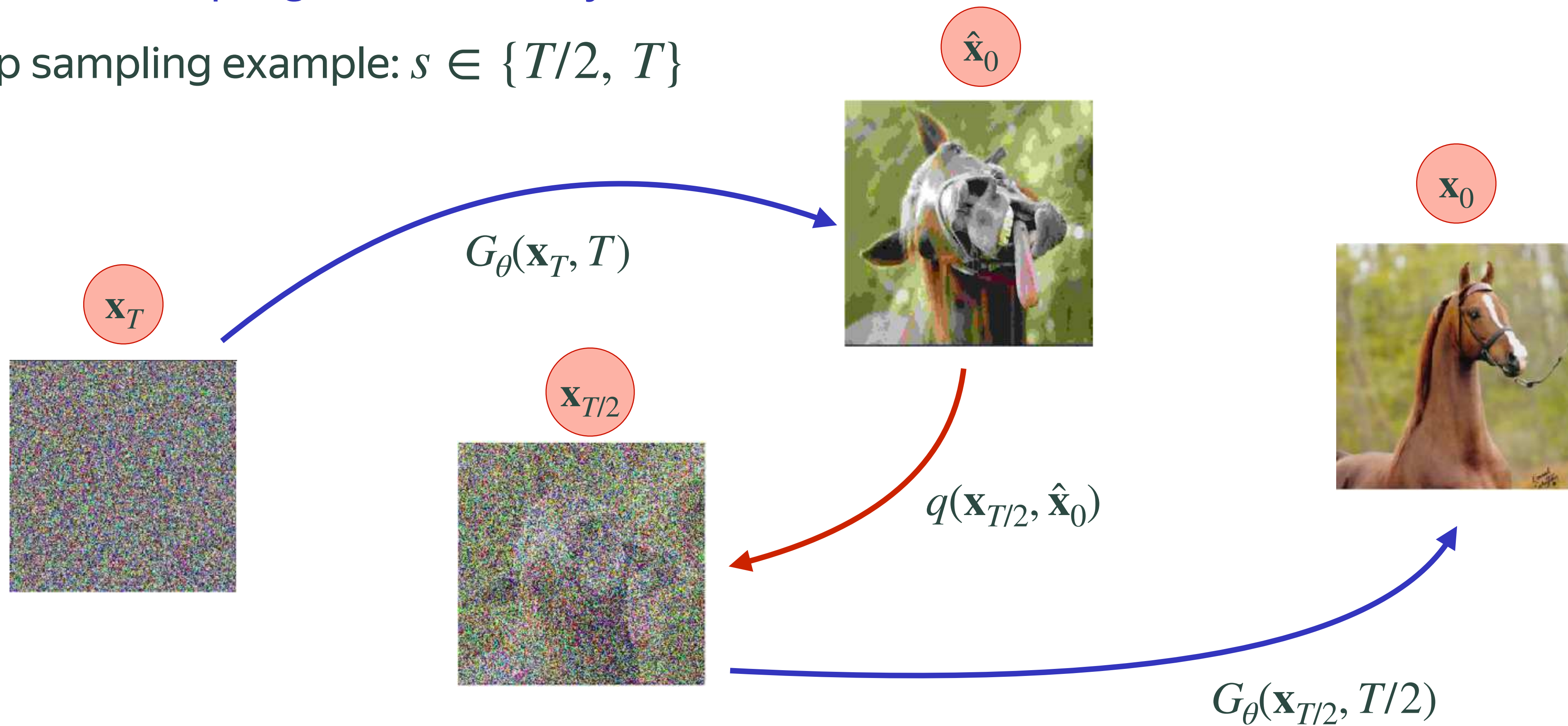
SDS plays a minor role in the final performance



ADD sampling

Identical to sampling in Consistency Models

2-step sampling example: $s \in \{T/2, T\}$



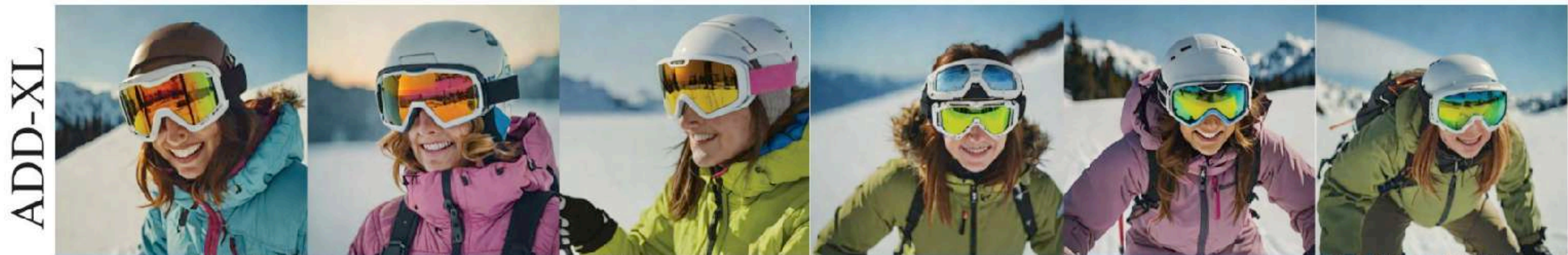
ADD summary

SDXL-Turbo

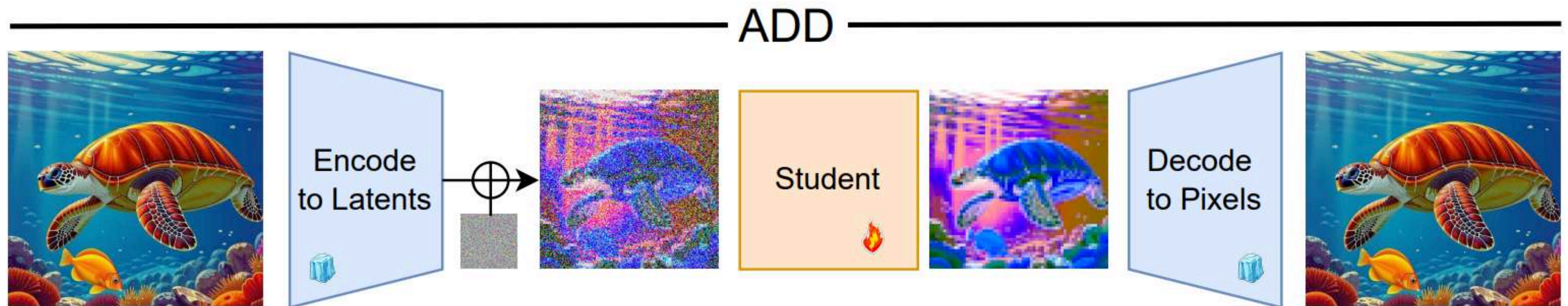
Key idea: DM-initialized GAN + SDS for negligible polishing

Both objectives are prone to mode collapse → poor sample diversity for a given prompt

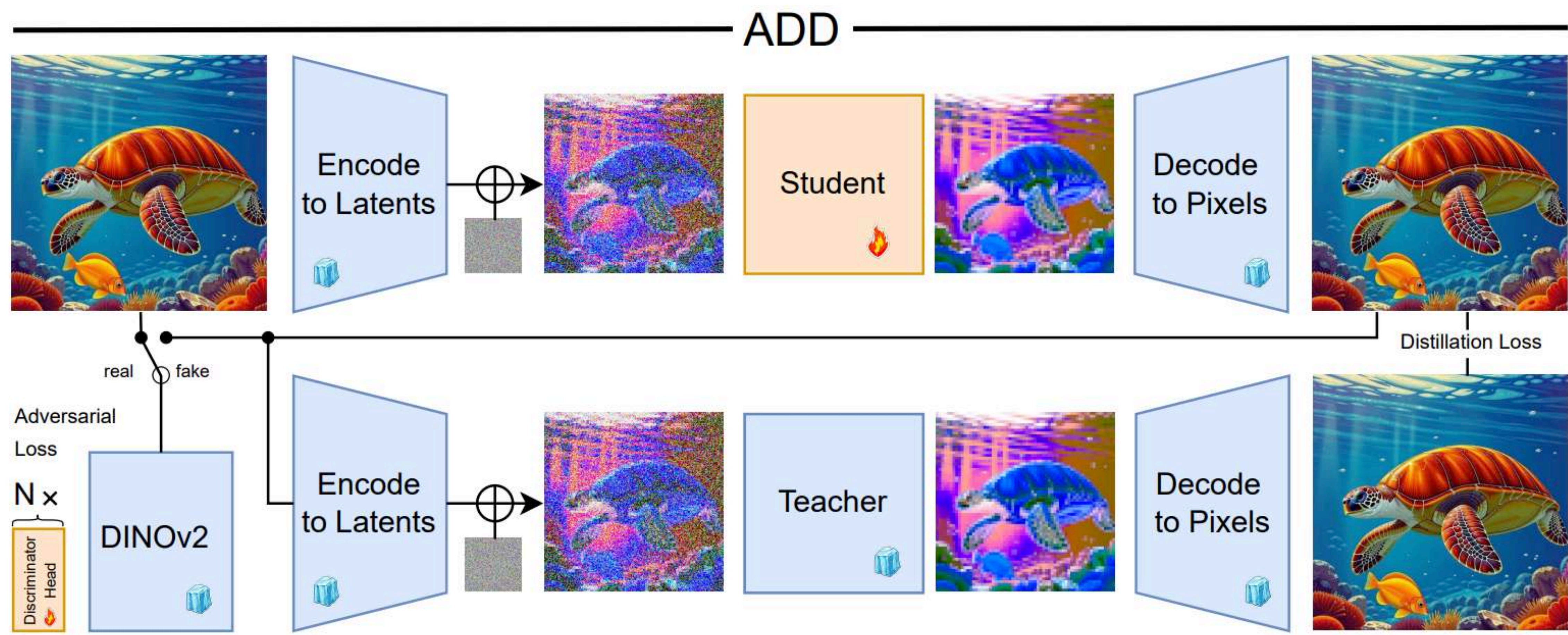
Woman bent slightly on skis wearing goggles and snowsuit.



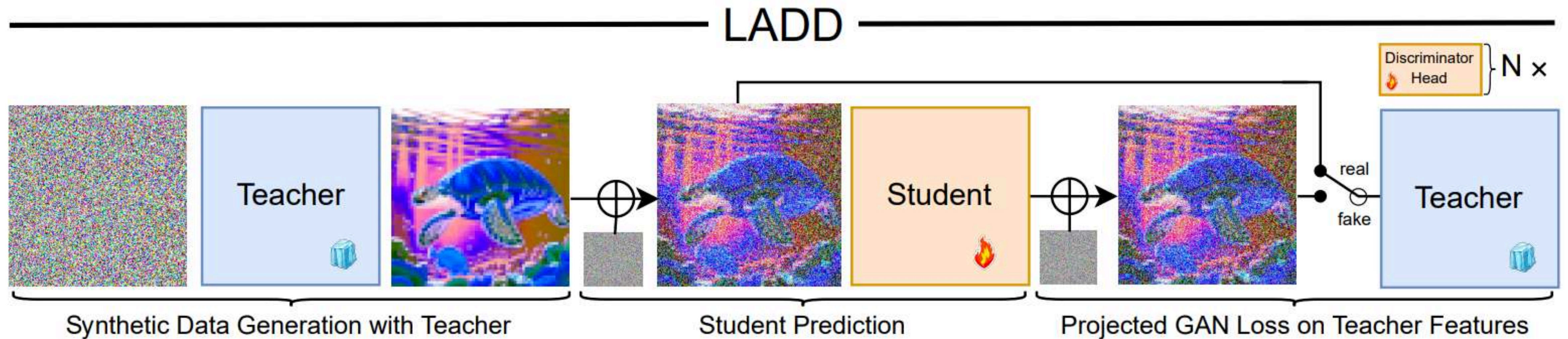
Adversarial Diffusion Distillation



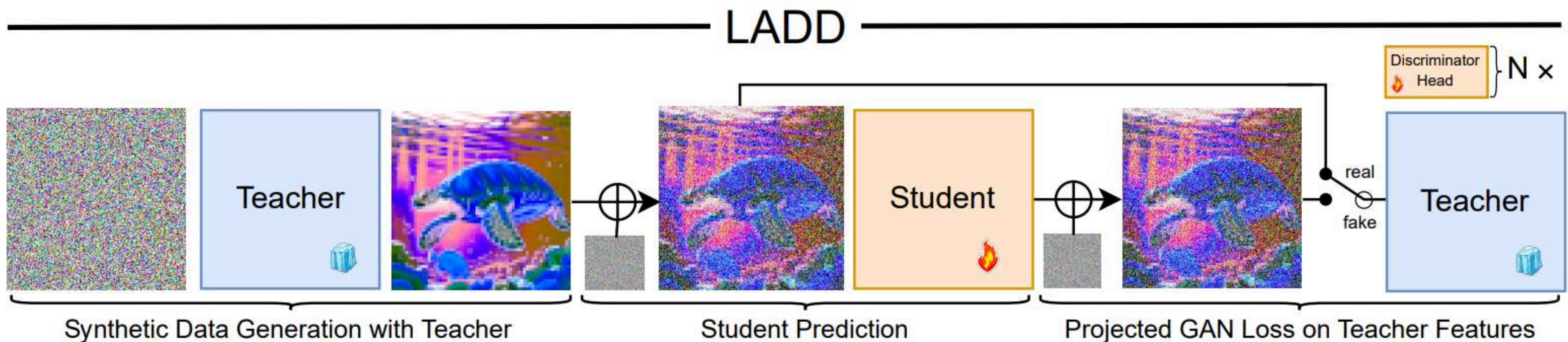
Adversarial Diffusion Distillation



Latent Adversarial Diffusion Distillation



Latent Adversarial Diffusion Distillation



Practical note: If you have higher quality external data than teacher samples, then use it

LADD vs ADD

Stays in VAE latent space → more efficient training

Discriminator is initialized with a teacher DM → feedback from different noise levels

SDS is not needed

LADD is a reasonable choice in production

What about ...

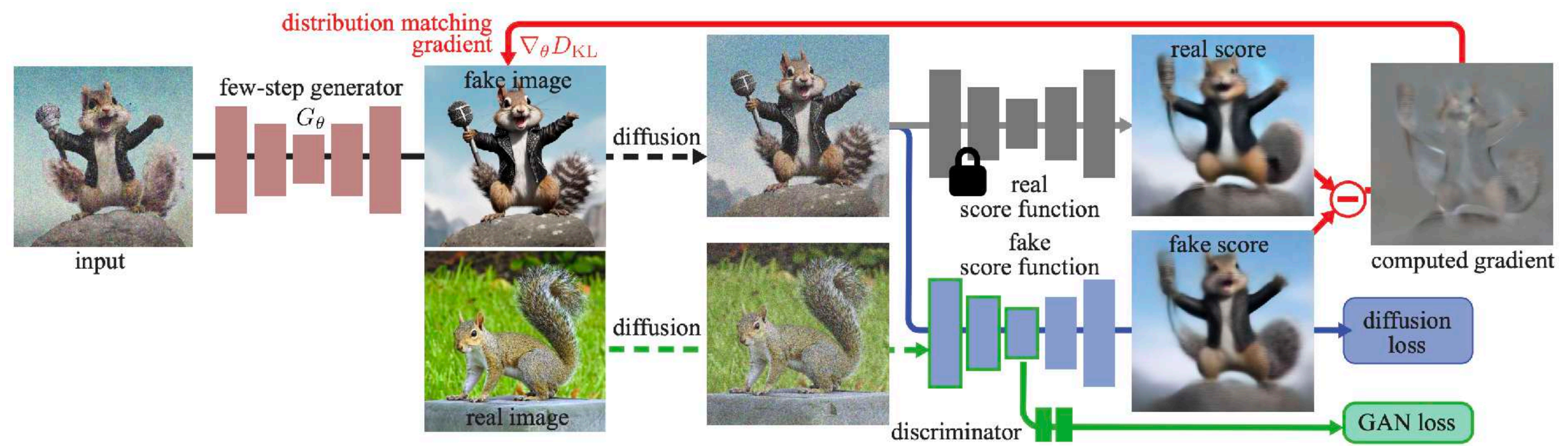
DMD

LADD



Improved DMD | DMD2

Let's combine GAN and DMD losses



Improved DMD | DMD2

DMD

- KD for stability and mode coverage
- Single step generator
- 1 update step for fake DM

DMD2

- GAN
- Multistep generator
- 5 update steps for fake DM (important)



DMD2 results

4-step DMD2-Qwen-Image



4-step DMD2-SDXL



Takeaways

DMD — Use DMs for reverse KL gradient estimation

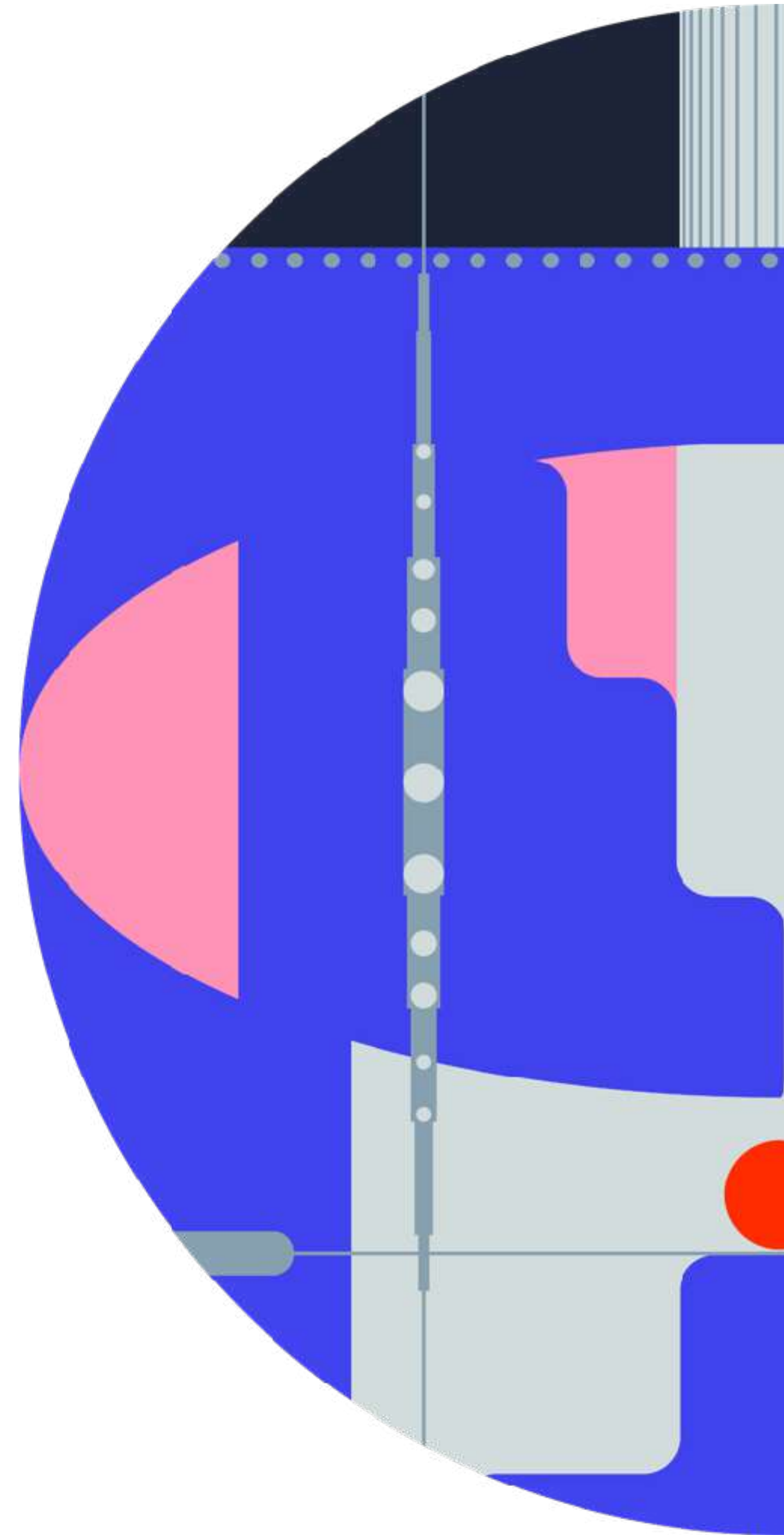
ADD, LADD — Diffusion-based GANs, where DM initialization plays a crucial role

DMD2 — DMD + GAN loss — **current state-of-the-art**

All of them trade diversity for image fidelity but that is what we are okay to trade in practice

All of them can outperform their teachers due to lower coverage

Maximum Mean Discrepancy Distillation



Motivation

Goal: effective and scalable few-step method

w/o adversarial objectives and extra trainable models

Better and cheaper than flow-map models

Maximum Mean Discrepancy

$$\text{MMD}^2(P, Q) = \mathbb{E}_{\mathbf{x}, \mathbf{x}' \sim P}[k(\mathbf{x}, \mathbf{x}')] + \mathbb{E}_{\mathbf{y}, \mathbf{y}' \sim Q}[k(\mathbf{y}, \mathbf{y}')] - 2 \mathbb{E}_{\mathbf{x} \sim P, \mathbf{y} \sim Q}[k(\mathbf{x}, \mathbf{y})]$$

$k(\cdot, \cdot)$ — kernel function

Linear $k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$

Gaussian (RBF) $k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}\right)$

Energy $k(\mathbf{x}, \mathbf{y}) = -\|\mathbf{x} - \mathbf{y}\| \Rightarrow \text{MMD}^2 = \text{ED}$ — Energy Distance

In practice, simply a linear combination of all possible pairwise sample distances

Problem with MMD-based models

Problem

Images are high dimensional



curse of dimensionality



Requires too many samples for accurate estimation

Problem with MMD-based models

Problem

Images are high dimensional



curse of dimensionality



Requires too many samples for accurate estimation

Solution

Work in lower dimensional **feature space**

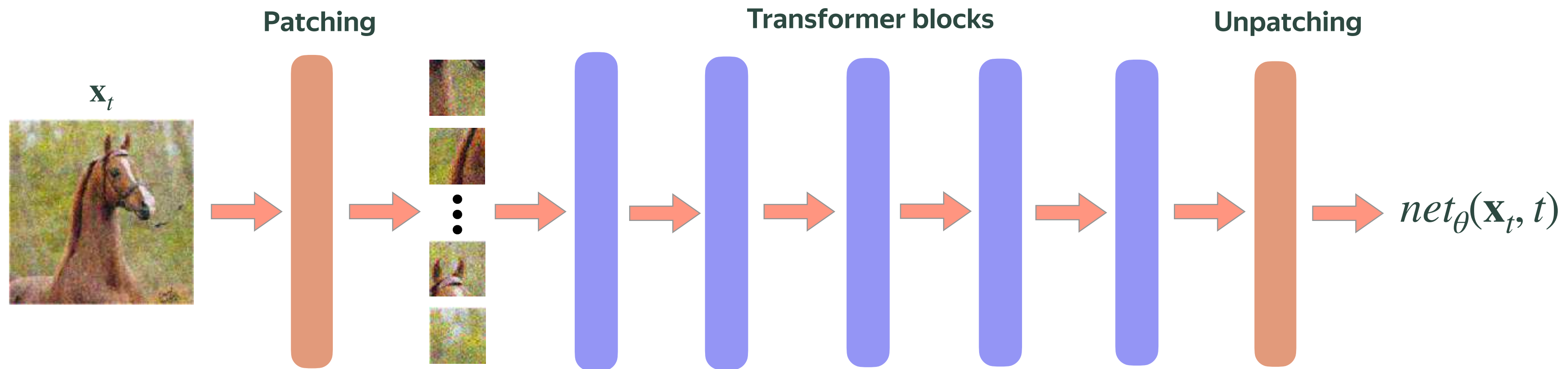
Image distribution → **patch-level** distribution



Much more lower dimension samples for free

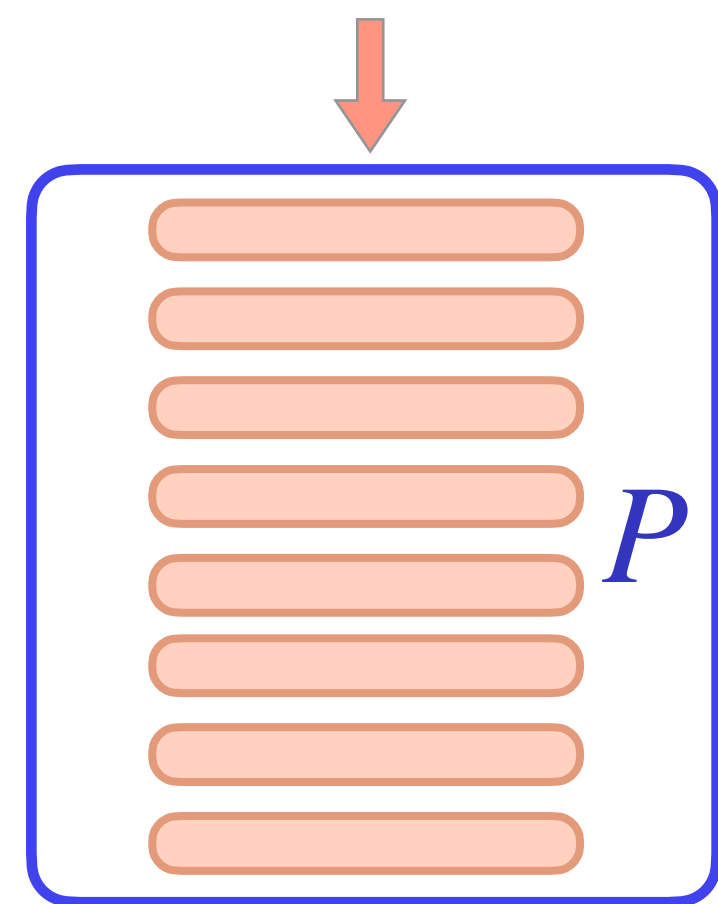
For example: 1 image → 256 patches

Diffusion models for feature extraction



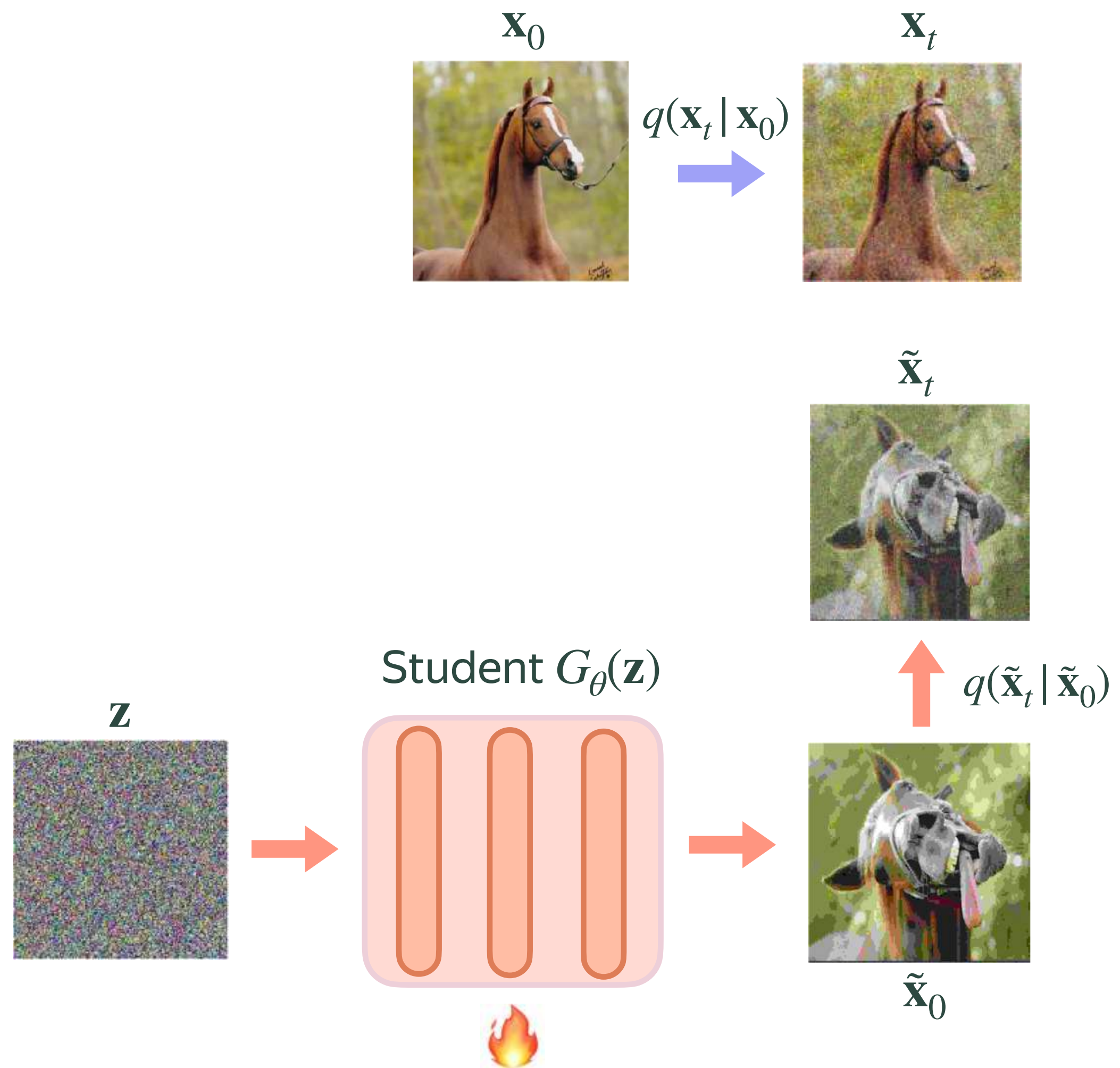
DiT feature tokens are patch representations

P.S. similar for the UNet models but not in patch terms

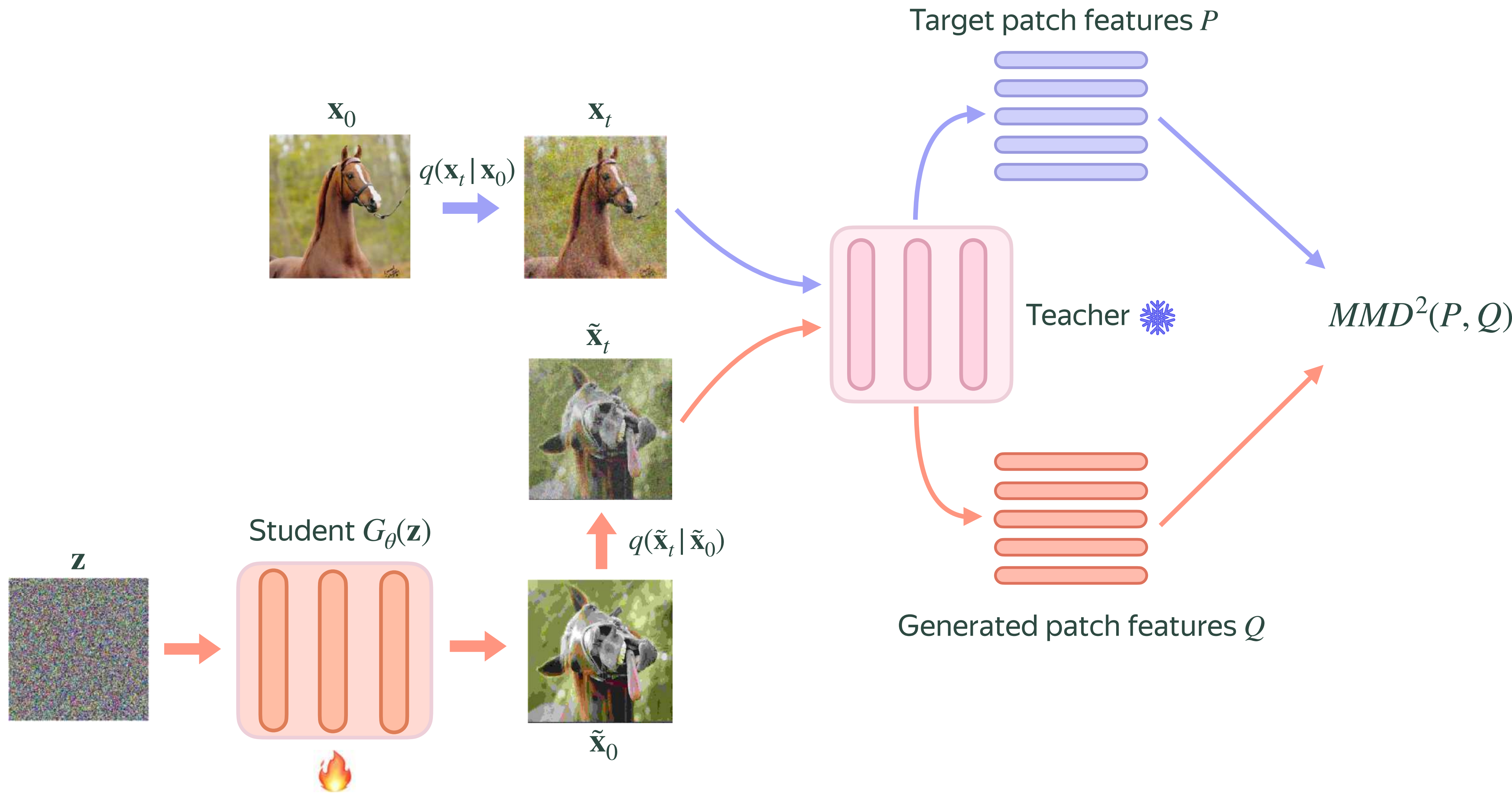


Distribution of patch-level image representations

MMD Distillation



MMD Distillation



MMD is a strong standalone objective

DMD2 + MMD loss

MMD loss only



Only slight degradation in defects compared to DMD2

Doesn't require discriminators or fake DMs

Stable & 7x faster training

Future work: make MMD a self-contained loss

Drifting Models



Drifting Models

Let's define **drifting fields** $\mathbf{V}_{p,q}(\cdot)$

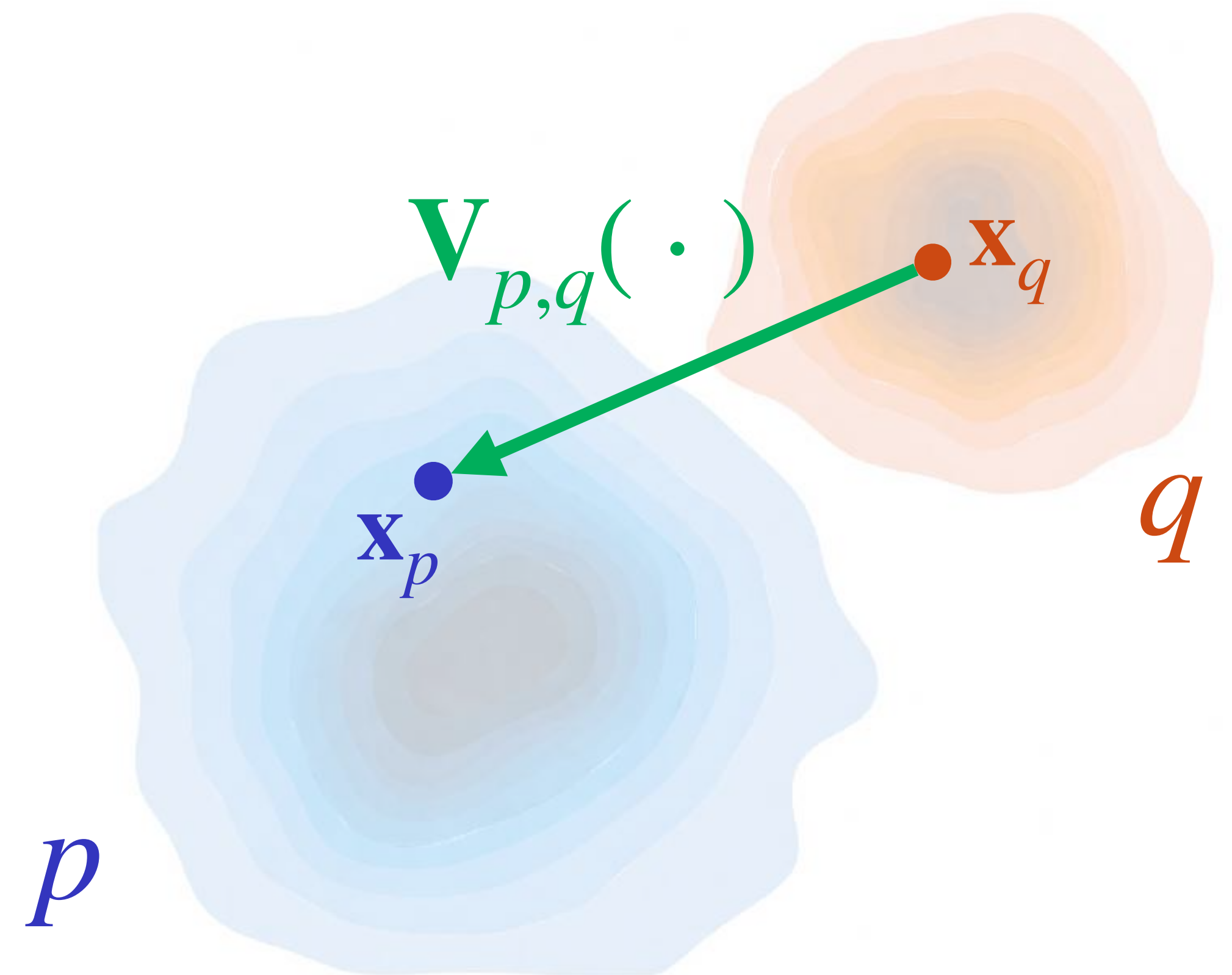
p — data distribution

q — generated distribution

$\mathbf{V}_{p,q}(\cdot)$ — drifting direction from $\mathbf{x}_q \rightarrow \mathbf{x}_p$

Equilibrium condition

$$q = p \Rightarrow \mathbf{V}_{p,q}(\mathbf{x}) = 0, \forall \mathbf{x}$$

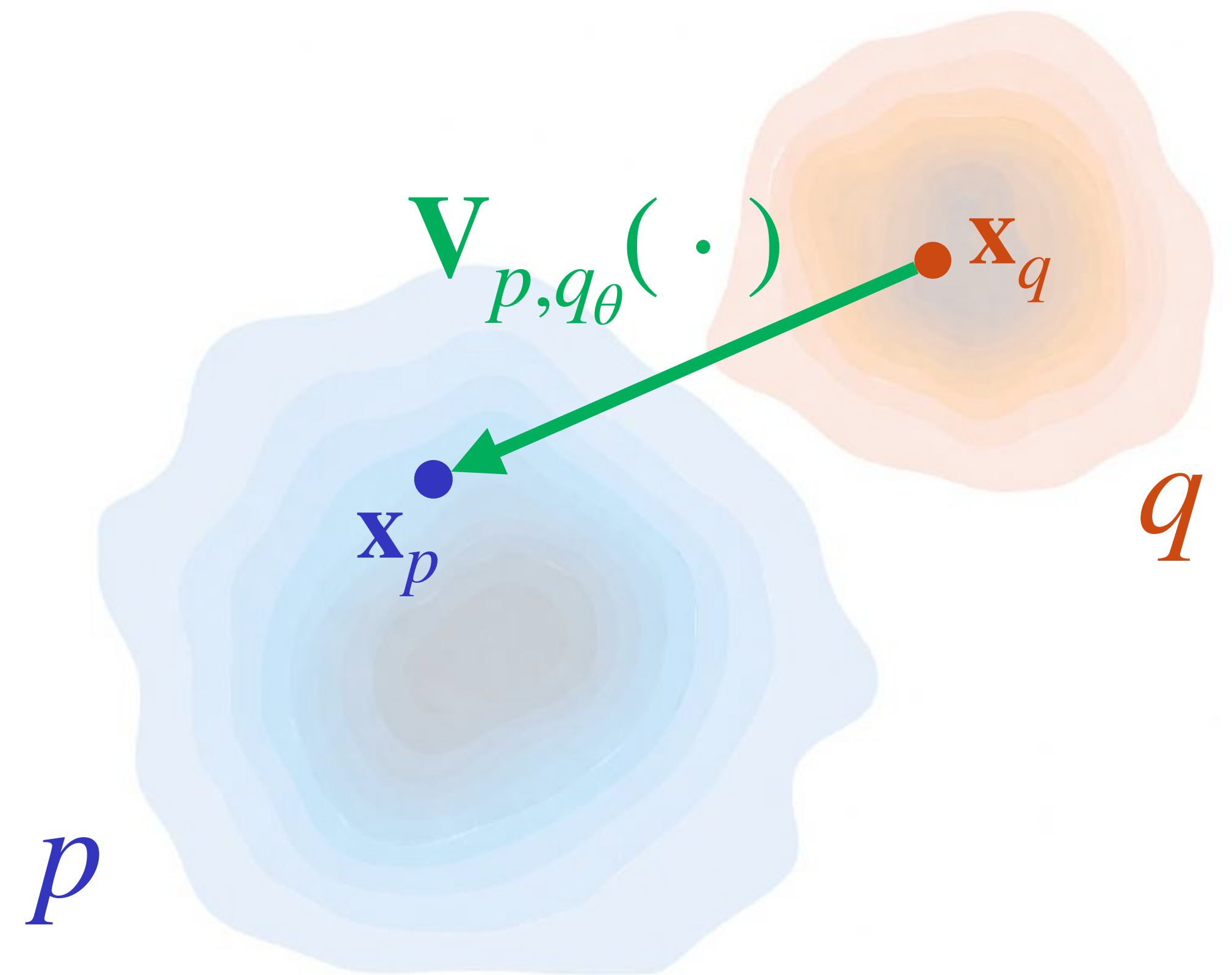


Drifting Models

Train a generator $G_{\theta}(\mathbf{z}) \rightarrow \mathbf{x}$

$$G_{\theta_{i+1}}(\mathbf{z}) \leftarrow G_{\theta_i}(\mathbf{z}) + \mathbf{V}_{p,q_{\theta_i}}\left(G_{\theta_i}(\mathbf{z})\right)$$

$\mathbf{V}_{p,q_{\theta_i}}\left(G_{\theta_i}(\mathbf{z})\right)$ — “gradient” direction to real distribution



Drifting Models

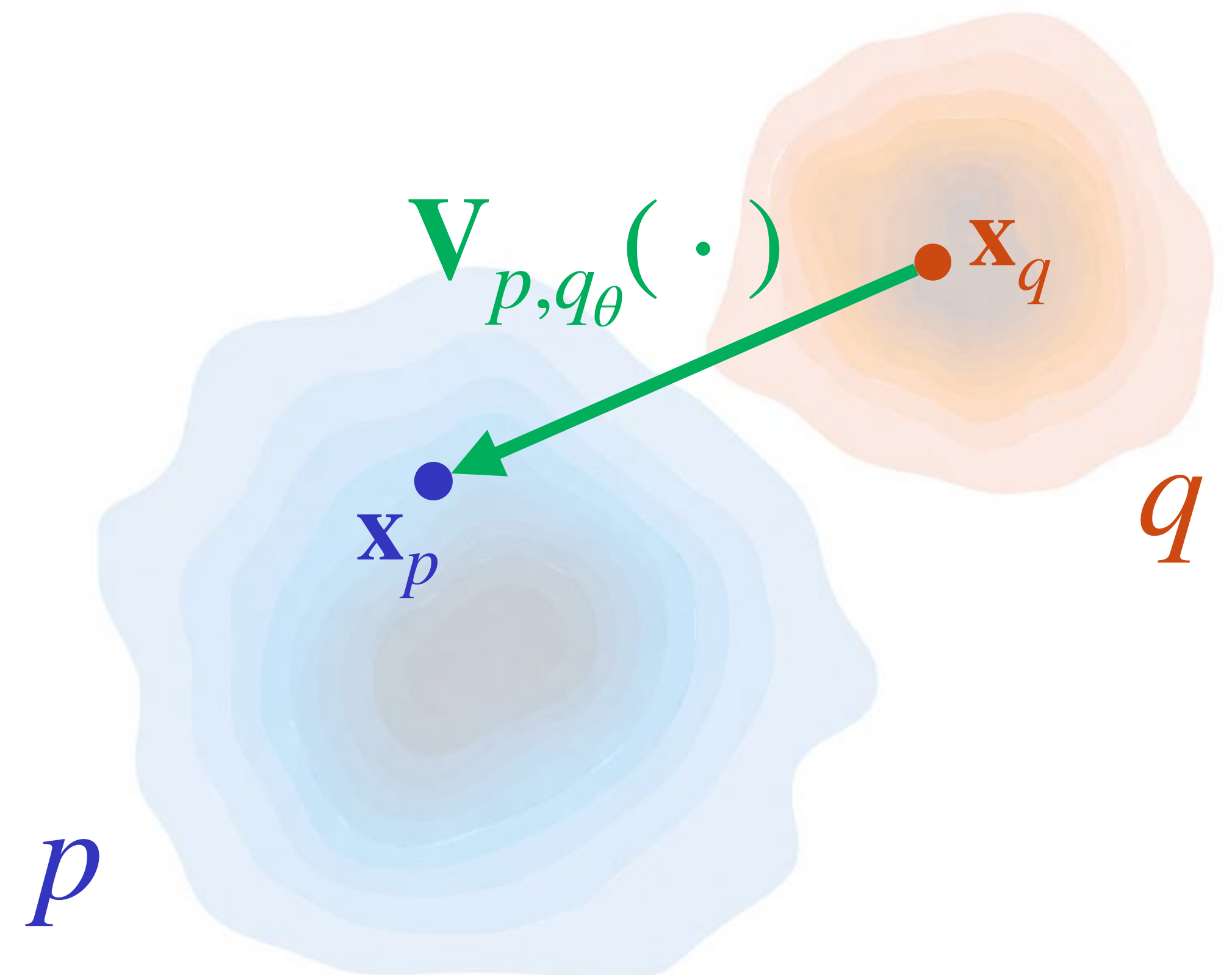
Train a generator $G_{\theta}(\mathbf{z}) \rightarrow \mathbf{x}$

$$G_{\theta_{i+1}}(\mathbf{z}) \leftarrow G_{\theta_i}(\mathbf{z}) + \mathbf{V}_{p,q_{\theta_i}}\left(G_{\theta_i}(\mathbf{z})\right)$$

$\mathbf{V}_{p,q_{\theta_i}}\left(G_{\theta_i}(\mathbf{z})\right)$ —“gradient” direction to real distribution

$$L = \mathbb{E}_{\mathbf{z}} \left[\left\| G_{\theta}(\mathbf{z}) - \text{sg}\left(G_{\theta}(\mathbf{z}) + \mathbf{V}_{p,q_{\theta}}\left(G_{\theta}(\mathbf{z})\right)\right) \right\|^2 \right]$$

Hm, looks familiar...



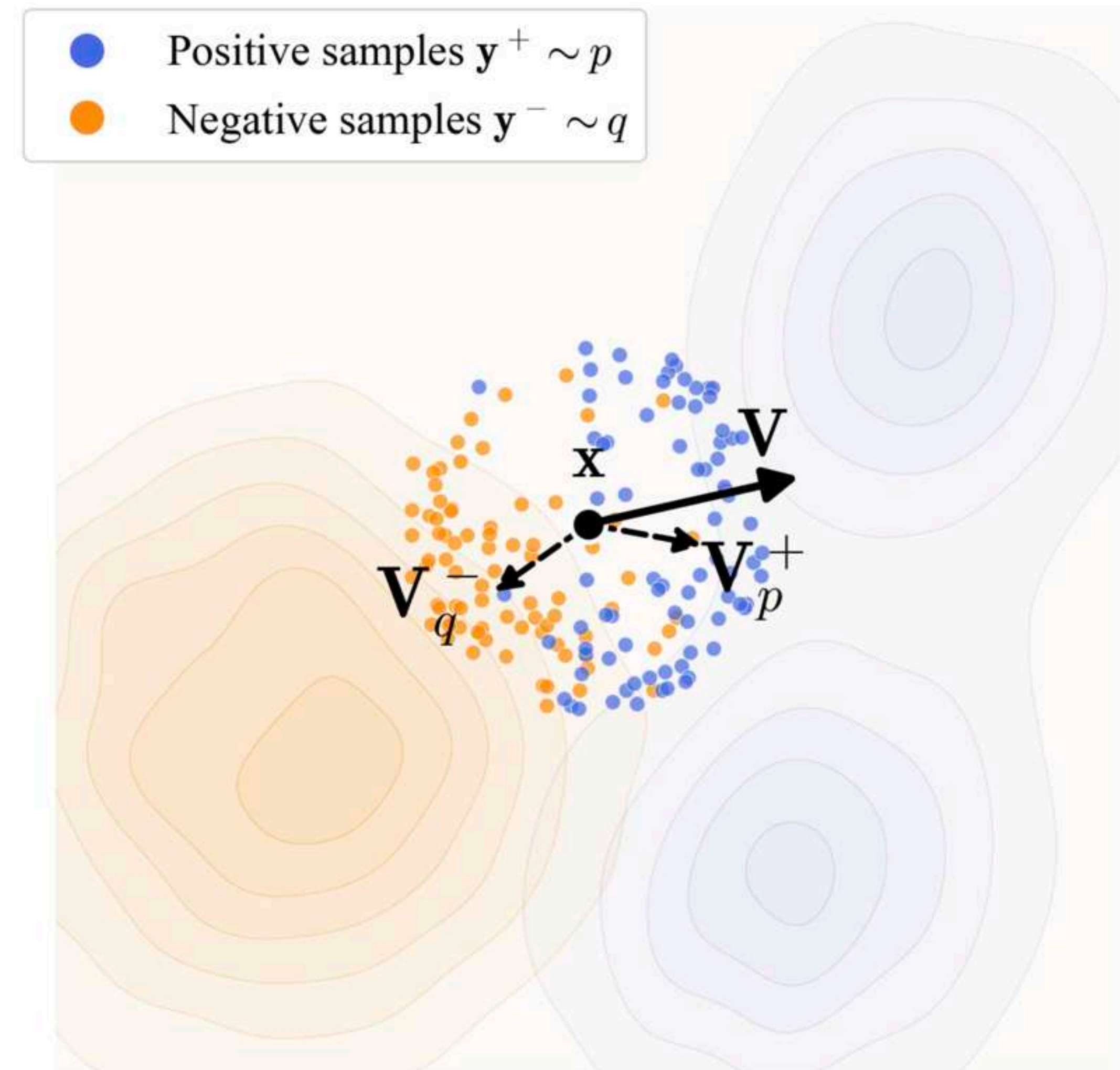
Instantiation of Drifting Models

The proposed variant of the drifting field in the paper

$$\mathbf{V}_{p,q}(\mathbf{x}) = \mathbf{V}_p^+(\mathbf{x}) - \mathbf{V}_q^-(\mathbf{x})$$

$\mathbf{V}_p^+(\mathbf{x})$ — local mean of real samples $\mathbf{y}^+$ (positives)

$\mathbf{V}_q^-(\mathbf{x})$ — local mean of fake samples $\mathbf{y}^-$ (negatives)



Drifting: a Gaussian-kernel variant

For the score connection, use Gaussian kernels and normalization over y :

$$\begin{aligned} \mathbf{V}_p^+(\mathbf{x}) &:= \frac{1}{Z_p(\mathbf{x})} \mathbb{E}_p \left[k(\mathbf{x}, \mathbf{y}^+) (\mathbf{y}^+ - \mathbf{x}) \right], & Z_p(\mathbf{x}) &:= \mathbb{E}_p \left[k(\mathbf{x}, \mathbf{y}^+) \right] \\ \mathbf{V}_q^-(\mathbf{x}) &:= \frac{1}{Z_q(\mathbf{x})} \mathbb{E}_q \left[k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^- - \mathbf{x}) \right], & Z_q(\mathbf{x}) &:= \mathbb{E}_q \left[k(\mathbf{x}, \mathbf{y}^-) \right] \end{aligned}$$

$k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2/(2\sigma^2)}$
Gaussian kernel

The paper uses $\exp(-\|\mathbf{x}-\mathbf{y}\| / \tau)$; its implementation normalizes over both sample axes.

Drifting: a Gaussian-kernel variant

For the score connection, use Gaussian kernels and normalization over y :

$$\begin{aligned} \mathbf{V}_p^+(\mathbf{x}) &:= \frac{1}{Z_p(\mathbf{x})} \mathbb{E}_p \left[k(\mathbf{x}, \mathbf{y}^+) (\mathbf{y}^+ - \mathbf{x}) \right], & Z_p(\mathbf{x}) &:= \mathbb{E}_p \left[k(\mathbf{x}, \mathbf{y}^+) \right] \\ \mathbf{V}_q^-(\mathbf{x}) &:= \frac{1}{Z_q(\mathbf{x})} \mathbb{E}_q \left[k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^- - \mathbf{x}) \right], & Z_q(\mathbf{x}) &:= \mathbb{E}_q \left[k(\mathbf{x}, \mathbf{y}^-) \right] \end{aligned}$$

$k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2/(2\sigma^2)}$
Gaussian kernel



Some magic algebra

$$V_{p,q}(\mathbf{x}) = \mathbf{V}_p^+(\mathbf{x}) - \mathbf{V}_q^-(\mathbf{x}) = \frac{1}{Z_p Z_q} \mathbb{E}_{p,q} \left[k(\mathbf{x}, \mathbf{y}^+) k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^+ - \mathbf{y}^-) \right]$$

The paper uses $\exp(-\|\mathbf{x}-\mathbf{y}\| / \tau)$; its implementation normalizes over both sample axes.

Gaussian-kernel connection to DMD

For this Gaussian variant, the drift is a difference of ideal denoisers.

Ideal denoiser:
$$D^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i, \sigma^2 \mathbf{I}) \mathbf{y}_i}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i, \sigma^2 \mathbf{I})}$$



Gaussian-kernel connection to DMD

$$k(\mathbf{x}, \mathbf{y}) := \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma^2 I) \propto e^{-\|\mathbf{x}-\mathbf{y}\|^2/(2\sigma^2)}$$

$$k(\mathbf{x}, \mathbf{y}^+) = \mathcal{N}(\mathbf{x}; \mathbf{y}^+, \sigma^2 \mathbf{I})$$

$$k(\mathbf{x}, \mathbf{y}^-) = \mathcal{N}(\mathbf{x}; \mathbf{y}^-, \sigma^2 \mathbf{I})$$

$$Z_p(\mathbf{x}) = \mathbb{E}_p[k(\mathbf{x}, \mathbf{y}^+)] = \frac{1}{N_+} \sum_{i=1}^{N_+} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 I)$$

$$Z_q(\mathbf{x}) = \mathbb{E}_q[k(\mathbf{x}, \mathbf{y}^-)] = \frac{1}{N_-} \sum_{i=1}^{N_-} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 I)$$

Expectations are over the empirical positive and negative sample distributions.

Gaussian-kernel connection to DMD

Expectations are over the empirical positive and negative sample distributions.

$$k(\mathbf{x}, \mathbf{y}^+) = \mathcal{N}(\mathbf{x}; \mathbf{y}^+, \sigma^2 \mathbf{I})$$

$$k(\mathbf{x}, \mathbf{y}^-) = \mathcal{N}(\mathbf{x}; \mathbf{y}^-, \sigma^2 \mathbf{I})$$

$$Z_p(\mathbf{x}) = \mathbb{E}_p[k(\mathbf{x}, \mathbf{y}^+)] = \frac{1}{N_+} \sum_{i=1}^{N_+} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})$$

$$Z_q(\mathbf{x}) = \mathbb{E}_q[k(\mathbf{x}, \mathbf{y}^-)] = \frac{1}{N_-} \sum_{i=1}^{N_-} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})$$

$$D_{real}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I}) \mathbf{y}_i^+}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})}$$

$$D_{fake}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I}) \mathbf{y}_i^-}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})}$$

Gaussian-kernel connection to DMD

Expectations are over the empirical positive and negative sample distributions.

$$k(\mathbf{x}, \mathbf{y}^+) = \mathcal{N}(\mathbf{x}; \mathbf{y}^+, \sigma^2 \mathbf{I})$$

$$k(\mathbf{x}, \mathbf{y}^-) = \mathcal{N}(\mathbf{x}; \mathbf{y}^-, \sigma^2 \mathbf{I})$$

$$Z_p(\mathbf{x}) = \mathbb{E}_p[k(\mathbf{x}, \mathbf{y}^+)] = \frac{1}{N_+} \sum_{i=1}^{N_+} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})$$
$$Z_q(\mathbf{x}) = \mathbb{E}_q[k(\mathbf{x}, \mathbf{y}^-)] = \frac{1}{N_-} \sum_{i=1}^{N_-} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})$$

$$D_{real}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I}) \mathbf{y}_i^+}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})} = \frac{\sum_i k(\mathbf{x}, \mathbf{y}_i^+) \mathbf{y}_i^+}{N_+ Z_p(\mathbf{x})}$$

$$D_{fake}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I}) \mathbf{y}_i^-}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})} = \frac{\sum_i k(\mathbf{x}, \mathbf{y}_i^-) \mathbf{y}_i^-}{N_- Z_q(\mathbf{x})}$$

Gaussian-kernel connection to DMD

Expectations are over the empirical positive and negative sample distributions.

$$k(\mathbf{x}, \mathbf{y}^+) = \mathcal{N}(\mathbf{x}; \mathbf{y}^+, \sigma^2 \mathbf{I})$$

$$k(\mathbf{x}, \mathbf{y}^-) = \mathcal{N}(\mathbf{x}; \mathbf{y}^-, \sigma^2 \mathbf{I})$$

$$Z_p(\mathbf{x}) = \mathbb{E}_p[k(\mathbf{x}, \mathbf{y}^+)] = \frac{1}{N_+} \sum_{i=1}^{N_+} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})$$

$$Z_q(\mathbf{x}) = \mathbb{E}_q[k(\mathbf{x}, \mathbf{y}^-)] = \frac{1}{N_-} \sum_{i=1}^{N_-} \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})$$

$$D_{real}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I}) \mathbf{y}_i^+}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^+, \sigma^2 \mathbf{I})} = \frac{\sum_i k(\mathbf{x}, \mathbf{y}_i^+) \mathbf{y}_i^+}{N_+ Z_p(\mathbf{x})} = \frac{\mathbb{E}_p[k(\mathbf{x}, \mathbf{y}^+) \mathbf{y}^+]}{Z_p(\mathbf{x})}$$

$$D_{fake}^*(\mathbf{x}; \sigma) = \frac{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I}) \mathbf{y}_i^-}{\sum_i \mathcal{N}(\mathbf{x}; \mathbf{y}_i^-, \sigma^2 \mathbf{I})} = \frac{\sum_i k(\mathbf{x}, \mathbf{y}_i^-) \mathbf{y}_i^-}{N_- Z_q(\mathbf{x})} = \frac{\mathbb{E}_q[k(\mathbf{x}, \mathbf{y}^-) \mathbf{y}^-]}{Z_q(\mathbf{x})}$$

Gaussian case: objective manipulations

$$V_{p,q}(\mathbf{x}) = \frac{1}{Z_p Z_q} \mathbb{E}_{p,q} \left[k(\mathbf{x}, \mathbf{y}^+) k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^+ - \mathbf{y}^-) \right] =$$

Gaussian case: objective manipulations

$$\begin{aligned} V_{p,q}(\mathbf{x}) &= \frac{1}{Z_p Z_q} \mathbb{E}_{p,q} [k(\mathbf{x}, \mathbf{y}^+) k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^+ - \mathbf{y}^-)] = \\ &= \frac{1}{Z_q} \mathbb{E}_q \left[\underbrace{\mathbb{E}_p \left[\frac{k(\mathbf{x}, \mathbf{y}^+) \mathbf{y}^+}{Z_p} \right]}_{D_{real}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^-) \right] - \frac{1}{Z_p} \mathbb{E}_p \left[\underbrace{\mathbb{E}_q \left[\frac{k(\mathbf{x}, \mathbf{y}^-) \mathbf{y}^-}{Z_q} \right]}_{D_{fake}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^+) \right] = \end{aligned}$$

Gaussian case: objective manipulations

$$\begin{aligned} V_{p,q}(\mathbf{x}) &= \frac{1}{Z_p Z_q} \mathbb{E}_{p,q} [k(\mathbf{x}, \mathbf{y}^+) k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^+ - \mathbf{y}^-)] = \\ &= \frac{1}{Z_q} \mathbb{E}_q \left[\underbrace{\mathbb{E}_p \left[\frac{k(\mathbf{x}, \mathbf{y}^+) \mathbf{y}^+}{Z_p} \right]}_{D_{real}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^-) \right] - \frac{1}{Z_p} \mathbb{E}_p \left[\underbrace{\mathbb{E}_q \left[\frac{k(\mathbf{x}, \mathbf{y}^-) \mathbf{y}^-}{Z_q} \right]}_{D_{fake}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^+) \right] = \\ &= \frac{D_{real}^*(\mathbf{x}, \sigma)}{Z_q} \underbrace{\mathbb{E}_q [k(\mathbf{x}, \mathbf{y}^-)]}_{Z_q} - \frac{D_{fake}^*(\mathbf{x}, \sigma)}{Z_p} \underbrace{\mathbb{E}_p [k(\mathbf{x}, \mathbf{y}^+)]}_{Z_p} = \end{aligned}$$

Gaussian case: objective manipulations

$$\begin{aligned}
 V_{p,q}(\mathbf{x}) &= \frac{1}{Z_p Z_q} \mathbb{E}_{p,q} [k(\mathbf{x}, \mathbf{y}^+) k(\mathbf{x}, \mathbf{y}^-) (\mathbf{y}^+ - \mathbf{y}^-)] = \\
 &= \frac{1}{Z_q} \mathbb{E}_q \left[\underbrace{\mathbb{E}_p \left[\frac{k(\mathbf{x}, \mathbf{y}^+) \mathbf{y}^+}{Z_p} \right]}_{D_{real}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^-) \right] - \frac{1}{Z_p} \mathbb{E}_p \left[\underbrace{\mathbb{E}_q \left[\frac{k(\mathbf{x}, \mathbf{y}^-) \mathbf{y}^-}{Z_q} \right]}_{D_{fake}^*(\mathbf{x}, \sigma)} k(\mathbf{x}, \mathbf{y}^+) \right] = \\
 &= \frac{D_{real}^*(\mathbf{x}, \sigma)}{Z_q} \underbrace{\mathbb{E}_q [k(\mathbf{x}, \mathbf{y}^-)]}_{Z_q} - \frac{D_{fake}^*(\mathbf{x}, \sigma)}{Z_p} \underbrace{\mathbb{E}_p [k(\mathbf{x}, \mathbf{y}^+)]}_{Z_p} = \boxed{D_{real}^*(\mathbf{x}, \sigma) - D_{fake}^*(\mathbf{x}, \sigma)} \\
 &\quad \text{Ideal-denoiser difference at fixed noise level}
 \end{aligned}$$

Summary

Gaussian-kernel interpretation

For Gaussian kernels, normalized mean shifts give a difference of smoothed scores.
This explains a connection to DMD under the stated kernel and normalization choices.

Summary

Gaussian-kernel interpretation

For Gaussian kernels, normalized mean shifts give a difference of smoothed scores. This explains a connection to DMD under the stated kernel and normalization choices.

Practical considerations for Drifting models:

- 1) In the feature space of some external encoder (e.g., DINOv3)
- 2) Requires large batch sizes to be effective even on ImageNet

Summary

Gaussian-kernel interpretation

For Gaussian kernels, normalized mean shifts give a difference of smoothed scores. This explains a connection to DMD under the stated kernel and normalization choices.

Practical considerations for Drifting models:

- 1) In the feature space of some external encoder (e.g., DINOv3)
- 2) Requires large batch sizes to be effective even on ImageNet

Drifting models are similar to the MMD distillation but

- 1) Drifting uses multi-scale features: per-location vectors and local/global pooled statistics.
- 2) MMD distillation uses a teacher DM as a feature extractor

Summary

Drifting Models in general

Very general and productive perspective on **all** implicit generative models: GANs, DMD, MMD, etc

